

笔记作者：喂喂薇（小红书）
仅供学习交流，禁止商用

cs336 动手学大模型

Stanford CS336 | Language Modeling from Scratch

小红书同名作者：喂喂薇

目录

第一章 Introduction	9
1.1 列表样式类	9
1.1.1 无序列表	9
1.1.2 有序列表	9
1.1.3 多样式列表	9
1.2 自定义主题框类	10
1.3 图片样式	11
第二章 Assignment 1 基础篇:Building a Transformer LM	13
2.1 Assignment 1 Overview	13
2.1.1 每一部分的实验任务	13
2.1.2 准备工作	13
2.2 Byte-pair encoding (BPE) tokenizer	14
2.2.1 Unicode 字符集	14
2.2.2 subword tokenizer(子词分词器)	19
2.3 BPE Tokenizer Training	21
2.3.1 初始化词汇表	22
2.3.2 预分词	22
2.3.3 BPE 合并	23
2.3.4 特殊标记	24
2.3.5 BPE 分词器训练示例	24
2.4 BPE 分词器训练实操	24
2.5 利用 BPE 分词器进行编码和解码	26
2.5.1 编码	26
2.5.2 解码	28
2.6 BPE 讲义相关 Problems	29
2.7 BPE 章节实验	29
第三章 Transformer Language Model Architecture(Transformer 架构)	31
3.1 基础模块: 线性层和嵌入层	32
3.1.1 参数初始化	32
3.1.2 实验中参数初始化标准	33

2 目录

3.2	词嵌入层 (embedding layer)	33
3.3	两大基础模块: 线性模块和嵌入模块	33
3.3.1	线性模块 (Linear Module)	33
3.3.2	嵌入模块 (Embedding Module)	34
3.4	Pre-norm Transformer Block	34
3.4.1	RMSNorm 的原理	35
3.4.2	位置级前馈网络 (Position-wise Feed-Forward Network)	37
3.4.3	相对位置嵌入 (RoPE 旋转位置编码)	40
3.4.4	缩放点积注意力 scaled dot-product attention	46
3.4.5	多头因果注意力机制	47
3.5	Transformer 总结篇	47
3.5.1	输入部分	48
3.5.2	BPE tokenizer(预分词)	48
3.5.3	Embedding (词嵌入)	48
3.5.4	Position encoding (位置编码)	49
3.5.5	Transformer block	49
3.5.6	RMSnorm	49
3.5.7	Causal Multi-head self-attention (多头注意力机制)	49
3.5.8	SwiGLU	49
3.5.9	最终输出	50
第四章	Transformer 模型的训练	51
4.1	信息论前置知识 (简易版)	51
4.1.1	信息 (Information)	51
4.1.2	熵 (Entropy)	52
4.1.3	熵的相关扩展概念	52
4.1.4	一个生动的比喻: 猜数字游戏	54
4.2	交叉熵 (Cross-Entropy) 损失与 KL 散度	55
4.2.1	KL 散度 (相对熵)	55
4.2.2	交叉熵	55
4.2.3	交叉熵损失函数	55
4.2.4	实际代码的应用	56
4.2.5	具体在 Transformer 中的应用	57
4.2.6	Perplexity(困惑度)	58
4.3	Optimizer: 优化器	59
4.3.1	SGD (随机梯度下降)	59
4.3.2	动量法 (Momentum)	60
4.3.3	AdaGrad	61
4.3.4	RMSProp	62
4.3.5	Adam	63

4.3.6	AdamW	64
4.4	自适应学习率调整 (学习率调度器)	64
4.4.1	StepLR (阶梯式下降)	65
4.4.2	CosineAnnealingLR (余弦退火)	66
4.4.3	ReduceLROnPlateau (遇到平台期就减速)	66
4.4.4	Warmup (预热)	68
4.5	梯度裁剪	69
第五章	泛深度学习训练流程	71
5.1	数据集加载 (DataLoader)	71
5.1.1	数据增强的两种方式	71
5.1.2	自定义 Dataset	73
5.1.3	使用 DataLoader 加载数据	73
5.1.4	如果数据集过大无法载入内存怎么办?	74
5.1.5	内存映射	74
5.2	模型保存 (Checkpoint)	76
5.2.1	为什么 Checkpoint 至关重要?	76
5.2.2	一个 Checkpoint 文件里通常包含什么?	77
5.3	脚本化训练	78
5.4	消融实验 Ablation Studies	79
第六章	推理部分	81
6.1	Softmax 层与解码	81
6.2	温度缩放 (Temperature Scaling)	82
6.3	Top-p 采样	83
第七章	Assignment 2 系统篇:Systems and Parallelism	85
7.1	Assignment 2 Overview	85
第八章	性能分析与基准测试 Profiling and Benchmarking	87
8.1	端到端简单性能评估测试	87
8.1.1	Slurm 平台上的 submitit 工具简要介绍	87
8.1.2	前向和反向传播性能评估	89
8.2	Nsight Systems Profiler	90
8.2.1	nsys 简介	90
8.2.2	nvtx	90
8.3	混合精度训练	93
8.3.1	浮点数知识回顾	94
8.3.2	混合精度训练的实现	96
8.4	显存分析	97

第九章 FlashAttention-2 优化	101
9.1 GPU 内存架构	101
9.1.1 物理内存架构：片上内存与片下内存	101
9.1.2 逻辑内存层次与功能划分	102
9.2 FlashAttention	103
9.2.1 GPU 的 FLOPs 计算能力与内存吞吐能力	104
9.2.2 I/O 感知	104
9.2.3 FlashAttention 如何充分考虑 GPU 的内存层级结构？	104
9.2.4 实现细节	106
9.2.5 Flash Attention 的计算量分析	109
9.3 FlashAttention2 与 FlashAttention1 的区别	110
9.3.1 FlashAttentionv3	112
9.4 Triton	112
9.4.1 Triton 的核心思想：并行化与分块 (Parallelism and Tiling)	113
9.4.2 Triton 的”分块”特性	114
9.4.3 Triton 和 C/C++ 的编译方式的区别和联系	115
9.5 用 Triton 优化 FlashAttention	116
9.5.1 前向传播	116
9.5.2 反向传播	118
第十章 分布式并行训练 (Distributed Data Parallel Training)	121
10.1 单机多进程	121
10.2 AllReduce 算法	123
10.2.1 分布式通信算法	126
10.3 DDP	130
10.3.1 DDP 和 DP 的区别	130
10.3.2 DDP 算法优化	131
10.4 混合并行 (4D 并行)	135
10.4.1 每种并行技术简介	135
10.5 混合数据并行	143
10.5.1 DP+PP(数据并行 + 流水线并行)	143
10.5.2 3D 并行 (DP+PP+TP)	144
10.5.3 4D 并行 (DP+PP+TP+ZeRO)	146
10.5.4 专家并行/MOE 并行	148
10.6 CUDA 通信机制知识补充	148
10.6.1 主机与设备 (Host-Device) 通信	150
10.6.2 设备内部 (Intra-Device) 通信	151
第十一章 Appendix	153
11.1 拓展链接	153
11.2 经典反例	154

2

Assignment 1 基础篇:Building a Transformer LM

2.1. Assignment 1 Overview

参考资料 2.1

- uv 官方文档:<https://docs.astral.sh/uv/>
- uv 中文基础使用教程: <https://www.runoob.com/python3/uv-tutorial.html>
- 服务器租借平台: <https://www.autodl.com/>(平台不止这一个, 我感觉这个比较方便)

2.1.1. 每一部分的实验任务

1. BPE 分词器 (Byte-pair encoding (BPE) tokenizer)
2. Transformer 语言模型 (Transformer language model (LM))
3. 交叉熵损失函数与 AdamW 优化器
4. 完整实现模型训练

2.1.2. 准备工作

1. 准备一台配备 GPU 和 Linux 系统的电脑;
2. 下载代码仓库;
Assignment 1 仓库地址: github.com/stanford-cs336/assignment1-basics
3. 下载数据集:
TinyStory: huggingface.co/datasets/roneneldan/TinyStories/resolve/main/
OpenWebText: skylion007.github.io/OpenWebTextCorpus/
4. 学习了解 uv 的基础使用方法

微言大义 2.1

cs336 中的实验代码是 uv 管理的, 因此需要学习了解 uv 的基础使用方法 (uv 是比较流行的现代 python 项目管理工具, 学了不亏 ×)。

另外 cs336 的 5 个 Assignment 的目录格式基本都是一个 basics/, 一个 tests/, 其中 basics/ 目录下可以用来写自己的实验代码 (当然也可以写作别的地方), tests/ 目录下是测试代码。此外 tests/ 目录下的 adapter.py 是测试文件的

2.1	Assignment 1	
	Overview	13
2.1.1	每一部分的实验任务	13
2.1.2	准备工作	13
2.2	Byte-pair encoding	
	(BPE) tokenizer	14
2.2.1	Unicode 字符集	14
2.2.2	subword tokenizer(子词分词器)	19
2.3	BPE Tokenizer	
	Training	21
2.3.1	初始化词汇表	22
2.3.2	预分词	22
2.3.3	BPE 合并	23
2.3.4	特殊标记	24
2.3.5	BPE 分词器训练示例	24
2.4	BPE 分词器训练实操	24
2.5	利用 BPE 分词器进行	
	编码和解码	26
2.5.1	编码	26
2.5.2	解码	28
2.6	BPE 讲义相关 Problems	29
2.7	BPE 章节实验	29

适配器, 类似一个接口, 作业文件需要实现这个接口才能被测试文件正确调用。

2.2. Byte-pair encoding (BPE) tokenizer

参考资料 2.2

UTF-8 编码详解-CSDN 博客: <https://blog.csdn.net/whahu1989/article/details/118314154>

UTF-8 编码原理及与 ASCII 的兼容性-CSDN 博客: https://blog.csdn.net/baidu_25299117/article/details/139633315

2.2 实验目标: 实现一个 BPE 分词器。对应测试文件: tests/test_tokenizer.py

2.2.1. Unicode 字符集

在计算机发展的早期, 不同国家和地区为了表示本国语言的字符, 分别制定了自己的编码系统, 例如: 美国的 **ASCII**, 中国的 **GB2312** 等。但这些编码系统互不兼容, 就很不方便。

而 Unicode(统一码) 是一种统一的字符集标准, 为世界上所有文字和符号分配了一个唯一的编号 (称为 “码点” code point)。比如汉字 “牛” 的码点是 “U+29275”, 其中 “U+” 是一个无意义前缀, 表示这是一个 Unicode 码点, “29275” 表示这个码点的十进制值。

Extension 2.1 Unicode 字符集代码实操

在 python 里, 我们可以使用 **ord()** 函数获取一个字符的码点, 使用 **chr()** 函数获取一个码点对应的字符。比如:

```
ord('牛')
>>> 29275
chr(29275)
>>> '牛'
```

Unicode 定义了 “**字符** ↔ **码点**” 的映射, 而 UTF(Unicode Transformation Format) 则进一步定义如何把 **码点** 编码为 **字节**。常见的 UTF 编码有 UTF-8, UTF-16, UTF-32 等。

UTF-8 编码

UTF-8 是一种 **变长编码**, 使用 **1~4 个字节** 表示。unicode 字符, 是互联网的主导编码格式 (占有所有网页的 98% 以上)。

变长编码也就是说不同的字符编码的结果长度不一定相同, 有的是 1 个字节, 有的是 2 个字节或 3、4 个字节。

UTF-8 编码和 Unicode 码点范围的对应关系:

表 2.1: UTF-8 编码规则对照表

字节数	UTF-8 字节序列 (二进制)	Unicode 码点范围 (十六进制)	Unicode 码点范围 (十进制)
1	0xxxxxxx	U+0000 至 U+007F	0~127 (7 bits)
2	110xxxxx 10xxxxxx	U+0080 至 U+07FF	128~2047 (11 bits)
3	1110xxxx 10xxxxxx 10xxxxxx	U+0800 至 U+FFFF	2048~65535 (16 bits)
4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	U+10000 至 U+10FFFF	65536~1114111 (21 bits)

所有存储的字节被分为两类:**领头字节**(Leading Byte) 和**后续字节**(Continuation Byte)。

1. 单字节字符 (ASCII 范围)

- **规则**: 如果一个字节的最高位 (第 1 位) 是 '0', 那么它就是一个单字节字符。
- **格式**: '0xxxxxxx'
- **解读**: 1 字节时, UTF-8 完全等价于 ASCII: ASCII 0x41(A) ↔ UTF-8 0x41, UTF-8 编码格式是 0xxxxxxx, 正好能容纳 ASCII 0 127。

2. 多字节字符

- **规则**: 如果一个字节的开头是 '1', 那它就是一个多字节字符的其中一部分 (**开头有几个连续的 1 就代表几个字节**)。
- **领头字节**:
 - '110xxxxx': 表示这是一个**双字节**字符的第一个字节。
 - '1110xxxx': 表示这是一个**三字节**字符的第一个字节。
 - '11110xxx': 表示这是一个**四字节**字符的第一个字节。
- **后续字节**:
 - '10xxxxxx': 所有非领头的后续字节, 都必须以 '10' 开头。

Example 2.1 (UTF-8 编码示例)

汉字“中”的 Unicode 码点是 U+4E2D。U+4E2D 在 U+0800 到 U+FFFF 之间, 根据规则, 它需要用 3 个字节来编码。

首先将 U+4E2D 转换为二进制:

4 → 0100

E → 1110

2 → 0010

D → 1101

所以, U+4E2D 的二进制是 0100 1110 0010 1101。(总共 16 位)

3 字节的 UTF-8 模板是: 1110xxxx 10xxxxxx 10xxxxxx, 将二进制填入模板中, 得到: 11100100 10111000 10001011。

所以, 汉字“中”的 UTF-8 编码是:11100100 10111000 10001011。

Extension 2.2 Unicode 编码实操

在 python 里, 对于 string 类型数据, 我们可以使用 encode() 方法将字符串编码为 UTF-8 编码, 使用 decode() 方法将 UTF-8 编码解码为字符串。比如:

```
test_string = "hello! こんにちは!"
utf8_encoded = test_string.encode("utf-8")
>>> b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
list(utf8_encoded)
>>> [104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227,
    ↪ 129, 161, 227, 129, 175, 33]
utf8_encoded.decode("utf-8")
>>> "hello! こんにちは!"
```

通过 UTF-8 编码将 Unicode 码点转换为字节序列, 我们本质上是在将码点序列 (0 到 154997 范围内的整数) 转换为字节值序列 (0 到 255 范围内的整数)。256 长度的字节词汇表处理起来要容易得多。使用字节级分词时, 我们无需担心词汇表外的标记, 因为我们知道**任何输入文本都可以表示为 0 到 255 的整数序列**。

overlong 编码

UTF-8 要求用**最短的字节数**编码每个字符。

比如 U+002F(十进制下为 47, 十六进制下为 0x2F, 表示符号为 “/”) 按照规则只能用 1 个字节编码, 但如果我们非要把它编码为 2 个字节 (0xC0 0xAF), 它解码出来:

- 0xC0 0xAF → 11000000 10101111
- 11000000 10101111 → 47 → U+002F

同样也是 “/”, 这就是**overlong 编码**。

overlong 编码不守 UTF8 的最小字节编码的规矩, 因此是被明确禁止的, 如果采用 overlong 编码有可能绕过防火墙啥的, 有很大**危险性**。

Unicode 相关 Problem

文档作者：喂喂薇

Problem 2.1

1. **chr(0) 返回什么 Unicode 字符？**

答: 返回空字符

2. **这个字符的字符串表示 (__repr__()) 与其打印表示有何不同？**

答: 它的打印表示通常是不可见的（没有视觉输出），而其字符串表示 __repr__() 是一个明确的转义序列 'x00'。

3. **当这个字符出现在文本中会发生什么？**

答: 会出现一个截断, 效果类似空格。

Problem 2.2

1. 选择在 UTF-8 编码字节而非 UTF-16 或 UTF-32 上训练分词器的原因有哪些？

答:1. UTF-8 是一种变长编码,对英文、数字和常用符号等只用 1 个字节表示,而像汉字等字符通常用 3 个字节。相比之下,UTF-16 至少需要 2 个字节,UTF-32 固定为 4 个字节。对于以英文为主的语料库,UTF-8 的存储和处理效率远高于后两者

2. 一个分词器的词汇表 (Vocabulary) 大小是有限的。如果直接在 Unicode 字符 (UTF-32) 上操作,遇到词汇表中没有的字符 (例如一个新的 Emoji 或一个罕见的汉字),就只能将其标记为<UNK>。而基于 UTF-8 字节的分词器,其基础词汇表是固定的 256 个字节 (从 0x00 到 0xFF)。任何未知的、罕见的字符,甚至是乱码,都可以被分解成一串已知的字节序列来表示。这样就从根本上消除了 <UNK> 符号,使得模型能够处理任何形式的文本输入,而不会丢失信息。

2. 为什么如下函数是错误的？

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):  
    return "".join([bytes([b]).decode("utf-8") for b in  
        ↪ bytestring])  
>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))  
'hello'
```

答:UTF-8 是一种变长编码,一个 Unicode 字符可能由 1 到 4 个字节组成。

对于标准的 ASCII 字符 (如'h', 'e', 'l', 'l', 'o'), 它们在 UTF-8 中确实只由单个字节表示,所以这个函数对纯 ASCII 字符串"hello".encode("utf-8") 能够侥幸成功。

但是,对于任何非 ASCII 字符,需要用多个字节表示,比如汉字“中”,它的 UTF-8 编码是 `b'\xe4\xb8\xad'`,由三个字节组成。当 `decode_utf8_bytes_to_str_wrong` 函数处理 `b'\xe4\x8b\xad'` 时,它会:取出第一个字节 `b'\xe4'`,并尝试执行 `bytes([b'\xe4']).decode("utf-8")`。0xe4 (二进制 11100100) 是一个多字节字符的“起始字节”,它告诉解码器“后面还跟着 2 个字节”。单独解码它必然会失败,因为它的序列不完整。

此时,Python 会抛出 `UnicodeDecodeError` 异常,因为遇到了一个不完整或无效的 UTF-8 序列。**正确的解码方式必须在完整的字节序列上进行,而不是逐字节地割裂进行。**

3. 提供一个双字节序列,该序列无法解码为任何 Unicode 字符

答: 一个双字节字符的起始字节的二进制格式必须是 110xxxxx 10xxxxxx,因此只要不满足这个格式,就不能解码。

微言大义 2.2

Unicode 尤其 UTF8 编码是后面训练 BPE 分词器的基础概念, 内容实际上就是信息论的延伸, 理解了会对很多方面都有帮助。

2.2.2. subword tokenizer(子词分词器)

参考资料 2.3

jieba 分词: https://blog.csdn.net/qq_33957603/article/details/124640588

核心知识:词级 (word-level) 分词器、字符级 (character-level) 分词器、字节级 (byte-level) 分词器、子词级 (subword-level) 分词器之间的区别和联系

词级 (word-level) 分词器

- 核心思想: 最符合人类直觉的方式, 直接将句子按照空格或标点符号切分成一个个独立的单词。对于中文等没有天然分隔符的语言, 则需要依赖特定的分词算法 (如 jieba 分词)。
- 工作流程:
 1. **预处理 (可选)**: 可能包括小写化、去除标点、处理特殊字符等。
 2. **切分**: 主要根据空格和标点符号将句子切分成单词。例如, "Hello, world!" 可能会被切分为 ["Hello", ",", "world", "!"] 或 ["Hello", "world"] (如果标点被移除或单独处理)。
 3. **词汇表构建**:
 - 在训练数据上统计所有出现的词语及其**频率**。
 - 选择频率最高的 N 个词语构成词汇表 (vocabulary)。
 - 词汇表之外的词语 (未登录词, **Out-Of-Vocabulary, OOV**) 通常会被映射到一个特殊的 <UNK>(unknown) 标记。
 4. **Token ID 映射**: 将每个词语映射到其在词汇表中的唯一整数 ID。
- 优点: 语义完整, 每个 token 都是一个有完整意义的词, 非常直观。
- 缺点:
 - **词汇表巨大**: 需要为语言中几乎所有的词都创建一个条目, 词汇表高达几十万甚至上百万。
 - **OOV (Out-of-Vocabulary) 问题严重**: 当遇到一个词汇表中没有的词 (如新词、拼写错误、专业术语), 分词器就无法处理, 通常会将其替换为一个特殊的 <UNK>(unknown) 符号, 导致信息丢失。例如, 模型没见过 "chatbot", 就会将其视为 <UNK>。
 - **无法处理词形变化**: run、running、ran 会被视为三个完全不同的词, 模型无法直接看出它们之间的关联, 增加了学习负担。

字符级 (character-level) 分词器

- 核心思想：将文本拆分成一个个独立的字符。
- 示例：
 1. 英文： "I am a pig" → ["I","a","m","a","p","i","g"]
 2. 中文： "我是一头猪" → ["我","是","一","头","猪"]
- 优点：
 - **词汇表小**：词汇表只包含所有基本字符 (如 a-z, A-Z, 0-9, 标点, 中文字符等), 大小非常可控。
 - **无 OOV 问题**：任何单词都可以由字符组成, 因此不存在未知词的问题。
- 缺点：
 - **序列过长**：一个单词会被切成多个字符, 过于琐碎导致输入序列的长度急剧增加, 对模型的计算和内存都是巨大挑战。
 - **语义丢失**：单个字符通常不具备独立的语义, 模型需要从头学习如何将字符组合成有意义的词, 学习效率非常低。

字节级 (byte-level) 分词器

- 核心思想：比字符级更底层的切分方式, 它直接操作文本的原始字节 (Bytes)。所有文本最终都以字节形式存储 (如 UTF-8 编码), 一个英文字母通常占 1 个字节, 一个汉字可能占 3 个字节。
- 示例 (UTF-8 编码):
 1. 英文： "cat" → ["c","a","t"] → [99, 97, 116]
 2. 中文： "猫" → [227, 149, 131](三个字节共同表示一个“猫”字)
- 优点：
 - **词汇表小且固定**：字节的取值范围永远是 0-255, 所以词汇表大小固定为 256。
 - **无 OOV 问题**：任何文本都可以由字节组成, 因此不存在未知词的问题。
- 缺点：
 - **序列更长**：比字符级分词器更长, 因为一个字符可能由多个字节组成。
 - **语义几乎破碎**：模型需要学习从毫无关联的字节序列中重构语义, 学习难度极大。

子词级 (subword-level) 分词器

- 核心思想：目前 LLM(如 GPT、BERT 系列) 的标配, 介于字符级和词级之间, 它将单词拆分成更小的子词 (Subword)。核心思想是：高频词汇作为一个整

体保留, 低频词汇或未见过的词则拆分为更小的、有意义的子词单元。我们要实现的 BPE 分词器就是一种子词级分词器。

● 示例:

1. "the" → ["the"] 由于"the" 高频出现, 所以作为一个整体保留。
2. "wonderful" → ["wonder", "ful"] 由于"wonderful" 低频出现, 所以拆分为"wonder"和"ful" 两个子词。

● 优点:

- **平衡了词汇量和序列长度**: 词汇表大小适中 (通常 3 万-10 万), 序列长度也比字符/字节级短得多。
- **有效处理 OOV 问题**: 任何新词都可以由已知的子词组合而成, 例如模型不认识"webinar", 但可能认识"web" 和"inar", 可以将其切分为["web", "inar"], 从而理解其含义。
- **更灵活的词形变化处理**: 例如"laughing" 和"laughed" 都可以被拆分为["laugh", "ing"]和["laugh", "ed"], 模型能轻易捕捉到"laugh" 这个共同的词根, 理解不同词形间的关系。

● 缺点:

- **词汇表大小不易控制**: 需要手动调整合并频率来平衡词汇量和模型效果, 这在实际应用中可能比较麻烦。
- **训练成本较高**。

类型	单位	优点	缺点	举例
词级 (word)	词	语义清晰	词表极大, OOV	"I love NLP" → ["I", "love", "NLP"]
字符级 (character)	字符	词表极小, 无未知词	语义太碎, 序列太长	"I love" → ["I", " ", "l", "o", "v", "e"]
字节级 (byte)	字节	能统一多语言、符号	可读性差, 序列长	"Hi" → [72, 105](ASCII 码)
子词级 (subword)	词根词缀	权衡两者, 处理新词	稍复杂, 需要训练	"unbelievable" → ["un", "believ", "able"]

薇言大义 2.3

非常重要的基础知识, 是后面开始训练 BPE 分词器的基础。

2.3. BPE Tokenizer Training

参考资料 2.4

正则表达式相关知识: <https://www.runoob.com/regexp/regexp-intro.html>

整个 BPE 分词器训练过程可以分为以下三个步骤:

1. **初始化词汇表**
2. **预分词**

正则表达式 (Regular Expression, 常缩写为 regex 或 regexp) 是一个强大的文本模式匹配工具。它本质上是一种用特殊符号编写的“规则字符串”，可以用来**查找、替换、分割或验证任何符合该规则的文本**。可以理解为 ctrl+f 的超级加强版。

表 2.2: 正则表达式符号表

类型	符号/语法	解释说明	示例
普通字符	a, b, 1, 2	匹配它们自身。	cat 会精确匹配字符串"cat"。
元字符 (任意字符)	.	匹配除了换行符以外的任意单个字符。	c.t 会匹配"cat", "cot", "c_t" 等。
元字符 (重复次数)	*	匹配前面的元素 0 次或多次。	ca*t 会匹配"ct", "cat", "caaat"。
	+	匹配前面的元素 1 次或多次。	ca+t 会匹配"cat", "caaat", 但不匹配"ct"。
	?	匹配前面的元素 0 次或 1 次。	colou?r 会匹配"color" 和"colour"。
字符集	[...]	匹配方括号内的任意一个字符。	c[ao]t 只会匹配"cat" 和"cot"。
	[^...]	匹配不在方括号内的任意一个字符。	[^0-9] 会匹配任何非数字字符。
分组与或	(...)	将括号内的内容视为一个整体, 可以对整体做重复。	(ab)+ 会匹配"ab", "abab", "ababab"。
		表示"或" (OR) 逻辑。	catdog 会匹配"cat" 或者"dog"。
预定义字符类	\d	匹配任意一个数字 (Digit), 等同于 [0-9]。	\d\d\d 会匹配"123", "987"。
	\w	匹配任意一个单词字符, 包括字母、数字、下划线。	\w+ 会匹配一个完整的单词或数字。
	\s	匹配任意一个空白字符, 包括空格、制表符、换行符。	

Example 2.2 (在 python 代码中使用正则表达式)

re.findall 和 re.finditer 的区别:

re.findall 返回所有匹配的子字符串, 返回一个列表。

re.finditer 则是惰性的, 返回一个迭代器, 每次只返回一个匹配的子字符串, 需要手动调用 next() 方法来获取下一个匹配的子字符串。正因如此, 无论文本有多大, 匹配项有多少, 内存占用都极低, 因为它一次只处理一个匹配项。这是处理大文件的唯一可行方法。讲义中也推荐使用 re.finditer。

```
import regex as re
PAT = r'""""(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[\s\p{L}\p{N}]+\s+(?!\\S)|\s+""'
re.findall(PAT, "some text that i'll pre-tokenize")
>>> ['some', 'text', 'that', 'i', 'll', 'pre', '-', 'tokenize']
```

2.3.3. BPE 合并

等到预分词进行粗颗粒度划分之后, 每一个划分后的部分再转换成 UTF-8 序列, 就可以开始 BPE 合并 (即训练 BPE 分词器) 了。比如原始输入文本是"I am a pig", 预分词后得到 ["I", "am", "a", "pig"], 其中每一部分再转换成 UTF-8 序列得到 "I </w>": 1, "a m </w>": 1, "a </w>": 1, "p i g </w>": 1, (注意这里的" </w>" 是特殊符号, 表示一个词的结束) 然后开始 BPE 合并。

从高层次来看, BPE 算法会迭代统计每个字节对, 并识别出现频率最高的字节对 ("A", "B")。然后将这个最频繁出现的字节对 ("A", "B") 的所有实例进行合并, 即替换为一个新标记"AB"。这个新合并的标记会被添加到我们的词汇表中; 因此, BPE 训练后的最终词汇表大小等于初始词汇表 (在我们的案例中是 256 个), 加上训练过程中执行的 BPE 合并操作次数。

BPE 在合并的时候不考虑跨边界合并。例: 若预处理将"dog!" 和"dog." 切分为两个独立标记, 则"dog!" 中的 b'g' 与 b'!' 可合并, 但"dog!" 的 b'!' 与"dog." 的 b'd'

不会被统计（因属于不同标记，会有 `</w>` 符号将其分开）。

BPE 在频率相同时，采用选择**字典序更大的对优先的原则**。例如，若字节对（“A”，“B”）、（“A”，“C”）、（“B”，“ZZ”）和（“BA”，“A”）的频率均为最高，则我们会选择合并（“BA”，“A”）。

字典序一般就是位数多的字符排在位数低的后面，位数相同就按照英文单词顺序排，比如“A”在“B”之前，“B”在“C”之前。

2.3.4. 特殊标记

在文本编码过程中，经常会使用特定字符串（如 `<|endoftext|>`）来存储元数据（例如文档间的分界标记）。进行编码时，通常需要将某些字符串视为“特殊标记”即表示这些标记**永远不应被拆分为多个子标记（即始终作为独立标记保留）**。比如说，序列终止符 `<|endoftext|>` 必须始终作为独立标记（对应单一整数 ID）存在，以便语言模型知晓何时停止生成内容。这类特殊标记必须一开始就被加入词汇表，从而获得对应的固定标记 ID。

2.3.5. BPE 分词器训练示例

1. 原始文本（末尾包含 `<|endoftext|>`）:

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

2. **初始化词汇表**: 256 个固定字节以及特殊标记 `<|endoftext|>`
3. **预分词**: 按照空格划分结果 $\rightarrow \{\text{low: 5, lower: 2, widest: 3, newest: 6}\}$
4. **BPE 合并**: 首先把预分词的结果再去划分一下 $\rightarrow \{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,s,t): 3, (n,e,w,e,s,t): 6\}$; 然后不断迭代合并统计频率最高的字节对，得到新的词汇表。第一轮频率统计: $\{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6\}$, `('es')` 和 `('st')` 并列频率最高，按照字典序最大原则选择 `('st')` 作为这一轮合并的 token 添加到字典里，然后继续下一轮，以此类推。
5. 一般来讲，在大型的 BPE 合并时，训练终止的条件是字词大小到达某个预设值，比如预设 5000，我们初始词典大小时 256(假设无特殊标记)，那么合并 4744 次之后，词汇表大小达到 5000，训练终止。

2.4. BPE 分词器训练实操

参考资料 2.5

cProfile 相关知识: <https://docs.python.org/3/library/profile.html>

Scalene 相关知识: <https://github.com/plasma-umass/scalene>

接下来要实现在 TinyStory 数据集上训练 BPE 分词器。(TinyStory 数据集可从 github 上下载)

我们之前说了像 `<endoftext>` 这样的特殊标记必须一开始就被加入词汇表, 从而获得对应的固定标记 ID。在预分词的时候也要特殊对待一下特殊符号, 一般就会把特殊符号也作为一个分隔符。

我们之前所说的 BPE 训练的办法是基本、朴素的原理, 但在实际操作中这种方法效率会比较低, 速度较慢 (我一开始用的就是这种朴素的算法, 可以说非常慢了), 合适的办法是**直接记录下来所有词的频率到计数器中去, 每次 merge 只需要把对应的计数器内容给改了就好**。虽然都是两重循环, 但第二种办法只用该计数器, 它避免了对重复内容的重复处理, 因此会快不少。举个例子:

● 第一种方法（低效）：

1. 拿到班级花名册, 上面写着每个学生的姓名
2. 花名册上有: ”张三、李四、张三、王五、张三、李四、张三...” (总共 1000 个名字)
3. 你一个一个个地数: 张三、李四、张三、王五、张三、李四...
4. 每个名字都要处理一遍, 即使是重复的

● 第二种方法（高效）：

1. 你先把花名册整理成: 张三 (500 次)、李四 (300 次)、王五 (200 次)
2. 然后直接统计: 张三出现 500 次, 李四出现 300 次, 王五出现 200 次
3. 只需要处理 3 个唯一的名字

另外一些实践技巧: 利用 **cProfile 或 scalene 等工具** 可以来帮我们分析代码中的瓶颈; 在直接测试 TinyStory 数据集时, 可以先测试小部分数据快速检验效果, 避免因为数据量太大而浪费时间。

Extension 2.4 cProfile 和 scalene

cProfile 是 Python 内置的**性能分析 (Profiler) 模块**, 用于测量程序运行过程中**各个函数的执行时间、调用次数等性能数据**, 帮助开发者定位程序中的性能瓶颈。cProfile 会在程序运行时记录每个函数被调用的次数、每次调用消耗的时间以及总耗时, 最终生成一份详细的统计报告。

基本使用方法: `python -m cProfile test.py`

cProfile 会输出文本报告统计, 保存到 `result.prof`, 也可以结合 SnakeViz 来查看可视化报告, 即 `snakeviz result.prof`

Scalene 是一个高精度的 Python 性能分析器 (profiler), 比 cProfile 更先进。它不仅能分析 CPU 时间, 还能同时分析: **CPU 使用 (Python 与本地代码分离)、内存使用 (包括分配与释放)、GPU 使用 (可选)、行级别的性能分析结果**

可以在命令行中直接使用查看文本报告: `scalene test.py`

也可以生成网页可视化报告, 即 `scalene -html example.py`

2.5. 利用 BPE 分词器进行编码和解码

参考资料 2.6

BPE 分词器编码和解码相关知识: <https://github.com/huggingface/tokenizers>

我们训练 BPE tokenizers 的最终目的还是希望它对新的文本进行编解码, 以便后续的操作。

2.5.1. 编码

目标是利用我们已经训练好的 BPE 分词器将一个原始的文本字符串转换成一个整数 Token ID 列表。具体实现步骤和训练 BPE 分词器的过程类似, 但是不需要进行 BPE 合并。

1. 处理特殊符号:

- (a). **最优先处理**: 在进行任何其他操作之前, 你需要先将文本中的特殊符号替换为它们对应的占位符或直接分割出来。
- (b). 策略:
 - 如果特殊符号在 vocab 中已经有预定义的 ID (通常是在训练 BPE 之前就加入的), 可以用一个独特的、不会在普通文本中出现的字节序列来临时替换它们。或者先用特殊符号将整个文本分割成多个部分。例如, 如果文本是 "Hello <|endoftext|> World", 你可以先将其分割成 ["Hello ", "<|endoftext|>", " World"]。
 - 对于每个非特殊符号的文本块, 进行下面的预分词和合并。
 - 对于特殊符号块, 直接查找其在 vocab 中的 ID。

2. Pre-tokenize (预分词):

- (a). 目的: 将文本分割成一些“词块”(word chunks)。BPE 合并只在这些词块内部进行, 不会跨越词块边界。这通常是为了防止合并无意义的字符组合(比如一个词的末尾和下一个词的开头)。
- (b). 方法: 使用与 BPE 训练时相同的正则表达式。这个正则表达式通常会根据空格、标点符号等来切分文本。
- (c). 输出: 一个字符串列表, 每个字符串是一个预分词块。例如, "Hello world!" 可能被预分词为 ["Hello", " world", "!", ""] (注意空格可能被归属到某个块)。

3. Apply the merges (应用合并规则):

- (a). 对每一个预分词块单独执行以下操作:
 - 转换为字节序列列表: 将预分词块(字符串)编码为 UTF-8 字节序列, 然后将这个字节序列拆分成单个字节的列表。例如, "the" -> b'the' -> [b't', b'h', b'e']。

- 迭代应用合并规则：

- 遍历 merges 列表中的每一条合并规则 (pair_A, pair_B)。
- 在当前的字节序列列表中, 查找所有连续出现的 (pair_A, pair_B)。
- 将找到的第一个（或所有，取决于实现策略，但通常是迭代地、贪婪地合并最先出现的）(pair_A, pair_B) 替换为它们合并后的新字节序列 pair_A + pair_B。 (重要： 每应用一次合并，字节序列列表的结构就可能发生变化。你需要重新从 merges 列表的开头开始检查，或者更高效地只检查与新合并的 token 相关的可能合并。)
- 例如，当前序列是 [b't', b'h', b'e']，merges 中有 (b't', b'h')。应用后变成 [b'th', b'e']。然后假设 merges 中还有 (b'th', b'e')，应用后变成 [b'the']。
- 查找 Token ID： 当一个预分词块不能再进行任何合并时，它内部的每个（可能是合并后的）字节序列都应该对应 vocab 中的一个 Token ID。将这些字节序列转换为它们的 ID。

(b). 拼接结果： 将所有预分词块（以及特殊符号）得到的 Token ID 列表按顺序拼接起来，得到最终的编码结果。

举例： 输入 -> 'the cat ate'

- vocab: 0: b'', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'
- merges: [(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]
- special_tokens: (假设没有)

处理过程：

1. **Pre-tokenize:** ['the', ' cat', ' ate'] (注意空格的归属)

2. **处理 'the':**

- 初始字节: [b't', b'h', b'e']
- 遍历 merges: (b't', b'h') 可应用 -> [b'th', b'e']
- 从头遍历 merges (对于 [b'th', b'e']): (b'th', b'e') 可应用 -> [b'the']
- 从头遍历 merges (对于 [b'the']): 没有可应用的。
- 查找 ID: b'the' -> 9. 结果: [9]

3. **处理 ' cat':** (假设预分词包含前导空格)

- 初始字节: [b' ', b'c', b'a', b't'] (UTF-8 编码的空格字节，这里用 b' ' 示意)
- 遍历 merges: (b' ', b'c') 可应用 -> [b' c', b'a', b't']
- 从头遍历 merges (对于 [b' c', b'a', b't']): 根据讲义结果 [7, 1, 5]，它实际上是： b' c' -> 7 b'a' -> 1 b't' -> 5 这意味着在 [b' c', b'a', b't'] 状态下，

没有进一步的合并可以应用了，或者 (b'a', b't') 这个合并规则不存在或顺序靠后。或者，更可能的是，'cat' 预分词结果是 [' ', 'cat'], 然后 'cat' -> [b'c', b'a', b't'], 没有合并，直接查 ID 得 [2,1,5]。但讲义结果是 [7,1,5]，对应 b'c', b'a', b't'。这暗示了'cat' 的预分词结果就是'cat'，然后它被字节化为 [b' ', b'c', b'a', b't']。第一个合并是 (b' ', b'c') -> b'c' (ID 7)。剩下 [b'a', b't']。这两个不能再合并，所以分别是 b'a' (ID 1) 和 b't' (ID 5)。

4. 处理 'ate': (假设预分词包含前导空格)

- 初始字节: [b' ', b'a', b't', b'e']
- 遍历 merges: (b' ', b'a') 可应用 -> [b'a', b't', b'e']
- 从头遍历 merges (对于 [b'a', b't', b'e']): (b'a', b't') (这里指 b'a' 和 b't' 合并) 可应用 -> [b'at', b'e'] (注意 b'at' 是 ID 10)
- 从头遍历 merges (对于 [b'at', b'e']): 没有可应用的。
- 查找 ID: b'at' -> 10, b'e' -> 3. 结果: [10, 3]

5. 最终结果: [9] + [7, 1, 5] + [10, 3] = [9, 7, 1, 5, 10, 3]。

2.5.2. 解码

目标是将一个整数 Token ID 列表转换回原始的文本字符串。

详细步骤：

1. ID to Bytes (ID 转字节序列):

- 遍历输入的 Token ID 列表。
- 对于每个 ID，使用 vocab 查找其对应的字节序列。
- 将所有查找到的字节序列拼接起来，形成一个单一的字节串。
- 例如，[9, 7, 1, 5, 10, 3] -> b'the' + b'c' + b'a' + b't' + b'at' + b'e' -> b'the cat ate'。

2. Bytes to String (字节串转字符串):

- 使用 UTF-8 解码器将拼接后的字节串转换回 Unicode 字符串。
- **errors='replace'**: 这很关键。如果解码过程中遇到无效的 UTF-8 字节序列（比如用户提供了一个非法的 ID 序列，或者你的 vocab 中存在一些不能组成有效 UTF-8 的字节片段），errors='replace' 会用 Unicode 的替换字符 U+FFFD 来代替这些无效部分，而不是抛出 UnicodeDecodeError。

2.6. BPE 讲义相关 Problems

Problem 2.3

1. 从 TinyStories 和 OpenWebText 中各抽取 10 份文档样本。使用您先前训练的 TinyStories 和 OpenWebText 分词器（词汇表大小分别为 1 万和 3.2 万），将这些抽样文档编码为整数 ID。这两个分词器的压缩比（字节/标记）分别是多少？

答: 对每个文档: 计算原始 UTF-8 编码下的字节数。用对应的 tokenizer 编码成 token ID 序列, 并统计 token 数。最后可以得到平均压缩率: $\text{bytes/token} = \text{总字节数} / \text{总 token 数}$

2. 如果用 TinyStories 分词器处理 OpenWebText 样本会发生什么? 比较压缩率和/或定性描述产生的结果。

答: 如果用 TinyStories 分词器处理 OpenWebText 样本压缩比会降低很多。原因一是因为 TinyStories 分词器的词汇表大小只有 1 万, 而 OpenWebText 分词器的词汇表大小有 3.2 万, 能表示的 token 会少很多。二是 TinyStories tokenizer 是在简单的儿童故事上训练的, 而 OpenWebText 包含各种网络文本 (新闻、技术文章、论坛讨论等)。当 tokenizer 遇到训练时未见过的词汇模式时, 会将其分解成更多的小片段, 降低压缩效率。

3. 估算你的分词器吞吐量 (例如以字节/秒为单位)。处理 Pile 数据集 (825GB 文本) 需要多长时间?

答: 这个在不同的分词器的实现方式和硬件条件下吞吐量也有很大不同。

4. 使用您的 TinyStories 和 OpenWebText 分词器, 将相应的训练集和开发集编码为整数标记 ID 序列。我们稍后将用此来训练语言模型。建议将标记 ID 序列化为 uint16 数据类型的 NumPy 数组。为何 uint16 是合适的选择?

答: uint16 范围: $0 \text{ 到 } 65,535 - 1$, 选择 uint16 是因为它可以表示 0 到 65,535 的整数范围, 足够覆盖 10K 和 32K 词汇量的所有 token ID, 同时相比 uint32 节省了一半的存储空间。

2.7. BPE 章节实验

1. train_bpe.py: 编写一个函数, 给定输入文本文件的路径, 训练一个 (字节级) BPE 分词器。您的 BPE 训练函数应至少处理以下输入参数: input_path (输入文本文件路径, 这里是 TinyStories 和 OpenWebText 数据集), vocab_size (词汇表大小), special_tokens (特殊符号列表), vocab (分词器词汇表, 一个从整型 (词汇表中的标记 ID) 到字节 (标记字节) 的映射关系。), merges (训练生成的 BPE 合并操作列表。每个列表项为一个字节元组 (<token1>, <token2>), 表示 <token1> 与 <token2> 进行了合并。这些合并操作应按创建顺序排列。)

`uv run pytest tests/test_train_bpe.py` 来运行测试文件。

2. `tokenizer.py` : 实现一个分词器类，该分词器在给定词汇表和合并规则列表的情况下，能够将文本编码为整数 ID，并将整数 ID 解码回文本。该分词器还应支持用户提供的特殊标记（若这些标记尚未存在于词汇表中，则将其追加至词汇表）。

`uv run pytest tests/test_tokenizer.py` 来运行测试文件。

3

Transformer Language Model Architecture(Transformer 架构)

首先要明确，语言模型是一种**自回归**的**预测**模型。

● 输入：

- 语言模型不直接处理文字，而是处理数字。每个词、标点符号或更小的语言单位（如“ing”、“un-”）都会被赋予一个唯一的整数 ID(也就是上一章提到的**token**)。
- 利用**并行计算**，模型可以按照 batch 同时处理多段文本。

● 输出：

- 对输入序列的每一个词，模型都会预测下一个词是什么，即给出概率分布。
- 所有可能词的概率**必须标准化**，即和为 1。

● 训练：

- 现代语言模型（尤其是像 GPT 这样的自回归模型）的预训练过程，本质上是一种基于大规模文本语料库的**自监督学习 (Self-Supervised Learning)**。其核心目标是最大化给定上文序列（Context）条件下，预测下一个词元（Token）的条件概率。
 - **目标任务：下一个词元预测**。模型学习的是一个概率分布 $P(w_n | w_1, w_2, \dots, w_{n-1})$ ，即在已知前面所有词的条件，下一个词 w_n 出现的概率。
 - **学习范式：自监督学习 (Self-Supervised Learning)**。由于训练的标签是从输入数据本身中**自动获取的**，而非人工标注，因此被称为“自监督”。这种范式使得模型可以利用海量的、无标签的原始文本进行学习，是大型语言模型能够成功训练的关键。
 - **优化过程：最小化损失函数 (Loss Function Minimization)**。当模型做出预测后，会通过一个名为**交叉熵损失 (Cross-Entropy Loss)** 的函数，来量化其预测的概率分布与“真实标签”（即下一个词的**独热编码 one-hot vector**）之间的差距。然后，模型使用**反向传播 (Back-propagation)** 算法来调整内部数以亿计的参数，其目标就是让这个损失值尽可能小。这个不断调整参数以减少误差的过程，就是模型的“**学习**”过程。

3.1 基础模块: 线性层和嵌入层	32
3.1.1 参数初始化	32
3.1.2 实验中参数初始化标准	33
3.2 词嵌入层 (embedding layer)	33
3.3 两大基础模块: 线性模块和嵌入模块	33
3.3.1 线性模块 (Linear Module)	33
3.3.2 嵌入模块 (Embedding Module)	34
3.4 Pre-norm Transformer Block	34
3.4.1 RMSNorm 的原理	35
3.4.2 位置级前馈网络 (Position-wise Feed-Forward Network)	37
3.4.3 相对位置嵌入 (RoPE 旋转位置编码)	40
3.4.4 缩放点积注意力 scaled dot-product attention	46
3.4.5 多头因果注意力机制	47
3.5 Transformer 总结篇	47
3.5.1 输入部分	48
3.5.2 BPE tokenizer(预分词)	48
3.5.3 Embedding (词嵌入)	48
3.5.4 Position encoding (位置编码)	49
3.5.5 Transformer block	49
3.5.6 RMSnorm	49
3.5.7 Causal Multi-head self-attention (多头注意力机制)	49
3.5.8 SwiGLU	49
3.5.9 最终输出	50

3.1. 基础模块: 线性层和嵌入层

参考资料 3.1

He 初始化: <https://zhuanlan.zhihu.com/p/40175178> 《神经网络与深度学习》邱锡鹏相关章节 (讲的最好)

3.1.1. 参数初始化

我们训练神经网络的目的是为了找到一组参数, 使得模型在训练数据上的表现最好。但是, 如果初始参数设置不当, 可能会导致模型无法收敛或者收敛到局部最优解甚至出现**梯度爆炸**、**梯度消失**等问题。因此, 参数初始化也是一门需要研究的学问。

最常见的两种参数初始化方法: **Xavier 初始化和 He 初始化**。

● Xavier 初始化 (Glorot Initialization)

Xavier 初始化由 Xavier Glorot 和 Yoshua Bengio 在 2010 年提出。它的核心思想是保持每一层激活值的**方差**和反向传播时梯度的方差在前向和反向传播中保持不变。Xavier 初始化适用于**Sigmoid, Tanh** 等对称函数。

Xavier 初始化通常有两种分布形式:

- **均匀分布 (Uniform)**: 权重从均匀分布 $U[-r, r]$ 中采样, 其中: $r = \sqrt{\frac{6}{fan_{in} + fan_{out}}}$
均匀分布方差为 $\frac{(b-a)^2}{12} = \frac{r^2}{3}$ 。这里的 fan_{in} 是一层网络输入神经元数量, fan_{out} 是输出的神经元数量。
- **正态分布 (Normal)**: 权重从均值为 0, 标准差为 σ 的正态分布中采样, 其中: $\sigma = \sqrt{\frac{2}{fan_{in} + fan_{out}}}$ 这是由于每经过一层, 权重的方差就会变成原来的 $\frac{1}{fan}$, 其中 fan 表示这一层的神经元的数量。

工作原理简述: 通过同时考虑输入和输出神经元的数量, Xavier 初始化试图在层与层之间找到一个平衡点, 使得信号的方差既不会在传播中衰减, 也不会无限放大。

● He 初始化 (Kaiming Initialization)

He 初始化由 Kaiming He (何凯明, 残差网络也是他提出的) 在 2015 年提出。针对 Xavier 初始化在**ReLU 激活函数**上效果不佳的问题, He 初始化就适配于**ReLU 激活函数**。

ReLU 函数 $f(x) = \max(0, x)$ 的特性是, 它会将所有负输入都变为 0。这破坏了 Xavier 初始化所依赖的“激活函数关于原点对称”的假设, 并导致大约一半的神经元输出为 0, 从而改变了输出的方差。

He 初始化考虑到 ReLU 会将一半的输入置为零, 这会使得输出方差减半。为了补偿这一点, He 初始化在计算方差时引入了一个因子 2。其他和 Xavier 初始思路一致。

- **均匀分布 (Uniform)**: 权重从均匀分布 $U[-r, r]$ 中采样, 其中: $r = \sqrt{\frac{6}{fan}}$

- **正态分布 (Normal)**: 权重从均值为 0, 标准差为 σ 的正态分布中采样, 其中: $\sigma = \sqrt{\frac{2}{fan_{in}}}$

之所以 Kaiming 初始化只考虑 fan_{in} , 是因为这个更注重**前向传播**, 不是很在意反向传播。

3.1.2. 实验中参数初始化标准

- 线性层权重: $\mathcal{N}(\mu = 0, \sigma^2 = \frac{d_{in}+d_{out}}{2})$, 范围限制在 $\pm 3\sigma$ 以内。
- 词嵌入权重: $\mathcal{N}(\mu = 0, \sigma^2 = 1)$, 范围限制在 ± 3 以内。
- RMSNorm 权重: 1

3.2. 词嵌入层 (embedding layer)

- **输入**: 整数 token ID 序列 (这里已经是由之前的 BPE tokenizer 处理文本之后的结果了)
- **词嵌入 (embedding)**: 目的是为了将 token ID 转换为**稠密向量**; 一个词嵌入层 (embedding layer) 的作用就是把这些离散的、没有语义的整数 ID, 转换为连续的、包含语义信息的**向量**。

比如, “猫” 的向量可能和 “狗” 的向量在某种 “动物” 维度上比较接近, 而和 “汽车” 的向量相距较远。每个嵌入层接收形状为 (batch_size, sequence_length) 的整数张量, 并生成形状为 (batch_size, sequence_length, d_model) 的向量序列。d_model 代表输出维度, 越大代表语义信息越丰富, 模型越强大。

3.3. 两大基础模块: 线性模块和嵌入模块

3.3.1. 线性模块 (Linear Module)

线性模块是神经网络里面最最基础的模块了, 它就是一个线性变换, 将输入的向量映射到另一个向量。

$$y = Wx + b$$

其中, W 是权重矩阵, b 是偏置向量, x 是输入向量, y 是输出向量。

实现的过程中**别忘了初始化**就好。

```
class LinearModule(nn.Module):
    def __init__(self, in_features: int, out_features: int, device: torch.device |
        ↪ None = None, dtype: torch.dtype | None = None):
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.device = device
        self.dtype = dtype
        self.W = nn.Parameter(torch.empty(self.out_features, self.in_features,
            ↪ device=self.device, dtype=self.dtype))
        # self.b = nn.Parameter(torch.empty(out_features, device=device,
            ↪ dtype=dtype))
        # 对权重进行 Xavier 初始化
        std = 2 / (self.in_features + self.out_features) ** 0.5
        torch.nn.init.trunc_normal_(self.W, std=std, a = -3 * std, b = 3 * std)
```

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    return x @ self.W.T
```

3.3.2. 嵌入模块 (Embedding Module)

嵌入模块就是我们之前所说的嵌入层所利用的模块，它的功能是将代表文本的整数“词元 ID” (token ID) 转换为高维的、模型能够理解的向量表示。

嵌入层用最高效的“查字典”方式，实现了在数学上等价于“**一个输入为独热编码的线性层**”的功能。输入为 (batch_size, sequence_length) 代表一个批次每个句子有 sequence_length 个 token，输出为 (batch_size, sequence_length, embedding_dim) 代表每个 token 的 embedding 向量。

```
class EmbeddingModule(nn.Module):
    def __init__(self, num_embeddings: int, embedding_dim: int, device: torch.device |
        ↪ None = None, dtype: torch.dtype | None = None):
        super().__init__()
        self.num_embeddings = num_embeddings # 词表 vocab_size 大小
        self.embedding_dim = embedding_dim # 词向量维度 d_model
        self.device = device
        self.dtype = dtype

        self.embedding_matrix = nn.Parameter(torch.empty(self.num_embeddings,
            ↪ self.embedding_dim, device=self.device, dtype=self.dtype))
        std = 1
        torch.nn.init.trunc_normal_(self.embedding_matrix, std=std, a = -3 * std, b = 3
            ↪ * std)

    def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
        return self.embedding_matrix[token_ids] # 从词表到词向量的映射的神经网络里读取
        ↪ token_ids 输出词向量
```

3.4. Pre-norm Transformer Block

● Transformer 块的基本结构：

- **多头自注意力机制 (Multi-Head Self-Attention mechanism)**：这是 Transformer 的核心，允许模型在处理序列时对输入的不同部分赋予不同的权重。

- **位置级前馈网络 (Position-wise Feed-Forward Network)**：一个简单的全连接层，独立地应用于序列中每个位置的向量。

● 残差连接 (Residual Connection)：

- 原始 Transformer 论文指出，每个子层都使用了残差连接。残差连接的目的是将子层的输入 (x) 直接加到子层的输出 (Sublayer(x)) 上，即 $x + \text{Sublayer}(x)$ 。

- 这种设计有助于解决深度网络中的**梯度消失问题**，使得信息可以直接通过多层网络传递。

● 层归一化（Layer Normalization）的不同应用方式：

- **后归一化**（Post-norm）Transformer：这是原始 Transformer 论文中使用的架构。层归一化是在每个子层的输出之后，残差连接之后应用的。即： $\text{LayerNorm}(x + \text{Sublayer}(x))$ 。
- **前归一化**（Pre-norm）Transformer：这是后续研究（Nguyen and Salazar, 2019; Xiong et al., 2020）发现的一种改进架构。层归一化是在每个子层的输入之前应用的。即： $x + \text{Sublayer}(\text{LayerNorm}(x))$ 。此外，在整个 Transformer 堆栈的最后一个 Transformer 块之后，还会有一个额外的**层归一化**。
- **优点**：多项工作发现，这种“前归一化”的方式改善了 Transformer 的训练稳定性。
- **直观解释（Intuition）**：前归一化会创建一个“干净的残差流（residual stream）”，即从输入嵌入到 Transformer 最终输出的路径上，没有任何归一化操作直接作用于残差连接本身。这种设计被认为能改善梯度流动（gradient flow），使得梯度更容易有效地反向传播通过多层网络。
- **当前标准**：由于其训练稳定性的优势，“前归一化”已经成为现代大型语言模型（如 GPT-3, LLaMA, PaLM 等）的标准实践。

3.4.1. RMSNorm 的原理

RMSnorm 是一种**简化版的层归一化（Layer Normalization）**。

● LN 层归一化的公式是：

$$LN(x) = \gamma \odot \frac{x - \mu}{\sigma} + \beta \quad (3.1)$$

- x 是输入特征向量（对于 NLP 通常是 $(\text{batch_size}, \text{sequence_length}, \text{hidden_size})$ 中的一个 hidden_size 维度上的向量），这里的 hidden_size 通常就是 d_{model} 。
- μ 是**平均值（mean）**，计算的是 x 在其最后一个维度上的均值。
- σ 是**标准差（standard deviation）**，计算的是 x 在其最后一个维度上的标准差。
- γ 和 β 是**可学习的缩放（scale）和偏移（shift）参数**，它们的维度与 x 的最后一个维度相同。它们允许归一化后的数据进行仿射变换，从而恢复模型的表达能力。

● RMSnorm 的公式是：

$$\text{RMSnorm}(x) = \gamma \frac{x}{\text{RMS}(x)} \quad (3.2)$$

- x 是输入特征向量。

- $RMS(x)$ 是 x 的**均方根 (Root Mean Square)**。

$$RMS(x) = \sqrt{\frac{1}{D} \sum_{i=1}^D x_i^2} = \sqrt{mean(x^2)} \quad (3.3)$$

- 这里的 D 是 x 的特征维度大小 (即 `hidden_size`)。
- γ 是**可学习的缩放 (scale) 参数**，其维度与 x 的最后一个维度相同。
- **注意**：RMSnorm **没有** (偏移) 参数 β 。

移除均值计算可以带来以下好处：

- **计算效率更高**：计算均值需要对所有元素求和再除以维度，这本身是一个 $O(D)$ 的操作。移除这一步可以减少计算量，尤其是在硬件层面上可能更高效。
- **内存占用更少**：不计算和存储均值，也不需要 β 参数。
- **简化模型**：参数更少，模型更简洁。

让我们再详细解释一下 (`batch_size`, `sequence_length`, `hidden_size`) 这种形状的张量在 Layer Normalization (LN) 或 RMSnorm 中是如何被归一化的。

1. (`batch_size`, `sequence_length`, `hidden_size`) 张量的含义

- **`batch_size`**：批次大小，表示一次性处理多少个独立的序列 (或句子)。
- **`sequence_length`**：序列长度，表示每个序列中有多少个 token (词或子词)。
- **`hidden_size`**：隐藏层维度，也常被称为 **`d_model`**。这是每个 token 经过词嵌入层或其他层后，所对应的**稠密向量的维度**。

可以把这个三维张量想象成：`batch_size` 个矩阵，每个矩阵的形状是 (`sequence_length`, `hidden_size`)。而每个 (`hidden_size`) 向量，就是对应于序列中某个位置的某个 token 的表示。

2. Layer Normalization 和 RMSnorm 的归一化范围

当你看到 x 在 LN/RMSnorm 的公式中时，这个 x 指的是：

针对 (`batch_size`, `sequence_length`, `hidden_size`) 张量中的每一个 (`batch_idx`, `sequence_idx`) 对应的 `hidden_size` 维度上的向量。

也就是说：对于批次中的每个样本 (`batch_idx`)，对于序列中的每个位置 (`sequence_idx`)，都会独立地提取出一个形状为 (`hidden_size`,) 的向量。Layer Normalization 或 RMSnorm 的均值/RMS 和标准差，就是在这个 (`hidden_size`,) 维度的向量上计算的。

Example 3.1

具体地说：如果你的输入张量是 `input_tensor`，其形状为 (`B`, `S`, `D`) (`B`=`batch_size`, `S`=`sequence_length`, `D`=`hidden_size`)。当你应用 Layer Normalization 或 RMSnorm 时，它们会遍历：`b` 从 0 到 `B-1`, `s` 从 0 到 `S-1`，然后，对于每一个 (`b`, `s`) 对，它会取 `input_tensor[b, s, :]` 这个子向量 (其

形状为 $(D,)$ ，并对这个 $(D,)$ 维度的向量进行归一化。所以，归一化操作是独立地应用于 $B * S$ 个这样的 D 维向量。

3. **词嵌入层之后的稠密向量** `batch_size` 中一个，然后这个对应映射后的 `hidden_size` 也就是经过词嵌入层之后的稠密向量，对这个向量在进入 Transformer 块之前归一化。

- **batch_size 中一个**：对应于 `input_tensor[b, :, :]`，即批次中的一个完整序列。
- **这个对应映射后的 hidden_size**：对应于 `input_tensor[b, s, :]`，即该序列中某个特定 token（在 `s` 位置）的 `hidden_size` 维度的向量。
- **也就是经过词嵌入层之后的稠密向量**：正是如此。词嵌入层的输出形状就是 $(batch_size, sequence_length, embedding_dim)$ ，其中 `embedding_dim` 通常就是 `hidden_size` (或 `d_model`)。
- **对这个向量在进入 Transformer 块之前归一化**：这正是“前归一化”(pre-norm) Transformer 的核心思想。在每个子层（多头自注意力或前馈网络）的输入端，都会对这个 $(hidden_size,)$ 维度的向量进行归一化。

总结一下：

Layer Normalization 和 RMSnorm 总是沿着**特征维度**（通常是最后一个维度，即 `hidden_size` 或 `embedding_dim`）进行归一化，并且这种归一化是**独立地**应用于每个样本的每个序列位置的。它不涉及批次维度或序列长度维度上的统计计算。

这个操作确保了进入 Transformer 子层的每个 token 的向量表示，其数值范围都得到了稳定，从而有助于训练的**稳定性和效率**。

```
class RMSNorm(nn.Module):
    def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(d_model, device=device, dtype=dtype))
        ↪ #weight 对应缩放参数 gamma

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # 题目要求对于不同的精度要先转换为 float32 再进行归一化，最后再转换回原来的精度
        origin_dtype = x.dtype
        x_fp32 = x.to(torch.float32)

        norm_x = x / (x.pow(2).mean(dim=-1, keepdim=True) + self.eps).sqrt()
        x_norm = norm_x.to(origin_dtype)
        return x_norm * self.weight
```

3.4.2. 位置级前馈网络 (Position-wise Feed-Forward Network)

原始 Transformer 的 FFN (基于 ReLU)

- **结构**：包含两个线性变换层，中间夹着一个 **ReLU 激活函数**。
- **公式**： $FFN(x) = Linear_2(ReLU(Linear_1(x)))$

- **维度**：中间隐藏层（Linear_1 的输出）的维度 d_{ff} 通常是输入维度 d_{model} 的 4 倍 ($d_{ff} = 4 * d_{model}$)。

Extension 3.1 常见激活函数及对应场景

Sigmoid 函数

- 公式： $f(x) = \frac{1}{1+e^{-x}}$
- 范围： $(0, 1)$
- 优点：
 - 输出平滑，适合于**二分类问题**。
 - 可以将输出映射到 $(0, 1)$ 区间。
- 缺点：
 - 梯度消失：在输入值很大或很小时，导数接近 0，导致更新缓慢。
 - 输出不以 0 为中心，可能导致优化效率下降。

Tanh 函数

- 公式： $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- 范围： $(-1, 1)$
- 优点：
 - 比 Sigmoid 更为平滑，输出以 0 为中心。
 - 解决了**Sigmoid 的输出不以 0 为中心的问题**。
- 缺点：
 - 仍然存在**梯度消失**的问题。

ReLU 函数

- 公式： $f(x) = \max(0, x)$
- 范围： $[0, +\infty)$
- 优点：
 - 计算简单，收敛速度快。
 - 在正区间内，梯度恒为 1，避免了梯度消失问题。
- 缺点：
 - 梯度消失问题：对于负输入，梯度为 0，可能导致“死亡神经元”现象。

Softmax 函数

- 公式： $f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$ 适用于**多分类**任务。

- 范围：输出值在 (0, 1) 之间，且所有输出的和为 1。
- 优点：
 - 将输出转换为概率分布，适合于多分类问题。

SWiGLU

SiLU (Sigmoid Linear Unit) / Swish 激活函数

- 定义： $SiLU(x) = x \cdot \sigma(x) = x / (1 + e^{-x})$ 。
- 特点：类似于 ReLU，但在零点附近是平滑的，这有助于缓解梯度消失问题，并允许负数输入通过。

门控线性单元 (Gated Linear Units, GLUs)

- 定义： $GLU(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x$ 。
- 其中, $\odot$ 代表 **元素级乘法**。e.g. $[1.227, -0.162] \odot [-1.6, 1.5] = [1.227 \times (-1.6), -0.162 \times 1.5] = [-1.963, -0.243]$
- 特点：将一个线性变换的结果通过 **sigmoid 函数**（作为门控信号），再与另一个线性变换的结果进行逐元素相乘。
- 优点：建议通过提供一个“线性路径”来“减少深度架构中的梯度消失问题”，同时保留非线性能力。

对于 SwiGLU 的定义：

$$SwiGLU(x, W_1, W_2, W_3) = W_2 (SiLU(W_1 x) \odot W_3 x) \quad (3.4)$$

- $x \in \mathbb{R}^{d_{model}}$ 是输入向量(在实际中,通常是 (batch_size, sequence_length, d_model) 形状张量的最后一个维度)，
- $W_1, W_3 \in \mathbb{R}^{d_{ff} \times d_{model}}$ 是线性变换的权重矩阵,
- $W_2 \in \mathbb{R}^{d_{model} \times d_{ff}}$ 是另一个线性变换的权重矩阵,
- $d_{ff} = \frac{8}{3} d_{model}$, d_{ff} 是中间隐藏层的维度，通常是 d_{model} 的 8/3 倍（并确保是 64 的倍数）

3.4.3. 相对位置嵌入 (RoPE 旋转位置编码)

参考资料 3.2

RoPE 旋转位置编码相关知识：

- https://www.bilibili.com/video/BV1CQoaY2EU2?spm_id_from=333.788.player.player_end_recommend_autoplay&vd_source=453c2363ee43bdaa84f759f243a88819
- https://www.bilibili.com/video/BV1Mj421R7JQ/?spm_id_from=333.1007.top_right_bar_window_history.content.click&vd_source=453c2363ee43bdaa84f759f243a88819
- <https://zhuanlan.zhihu.com/p/647109286>
- <https://zhuanlan.zhihu.com/p/642884818>

Transformer 中常见的位置编码包括**绝对位置编码**和**相对位置编码**两大类。

Rope 旋转位置编码就是属于相对位置编码，是一种经常被用在大模型（比如 LLaMA、chatGLM）里面的方式。

为什么需要位置编码？

当我们想要利用 **Transformer 架构**来训练一个大语言模型时

1. 首先会输入一段文本，或者通俗地讲是一句话，比如“我喜欢你”。
2. 这句话会经过已经训练好的分词器比如 **BPE tokenizer** 进行初步分词，转换为 token 数学向量形式；
3. 之后，token 会经过**词嵌入层 (input embedding)**，其神经网络输入输出形状为 (batch_size, sequence_length, d_model) 转换为**稠密向量 X**；
4. 转换后的稠密向量 X 会输入到**自注意力机制**中，具体的运算就是 $Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$ ，其中 Q(query)、K(key)、V(value) 是 X 分别经过 W_q, W_k, W_v 三个神经网络之后的结果。
5. 那么实际上 $QK^T = W_qXX^TW_k$ 的运算主要依赖于原始的稠密向量 XX^T 。而对于一句话来说，如果不去关注每一个字的**位置信息**，只是关注字词内容本身，那么每个 token 所转换成的稠密向量 X 肯定是相同的，经过注意力机制后相同文本不同位置的语句结果也不会变化，但这是不符合实际情况的。

Example 3.2

比如，“我喜欢你” → [“我”，“喜欢”，“你”] → [[1,2,3],[4,5,6],[7,8,9]]
这句话如果改为“你喜欢我” → [“你”，“喜欢”，“我”] → [[7,8,9],[4,5,6],[1,2,3]]
可以看到，虽然“我喜欢你”和“你喜欢我”这两句话语义有很大的不同，但最后转换成的稠密向量 X 在具体的值上没有变化，都是 [1,2,3,4,5,6,7,8,9]。
我们引入位置编码的目标就是希望额外保留原有文本的位置（索引）信息，

使得”你喜欢我”变成类似
[[74,86,96],[433,53,62],[12,42,63]] 这种和”我喜欢你”生成的 X 有明显区别。

这个章节主要讲解相对位置编码下的 **Rope (旋转位置编码)**

什么样的位置编码是好的位置编码

1. 相对位置
2. 外推性好

外推性好代表着如果你训练时最长文本是 1000，但真实推理时是 5000，也能很好适应。

什么是旋转矩阵？

旋转矩阵是在乘以一个向量的时候改变了向量的方向但不改变大小的效果并保持了**手性**的矩阵。

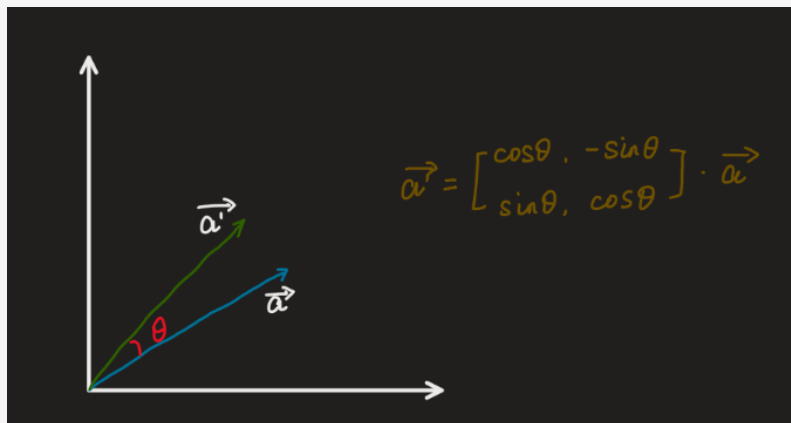
手性指左手右手坐标系，类似物理上的左手定则，右手定则；

性质

设 M 是任何维的一般旋转矩阵: $M \in \mathbb{R}^{n \times n}$

- 两个向量的内积在它们都被一个旋转矩阵操作之后保持不变: $a^T \cdot b = (Ma)^T \cdot Mb$
- 旋转矩阵的逆矩阵是它的转置矩阵: $MM^{-1} = MM^T = I$ 这里的 I 是单位矩阵。
- 若用 $M(\theta)$ 表示逆时针旋转的角度为 θ ，那么有 $M(\alpha + \beta) = M(\alpha)M(\beta)$
- 旋转矩阵的转置等于原矩阵角度取负，即 $M(\theta)^T = M(-\theta)$
- 一个矩阵是旋转矩阵，当且仅当它是正交矩阵并且它的行列式是 1。正交矩阵的行列式是 ± 1 ；如果行列式是 -1 ，则它包含了一个反射而不是真旋转矩阵。

直观上理解就是乘 $M(\alpha)$ 代表先旋转 α ，再乘 $M(\beta)$ 代表再选择 β ，合计旋转了 $\alpha + \beta$ ，等效于乘 $M(\alpha + \beta)$ 。



二维空间下的旋转矩阵

在二维空间中，旋转可以用一个单一的角 θ 定义。作为约定，**正角表示逆时针旋转**。把笛卡尔坐标的列向量关于原点逆时针旋转 θ 的矩阵是：

$$M(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} = \cos \theta \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \sin \theta \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \exp \left(\theta \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \right)$$

欧拉公式： $e^{i\theta} = \cos \theta + i \sin \theta$

二维向量和复数域

一般的复数表示法为 $z = a + ib$ ，其中 i 为复数单位，我们就可以根据实轴和虚轴对一个复数向量化表示，比如 $z_1 = 3 + 4i \rightarrow (3, 4)$

将位置信息注入到稠密向量中

前面我们讲到，原始不加入位置信息的经过词嵌入层之后的稠密向量 X 存在着较大缺陷，因此，我们要在原始向量的基础上添加位置信息，即 $f(x_i, i)$ ，其中 i 代表该元素的位置信息（索引）。

相对位置编码希望能利用上 token 之间的相对位置信息，假定 query 向量 q_m 和 key 向量 k_n 之间的内积操作可以被一个函数 g 表示，该函数 g 的输入是词嵌入向量 x_m ， x_n 和它们之间的相对位置 $m-n$ ：
 $\langle f_q(x_m, m), f_k(x_n, n) \rangle = g(x_m, x_n, m-n)$

二维场景

在二维场景下，定义为

$$f_q(x_m, m) = (W_q x_m) e^{im\theta} = q_m e^{im\theta}$$

$$f_k(x_n, n) = (W_k x_n) e^{in\theta} = k_n e^{in\theta}$$

$$g(x_m, x_n, m-n) = \text{Re} \left[(W_q x_m) (W_k x_n)^* e^{i(m-n)\theta} \right]$$

其实和无位置编码的 Transformer 架构比起来的区别只是增加了 $e^{im\theta}$ 项

然后对 QK^T （即求内积）= g 的推导如下所示：

● 方法一：暴力推导

$$\begin{aligned}
 f_q &= (w_q x_m) e^{im\theta} = q_m e^{im\theta} \quad \text{其中, } q_m = q_m' + i q_m'' \quad (= \text{二维向量虚数表示}) \\
 q_m &= \begin{pmatrix} w_q^{11} & w_q^{12} \\ w_q^{21} & w_q^{22} \end{pmatrix} \cdot \begin{pmatrix} x_m^1 \\ x_m^2 \end{pmatrix} = \begin{pmatrix} q_m^1 \\ q_m^2 \end{pmatrix} \\
 f_k &= (w_k x_n) e^{in\theta} = k_n e^{in\theta} \\
 \text{欧拉公式: } e^{i\theta} &= \cos\theta + i\sin\theta \Rightarrow \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix} \\
 \text{因此 } q_m e^{im\theta} &= (q_m' + i q_m'') (\cos m\theta + i \sin m\theta) \\
 &= (q_m^1 \cos m\theta - q_m^2 \sin m\theta) + i (q_m^2 \cos m\theta + q_m^1 \sin m\theta) \\
 &= \begin{pmatrix} q_m^1 \cos m\theta - q_m^2 \sin m\theta \\ q_m^2 \cos m\theta + q_m^1 \sin m\theta \end{pmatrix} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} q_m^1 \\ q_m^2 \end{pmatrix} \\
 \text{同理: } k_n e^{in\theta} &= \begin{pmatrix} \cos n\theta & -\sin n\theta \\ \sin n\theta & \cos n\theta \end{pmatrix} \cdot \begin{pmatrix} k_n^1 \\ k_n^2 \end{pmatrix} \\
 \text{求内积: } f_q \cdot f_k &\Rightarrow (q_m e^{im\theta})^T \cdot k_n e^{in\theta} \\
 &= (q_m^1, q_m^2) \cdot \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} \cos n\theta & -\sin n\theta \\ \sin n\theta & \cos n\theta \end{pmatrix} \begin{pmatrix} k_n^1 \\ k_n^2 \end{pmatrix} \\
 &= (q_m^1, q_m^2) \underbrace{\begin{pmatrix} \cos(m-n)\theta & -\sin(m-n)\theta \\ \sin(m-n)\theta & \cos(m-n)\theta \end{pmatrix}}_{\substack{|| \\ k_n e^{i(m-n)\theta}}} \begin{pmatrix} k_n^1 \\ k_n^2 \end{pmatrix} \\
 &= \text{Re}[q_m k_n^* e^{i(m-n)\theta}] \\
 &= \text{Re}[(w_q x_m) (w_k x_n)^* e^{i(m-n)\theta}]
 \end{aligned}$$

● 方法二：利用旋转矩阵的性质

对于 $f_q = q_m e^{im\theta}$ 可以理解 q_m 逆时针旋转的角度为 $m\theta$ ，即 $M(m\theta)$ 而 $f_k = k_n e^{in\theta}$ 可以理解 k_n 逆时针旋转的角度为 $n\theta$ ，即 $M(n\theta)$

那么两者内积 $\langle f_q, f_k \rangle = \langle M(m\theta)q_m, M(n\theta)k_n \rangle = q_m^T M(m\theta)^T M(n\theta)k_n = q_m^T M(m\theta)M(n\theta)k_n = q_m^T M((n-m)\theta)k_n$ 之后仿照证明方法 1 也可得到 $g = \langle f_q, f_k \rangle = q_m^T M((n-m)\theta)k_n$

扩展至多维场景

实际的词嵌入维度 (d_{model}) 一般都不是二维的，不过处理的方式也不复杂，是**两两一分组**，每一组还是上面二维场景下的操作。

$$\begin{pmatrix} \cos m\theta_0 & -\sin m\theta_0 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_0 & \cos m\theta_0 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_1 & -\sin m\theta_1 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_1 & \cos m\theta_1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2-1} & -\sin m\theta_{d/2-1} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2-1} & \cos m\theta_{d/2-1} \end{pmatrix} \begin{pmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ \vdots \\ q_{d-2} \\ q_{d-1} \end{pmatrix}$$

对于此，因为有很多零参与计算会很浪费，优化的方法是：

$$\begin{pmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ \vdots \\ q_{d/2-1} \\ q_{d/2} \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_0 \\ \cos m\theta_0 \\ \cos m\theta_1 \\ \cos m\theta_1 \\ \vdots \\ \cos m\theta_{d/2-1} \\ \cos m\theta_{d/2-1} \end{pmatrix} + \begin{pmatrix} -q_1 \\ q_0 \\ -q_3 \\ q_2 \\ \vdots \\ -q_{d/2-1} \\ q_{d/2-2} \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_0 \\ \sin m\theta_0 \\ \sin m\theta_1 \\ \sin m\theta_1 \\ \vdots \\ \sin m\theta_{d/2-1} \\ \sin m\theta_{d/2-1} \end{pmatrix}$$

即对对应的元素相乘相加，这样做在代码里面可以用 `arrange` 函数处理了。

此外，对 RoPE 不是对所有维度都用同一个 进行旋转，而是给 **不同维度分配不同的旋转速度（频率）**。将 d_{model} 维的向量两两分组，共有 $\frac{d_{model}}{2}$ 个组。第 i 组 (i from 0 to $\frac{d_{model}}{2} - 1$) 的旋转角度 θ_i 定义为： $\theta_i = 10000^{(-\frac{2i}{d_{model}})}$

这里底数设置成 10000 是 **经验数值**，针对 4096 这类维度绰绰有余，实际上如果把底数设计的更大一些，比如 50000，会有更好的 **外推性**，因为表示的频率范围会扩大。

直观解释

由旋转矩阵的性质可知，原始 QK 经过内积之后并不会改变原始绝对值大小，

Rope 编码最后结果 $q_m^T M((n-m)\theta) k_n$ 里有一项是 $(n-m)\theta$ ，这就说明了每一项注意力计算结果会和两个 token 的 **相对位置 $n-m$ 有关系**。

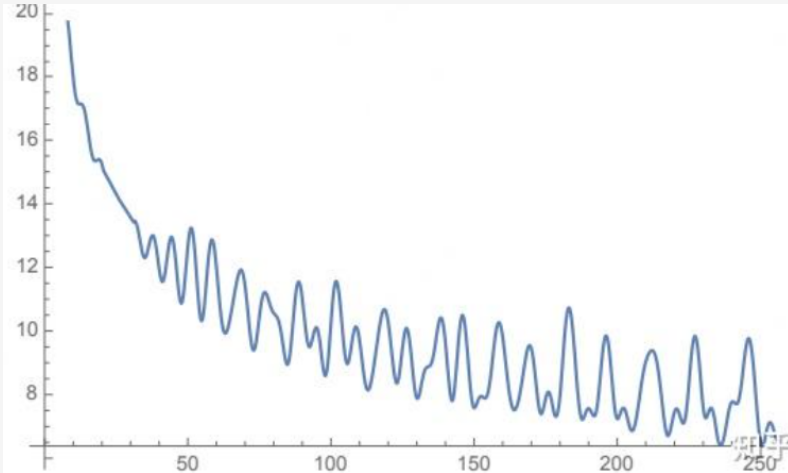
直观上，位置离的越近的两个 token 按理说关联应该越强，比如“锦瑟无端五十弦”和后面一句“一弦一柱思华年”关系比较紧密，但和“望帝春心托杜鹃”逻辑性可能没那么紧密，token 关联就小一点，自注意力机制结果也应该小一些。经过位置编码的结果也类似，极端情况下， $n=m$ 即 $n-m=0$ 两者同一个位置的时候， $\cos(n-m)\theta$ 就是 1， $-\sin(n-m)\theta = 0$ ，注意力最强。

旋转位置编码的远程衰减性

直观上想，距离越远的 token 之间越不相干，也就是 **注意力分数比较低**，即 QK^T 值比较低

我们知道由于 QK^T 内积运算的结果包括周期函数（即 $\cos\theta$ 和 $\sin\theta$ ），其最终求和的结果也一定是周期函数，而我们这里所讨论的远程衰减性实际上指一个周期就已经很长了，可能达到上万，而在这段区域内，内积的值是 **振荡衰减**的，所以我们称为远程衰减性。

证明有些复杂，可见：<https://zhuanlan.zhihu.com/p/647109286>



RoPE homework

```
class RoPE(nn.Module):
    def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None):
        super().__init__()
        if d_k % 2 != 0:
            raise ValueError("d_k must be even")
        self.theta = theta # 这个是 RoPE 的底数超参数，不是直接的角度
        self.d_k = d_k # d_k 就是 d_model，即嵌入之后的稠密向量，它必须为偶数
        self.max_seq_len = max_seq_len
        self.device = device
        # 计算频率
        freqs = 1.0 / (self.theta ** (torch.arange(0, self.d_k, 2).float() / self.d_k))
        # 记录每个 token 的位置信息
        positions = torch.arange(self.max_seq_len)
        # 计算正弦和余弦
        sinusoids = torch.outer(positions, freqs) # outer 是外积，即每个位置都与每个频率相乘
        self.register_buffer("cos_cache", sinusoids.cos(), persistent=False) # 利用
        # register_buffer 表示这是固定的，不需要学习
        self.register_buffer("sin_cache", sinusoids.sin(), persistent=False)

    def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor:
        # 这里的 x 是输入的稠密向量，token_positions 是 token 的位置信息
        cos = self.cos_cache[token_positions]
        sin = self.sin_cache[token_positions]

        cos = cos.unsqueeze(0)
        sin = sin.unsqueeze(0)

        x1 = x[..., 0::2] # 偶数位置
        x2 = x[..., 1::2] # 奇数位置

        output1 = x1 * cos - x2 * sin # 偶数位置乘以 cos，奇数位置乘以 sin
        output2 = x1 * sin + x2 * cos # 偶数位置乘以 sin，奇数位置乘以 cos
        out = torch.stack([output1, output2], dim=-1) # [batch, seq_len, d_k//2, 2]
        out = out.flatten(-2) # [batch, seq_len, d_k]
        return out
```

薇言大义 3.1

Rope 旋转位置编码是很重要的思想，主播在面试中也遇见过相关的问题。

3.4.4. 缩放点积注意力 scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3.5)$$

- **定义：**查询向量 Q 和所有词的键向量 K 进行点积 (Dot-Product) 运算。
- **为什么？**点积是衡量两个向量相似度的一种常用方法。如果 Q 和某个 K 的方向很接近，它们的点积就会很大，代表**相关性高**。 K^T 表示对 Key 矩阵进行转置，是为了让矩阵乘法能够顺利进行。
- **得到什么？**一个**注意力分数** (Attention Score) 矩阵。这个矩阵的每一行表示一个词的 Query，每一列表示它对句子中其他词的原始注意力分数。

Keynote 3.1**为什么需要缩放**

- **问题：**当 d_k (Key 向量的维度) 比较大时， $Q \cdot K$ 的点积结果的**方差**也会变大，这意味着点积的数值可能会非常大或非常小。
- **后果：**如果数值进入 Softmax 函数时过大，Softmax 的输出会趋近于“硬性”的 one-hot 分布 (比如 $[0, 0, 1, 0, 0]$)。这意味着梯度会变得极小 (**梯度消失**)，导致模型在训练时很难学习到有效的参数。
- **解决方案：**将点积结果除以 $\sqrt{d_k}$ 。论文作者证明，这样做可以使得点积结果的方差保持在 1 左右，从而避免了上述问题，让训练过程更加稳定。

Keynote 3.2**Mask 机制**

对于有些我们不希望产生注意力的 key，可以使用 mask 矩阵，在对应位置标上“False”或者其他的标识用来表示这个位置**不要产生注意力**，之后在标记“False”的位置填充替换为“ $-\infty$ ”，这样做的好处就是之后做 softmax 归一化时可以直接 $e^{-\infty} = 0$ ，忽略掉这一部分的注意力。

e.g. $Q^T K$ 相乘后的形状为 $R^{n \times m}$ ，那么 mask 矩阵的形状也应该是 $R^{n \times m}$

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \otimes \begin{pmatrix} \text{True} & \text{True} & \text{False} \\ \text{True} & \text{False} & \text{True} \\ \text{True} & \text{False} & \text{True} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & -\infty \\ a_{21} & -\infty & a_{23} \\ a_{31} & -\infty & a_{33} \end{pmatrix}$$

scaled dot-product attention hw

```
def softmax(x: torch.Tensor, dim: int) -> torch.Tensor:
    x_max = x.max(dim=dim, keepdim=True)[0]
    x_exp = torch.exp(x - x_max)
    return x_exp / x_exp.sum(dim=dim, keepdim=True)
```

```
class ScaledDotProductAttention(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, Q: torch.Tensor, K: torch.Tensor, V: torch.Tensor, mask:
        ↪ torch.Tensor | None = None):
        d_k = Q.shape[-1]
        scores = torch.matmul(Q, K.transpose(-2, -1)) / torch.sqrt(torch.tensor(d_k))
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9) # 如果 mask 为 0, 则将对对应位置的
            ↪ score 设置为-1e9
        attn_weights = torch.softmax(scores, dim=-1) # 对 key 这一维度进行 softmax 归一化
        return torch.matmul(attn_weights, V) # 将 attn_weights 与 value 相乘得到最终的输出
```

3.4.5. 多头因果注意力机制

多头注意力的核心思想：从多个不同的子空间中学习信息，让模型能够共同关注来自不同位置、不同维度的信息。

主要流程：原始的输入形状是 (batch_size, seq_len, d_model)

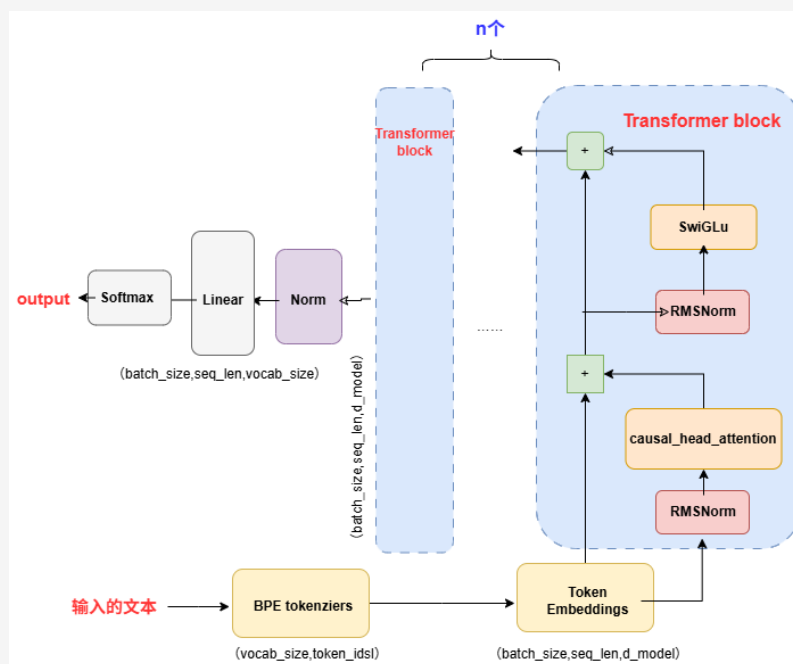
1. 对于输入，要先用 W_q, W_k, W_v 线性变换得到 q, k, v
2. 如果有 n_heads 个头，就把最后一个维度切分成 n_heads 份， q, k, v 每一部分都切分成 d_model/n_heads 的维度，
3. 对于每个头，对于 q, k, v 都去做 attention 操作。
4. 最后把所有的头按照最后一个维度 concat 起来，然后做一次线性变换。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \text{ for } \text{head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (3.6)$$

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x) \quad (3.7)$$

3.5. Transformer 总结篇

这一小节是第一章最重要的部分，有别于原始论文里提出的模型流程，它从整体上讲解了一些新的主流大模型的底层 Transformer 架构（主要就是 Llama，推荐直接阅读 Llama 的源代码，和作业重合度很高）。



3.5.1. 输入部分

模型的输入是**大规模**的文本语料库。其核心训练目标是学习文本的统计规律，从而能够**基于给定的上文 (context)**，**准确地预测下一个词元 (token)**。通过这种方式，模型可以生成连贯、流畅的文本。

3.5.2. BPE tokenizer(预分词)

原始文本是由字符组成的字符串，无法直接输入神经网络。Tokenizer 的作用是将**原始文本字符串转换为一个整数序列 (Integer IDs)**。我们使用的 BPE (Byte-Pair Encoding) 是一种亚词 (subword) 分词算法，它通过以下步骤工作：

1. **初始化**: 词典由所有单个字节 (0-255) 组成。
2. **迭代合并**: 在训练语料中，不断寻找出现**频率最高的相邻字节对 (或已合并的 token 对)**，将它们合并成一个新的 token，并加入词典。
3. **最终**: 经过指定次数的合并后，形成最终的词典 (vocabulary)。Tokenizer 利用这个词典和合并规则，将任意文本切分成一系列 token，并映射为其在词典中的**唯一整数 ID**。

3.5.3. Embedding (词嵌入)

预分词得到的结果维度是 $vocab_size$ ，即词典的大小，一般会用**独热码**来表示，这样做的稀疏向量太多，太浪费计算资源。为了把稀疏向量映射为**稠密向量**，维度一般是 $d_model=512$ 。一个词嵌入层 (embedding layer) 的作用就是把这些离散的、没有语义的整数 ID，转换为连续的、包含语义信息的**向量**。比如，“猫”的向量可能和“狗”的向量在某种“动物”维度上比较接近，而和“汽车”的向量相距较远。每个嵌入层接收形状为 $(batch_size, sequence_length)$ 的整数张量，并生成

形状为 (batch_size, sequence_length, d_model) 的向量序列。d_model 代表输出维度，越大代表语义信息越丰富，模型越强大。

3.5.4. Position encoding (位置编码)

在原始论文中，位置编码被放在 embedding 之后，但在作业和主流大模型实现里面，位置编码就直接放在多头注意力那里来只处理 query 和 key 了。

位置编码是由于仅仅只是词嵌入虽然能够保留语义，但不能很好地表示每个 token 的位置（索引）信息。位置编码包括绝对位置编码和相对位置编码两大类，一般相对位置编码的效果会更好，作业中使用的方法是 rope 旋转位置编码。

3.5.5. Transformer block

作业中的 Transformer block 由 RMSnorm、Causal 多头注意力机制、SwiGlu 这几部分组成。

3.5.6. RMSnorm

简化版的层归一化

$$RMSnorm(x) = \frac{x}{RMS(x)} \quad RMS(x) = \sqrt{\frac{1}{D} \sum_{i=1}^D x_i^2} = \sqrt{mean(x^2)}$$

值得注意的是，对于从 embedding 层输入过来的 (batch_size, sequence_length, d_model (或者 hidden_dim))，归一化的目标 x 是最后一维的 d_model。

3.5.7. Causal Multi-head self-attention (多头注意力机制)

- 多头机制 (Multi-Head)**: 将 d_model 维的 Q, K, V 向量按照 n_heads 分成 d_model//n_heads 多个头 (head)，让每个头关注输入序列的不同方面，增强了模型的表达能力，最后再拼接在一起。
- 旋转位置编码 (RoPE)**: 为了注入位置信息，RoPE 直接作用于 Q 和 K 向量，通过旋转它们来编码其绝对位置，并使注意力得分自然地依赖于它们的相对位置。V 向量不参与位置编码。
- 因果 Mask (Causal Masking)**: 在自回归的文本生成任务中，模型在预测位置 i 的词元时，只能关注到位置 i 及之前的所有词元，不能“偷看”未来的信息。这是通过在注意力得分矩阵上应用一个上三角遮罩实现的。”
- 还有在点积缩放运算里面使用 mask 来给 e 的指数赋值 $-\infty$ ，方便运算。

3.5.8. SwiGLU

1. 原始 Transformer 的 FFN (基于 ReLU):

- 结构**: 包含两个线性变换层，中间夹着一个 ReLU 激活函数。
- 公式**: $FFN(x) = \text{Linear}_2(\text{ReLU}(\text{Linear}_1(x)))$
- 维度**: 中间隐藏层 (Linear_1 的输出) 的维度 d_ff 通常是输入维度 d_model 的 4 倍 ($d_{ff} = 4 * d_{model}$)。

2. SiLU (Sigmoid Linear Unit) / Swish 激活函数:

- 定义**: $SiLU(x) = x \cdot \sigma(x) = \frac{x}{1+e^{-x}}$ 其中， $\sigma(x) = \frac{1}{1+e^{-x}}$ 代表 sigmoid 激活函数。

- **特点:** 类似于 ReLU, 但在零点附近是平滑的, 这有助于缓解梯度消失问题, 并允许负数输入通过。

3. 门控线性单元 (Gated Linear Units, GLUs):

- **定义:** $GLU(x, W1, W2) = (W1x)W2x$ 。
- σ 代表 Sigmoid 激活函数。
- $\odot$ 代表元素级乘法 (element-wise multiplication)。
- **优点:** 建议通过提供一个“线性路径”来“减少深度架构中的梯度消失问题”, 同时保留非线性能力。

3.5.9. 最终输出

从几个 Transformer block 输出的形状还是 $(batch_size, sequence_length, d_model)$, 此后继续归一化, Linear, Softmax。

归一化自然就是稳定梯度

Linear 是将 $(batch_size, sequence_length, d_model) \rightarrow (batch_size, sequence_length, vocab_size)$

之后利用 softmax 进行计算 vocab_size 里面每一个词的概率, 选择最大的词作为下一个词。

微言大义 3.2

这一部分内容可以复习或者一开始就看一看, 有一个整体的认识。

前面的章节讨论的是 Transformer 模型的架构和理论知识,这一章节侧重讲解 Transformer 模型的训练方法(事实上对于绝大多数深度学习的训练都适用)。主要涉及:

- **损失函数**: 需要定义交叉熵损失函数
- **优化器**: 需要定义 AdamW 优化器来最小化损失
- **训练循环**: 需要构建完整的基础设施来加载数据、保存检查点和管理训练过程

4.1. 信息论前置知识 (简易版)

4.1.1. 信息 (Information)

它用来**度量事件发生所消除的不确定性 (Uncertainty) 的大小**。

1. 直观理解

- “明天太阳会照常升起”: 这句话包含的信息量很小。因为它几乎是必然发生的,没有消除我们什么不确定性。
- “明天北京会下雪 (假设在夏天)”: 这句话包含的信息量极大。因为它是一个极小概率事件,一旦发生,就极大地消除了我们的不确定性。

2. 量化定义: 自信息 (Self-Information) 信息论使用“自信息”来量化单个事件发生时所提供的信息量。一个事件 x 的自信息量 $I(x)$ 定义为:

$$I(x) = -\log_b(p(x))$$

- $p(x)$: 事件 x 发生的概率。
- $\log_b$: 对数函数。底数 b 的选择决定了信息量的单位。
 - $b=2$: 单位是 **比特 (bit)**, 这是最常用的单位, 对应于二进制世界。
 - $b=e$: 单位是 **奈特 (nat)**。
 - $b=10$: 单位是 **哈特利 (hartley)**。
- 负号: 因为概率 $p(x)$ 在 $[0, 1]$ 之间, 它的对数是小于等于 0 的。加上负号可以确保信息量是一个非负数, 这符合我们的直觉。

3. 示例

- 假设我们抛一枚均匀的硬币:

4.1 信息论前置知识 (简易版)	51
4.1.1 信息 (Information)	51
4.1.2 熵 (Entropy)	52
4.1.3 熵的相关扩展概念	52
4.1.4 一个生动的比喻: 猜数字游戏	54
4.2 交叉熵 (Cross-Entropy) 损失与 KL 散度	55
4.2.1 KL 散度 (相对熵)	55
4.2.2 交叉熵	55
4.2.3 交叉熵损失函数	55
4.2.4 实际代码的应用	56
4.2.5 具体在 Transformer 中的应用	57
4.2.6 Perplexity (困惑度)	58
4.3 Optimizer: 优化器	59
4.3.1 SGD (随机梯度下降)	59
4.3.2 动量法 (Momentum)	60
4.3.3 AdaGrad	61
4.3.4 RMSProp	62
4.3.5 Adam	63
4.3.6 AdamW	64
4.4 自适应学习率调整 (学习率调度器)	64
4.4.1 StepLR (阶梯式下降)	65
4.4.2 CosineAnnealingLR (余弦退火)	66
4.4.3 ReduceLROnPlateau (遇到平台期就减速)	66
4.4.4 Warmup (预热)	68
4.5 梯度裁剪	69

- “正面朝上”的概率 $p(\text{正面}) = 0.5$ 。

它提供的信息量 $I() = -\log_2(0.5) = -\log_2(2^{-1}) = -(-1) = 1\text{bit}$ 。

- 同样，“反面朝上”提供的信息量也是 1 bit。

这告诉我们，要确定一个硬币的正反面，我们需要 1 bit 的信息。

4.1.2. 熵 (Entropy)

如果我们想衡量的是一个**系统 (随机变量) 的整体不确定性**，而不是单个事件，那就要用到“熵”的概念。

1. 直观理解

- 熵是**信息量的期望值 (数学期望)**。它衡量了一个随机变量所有可能结果的平均不确定性。
- **一个系统越混乱、越不可预测，它的熵就越高。**
- **一个系统越稳定、越可预测，它的熵就越低。**

2. 量化定义

- 对于一个离散随机变量 X ，它有多种可能的取值 $\{x_1, x_2, \dots, x_n\}$ ，对应的概率为 $\{p(x_1), p(x_2), \dots, p(x_n)\}$ 。那么，这个随机变量 X 的熵 $H(X)$ 定义为：
- $H(X) = E[I(X)] = \sum_{i=1}^n p(x_i) I(x_i) = -\sum_{i=1}^n p(x_i) \log_b(p(x_i))$

3. 示例

- 比较两枚硬币的熵：
 - 均匀硬币 (Fair Coin): $p(\text{正面})=0.5, p(\text{反面})=0.5$
 - 不均匀硬币 (Biased Coin): $p(\text{正面})=0.9, p(\text{反面})=0.1$
 - 两面都是正面的硬币 (Two-headed Coin): $p(\text{正面})=1, p(\text{反面})=0$
- 均匀硬币 (Fair Coin): $p(\text{正面})=0.5, p(\text{反面})=0.5$
 $H(X) = -[0.5 \times \log_2(0.5) + 0.5 \times \log_2(0.5)] = 1\text{bit}$
- 不均匀硬币 (Biased Coin): $p(\text{正面})=0.9, p(\text{反面})=0.1$
 $H(X) = -[0.9 \times \log_2(0.9) + 0.1 \times \log_2(0.1)] \approx 0.469\text{bit}$
- 两面都是正面的硬币 (Two-headed Coin): $p(\text{正面})=1, p(\text{反面})=0$
 $H(X) = -[1 \times \log_2(1) + 0 \times \log_2(0)] = 0(0 \log 0 = 0)$

4.1.3. 熵的相关扩展概念

理解了熵之后，其他几个重要概念就很容易理解了，它们描述了多个随机变量之间的关系。

1. 联合熵 (Joint Entropy)

- 衡量**两个或多个随机变量共同**的不确定性。对于两个变量 X 和 Y ，其联合熵 $H(X, Y)$ 为：
- $H(X, Y) = -\sum_{x \in X} \sum_{y \in Y} p(x, y) \log_2(p(x, y))$

- 其中 $p(x, y)$ 是 $X=x$ 和 $Y=y$ 同时发生的联合概率。

2. 条件熵 (Conditional Entropy)

- 在**已知一个随机变量 X 的情况下**, **另一个随机变量 Y 剩下的不确定性**。
记为 $H(Y|X)$ 。
- $H(Y|X) = \sum_{x \in X} p(x)H(Y|X = x)$
- 它表示,知道了 X 之后,对 Y 的不确定性还剩下多少。

3. 互信息 (Mutual Information)

- **一个随机变量 X 的信息中,有多少是与另一个随机变量 Y 共享的**。它衡量了两个变量之间的相关性。记为 $I(X; Y)$ 。
- **直观理解**: 知道了 X 之后, Y 的不确定性减少了多少。
- **计算公式**:
 - $I(X; Y) = H(Y) - H(Y|X)$ (**Y 的总不确定性 - 知道 X 后 Y 剩下的不确定性**)
 - $I(X; Y) = H(X) - H(X|Y)$ (**对称的**)
 - $I(X; Y) = H(X) + H(Y) - H(X, Y)$

Venn 图关系你可以把熵想象成一个集合,那么这些概念的关系就非常清晰了:

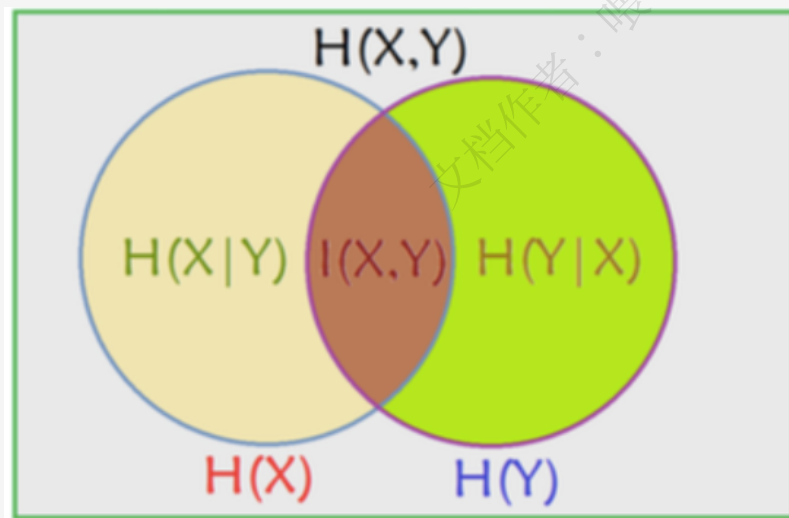


图 4.1: 信息论概念的 Venn 图关系

- 左边的圆圈是 $H(X)$
- 右边的圆圈是 $H(Y)$
- 两个圆圈的重叠部分是**互信息 $I(X; Y)$**
- $H(X)$ 中不重叠的部分是**条件熵 $H(X|Y)$**
- $H(Y)$ 中不重叠的部分是**条件熵 $H(Y|X)$**
- 整个两个圆圈覆盖的区域是**联合熵 $H(X, Y)$**

4.1.4. 一个生动的比喻: 猜数字游戏

这个比喻可以把所有概念串起来。

- **游戏**: 我从 1 到 8 之间想一个数字, 你来猜。
- **熵 $H(X)$** : 在游戏开始前, 这个数字是什么? 你完全不知道, 有 8 种可能性, 每种可能性概率为 $1/8$ 。这个系统的总不确定性是 $H(X) = -\sum_{i=1}^8 \frac{1}{8} \log_2(\frac{1}{8}) = \log_2(8) = 3\text{bits}$ 。这恰好是你用“二分法”猜中这个数字所需的最少问题数 (例如: “比 4 大吗?” “比 6 大吗?” “是 7 吗?”)。
- **信息 $I(x)$** : 我告诉你答案是“5”。这个具体事件提供的信息量是 $I("5") = -\log_2(1/8) = 3\text{bits}$ 。一旦你知道了这个信息, 所有不确定性都消除了。
- **互信息 $I(X;Y)$** : 现在, 我不直接告诉你答案, 而是给你一个提示 **Y**: “这个数字是奇数”。
 - 这个提示 **Y** 本身也有不确定性 (可能是奇数或偶数), 它的熵是 **$H(Y)=1\text{bit}$** 。
 - 这个提示 **Y** 给你提供了多少关于 **X** 的信息? 这就是互信息。知道了它是奇数, 可能性从 1,2,3,4,5,6,7,8 缩小到了 1,3,5,7。不确定性大大降低。
- **条件熵 $H(X|Y)$** : 你知道了“数字是奇数” (**Y**) 之后, 对数字 (**X**) 还剩下多少不确定性?
 - 现在只剩下 4 种可能性 1,3,5,7, 每种概率为 $1/4$ 。
 - 剩下的不确定性 (条件熵) 是 $H(X|Y = \text{"奇数"}) = -\sum_{i \in \{1,3,5,7\}} \frac{1}{4} \log_2(\frac{1}{4}) = -\log_2(4) = 2\text{bits}$ 。
 - 这意味你还需要问 2 个问题才能猜到。
- **关系验证**:
 - $I(X;Y) = H(X) - H(X|Y) = 3 - 2 = 1\text{bit}$
 - 这说明, “数字是奇数” 这个提示, 为你提供了 1 bit 的信息。

中文名称	描述	关键点
自信息	度量单个具体事件发生所消除的不确定性。	概率越小, 信息量越大。
熵	度量整个系统 (随机变量) 的平均不确定性。	信息量的数学期望。系统越混乱, 熵越高。
联合熵	度量两个或多个系统共同的不确定性。	把多个变量看成一个整体。
条件熵	在已知一个变量后, 另一个变量剩下的不确定性。	$H(Y X)$, 知道 X 后 Y 还有多乱。
互信息	两个变量共享的信息量, 即相关性程度。	$I(X;Y)$, 知道 X 能帮我们消除 Y 多少不确定性。

4.2. 交叉熵 (Cross-Entropy) 损失与 KL 散度

参考资料 4.1

交叉熵损失函数: <https://blog.csdn.net/SongGu1996/article/details/99056721>

建议先熟悉信息论中的基本概念, 如熵、相对熵、交叉熵等。

4.2.1. KL 散度 (相对熵)

假设随机变量 X 的真实概率分布为 $P(X)$, 而我们在处理实际问题时使用了一个近似的分布 $Q(X)$ 来进行建模。由于我们使用的 $Q(X)$ 是而不是真实的, 所以我们在具体化的取值时需要一些**附加的信息**来抵消分布不同造成的影响。我们需要的平均附加信息量可以使用**相对熵**, 或者叫**KL 散度 (Kullback-Leibler Divergence)**来计算, KL 散度可以用来**衡量两个分布的差异**:

$$D_{KL}(P||Q) = -\sum_{i=1}^n P(x_i) \log Q(x_i) - (-\sum_{i=1}^n P(x_i) \log P(x_i)) = \sum_{i=1}^n P(x_i) \log \frac{P(x_i)}{Q(x_i)}$$

KL 散度的性质:

1. KL 散度不是一个对称量 $D_{KL}(P||Q) \neq D_{KL}(Q||P)$
2. KL 散度永远大于等于 0 (因为最好也就是 $Q(X) = P(X)$, 此时额外信息量为 0)

4.2.2. 交叉熵

实际上就是 KL 散度的第一项值

$$D_{KL}(P||Q) = -\sum_{i=1}^n P(x_i) \log Q(x_i) - (-\sum_{i=1}^n P(x_i) \log P(x_i)) = -H(P) + H(P, Q)$$

$$H(P, Q) = H(P) + D_{KL}(P||Q) = -\sum_{i=1}^n P(x_i) \log Q(x_i)$$

对于真实分布 $P(X)$ 来讲, 它的熵 $H(P)$ 是一个固定值, **真正决定 KL 散度值的还是交叉熵**。

我们训练神经网络的目标就是希望近似分布 Q 逼近真实分布 P 。

4.2.3. 交叉熵损失函数

二分类

只有两类, 设:

- 真实标签为 $y \in \{0, 1\}$
- 模型输出的预测概率为 $\hat{y} \in [0, 1]$

则交叉熵损失函数为:

$$L = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

```
loss_fn = nn.BCELoss()
loss = loss_fn(y_pred, y_true)
```

多分类

设:

- 真实标签 $y \in \{0, 1, \dots, K-1\}$ 是类别索引
- 模型预测的是一个概率分布向量 $\hat{y} = [p_1, \dots, p_K]$

一般来讲, p_i 是要经过 softmax 处理的, 即 $p_i = \frac{e^{x_i}}{\sum_{k=0}^{K-1} e^{x_k}}$

多分类交叉熵公式为:

$$\text{CrossEntropy Loss} = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

```
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(logits, targets) # 注意 torch 里面会自动计算 softmax, 所以不需要人为再去
    ↪ 计算一遍了
```

4.2.4. 实际代码的应用

Exp-normalize 技巧 + LogSoftmax + NLLloss

Exp-normalize 技巧

Exp-normalize 技巧的目标是为了实现后续 softmax 计算数值的稳定性, 避免出现上溢或者下溢的情况。具体来说,

先将输入减去向量最大值: $z = x - \max(x)$

这样做, 所有的 $x_i \subseteq (-\infty, 0]$, 也就是 e^x 的值域只会在 y 轴左半部分, 这样做的好处是:

- 最大的值变为 0, 避免其可能原始数值很大, 导致上溢指数爆炸;
- 至少有一个 $\exp(0)=1$, 防止 softmax 里的分母为 0 下溢。
- 这种变换不改变最终 softmax 的输出, 因为指数任意平移不改变比值。

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}} = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}} = \text{softmax}(z)_i$$

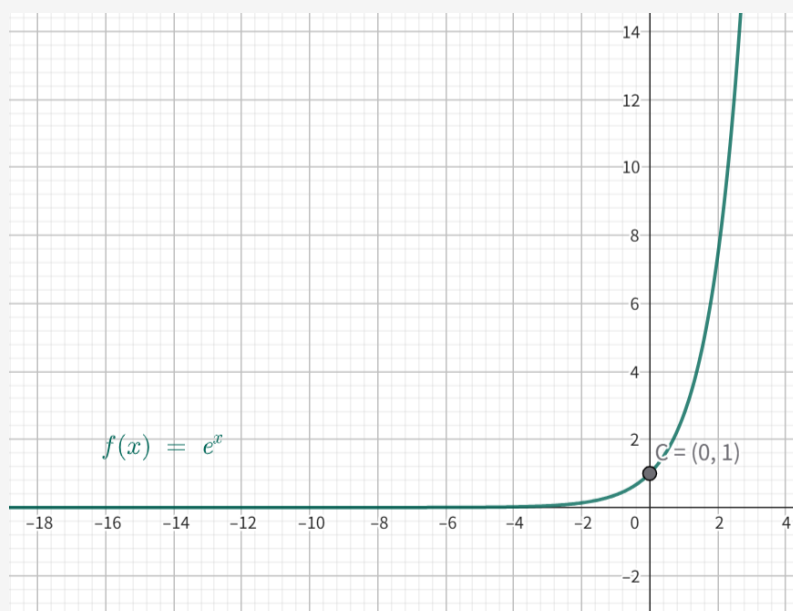


图 4.2: Exp-normalize 技巧

LogSoftmax

实际上就是把 $\log$ 和 softmax 两种运算给组合起来了。

softmax 运算:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

LogSoftmax 运算:

$$\log \frac{e^{x_i}}{\sum_j e^{x_j}} = x_i - \log \sum_j e^{x_j}$$

直接使用 Logsoftmax 而不是分步使用 $\log$ 和 softmax 的好处在于:

- 减少计算开销, 直接使用了 x_i 等价先 $\text{softmax}(x_i)$ 再 $\log$
- 直接使用 Logsoftmax 计算梯度会更加稳定

NLLloss 负对数似然损失函数

$$\text{公式: } \text{nllloss} = -\frac{1}{N} \sum_{k=1}^N P(y_k) \cdot (\log_softmax)$$

其实就是 Logsoftmax 后续的补充操作, 使得结果与 Crossentropy 结果一致。

```
logp = F.log_softmax(logits, dim=1) # 计算 log 概率: shape = [batch, K],
loss = F.nll_loss(logp, targets)     # targets 是 [batch] 维度的真实类索引
```

这两行代码等价于直接用 `crossentropy`

```
loss = F.cross_entropy(logits, targets)
```

4.2.5. 具体在 Transformer 中的应用

Transformer 作为一个**自回归的语言模型**, 其核心任务是**预测下一个词**。给定一个词序列的前 i 个词 $x_{1:i}$, 它需要预测出第 $i+1$ 个词 x_{i+1} 的概率分布 $p_\theta(x_{i+1}|x_{1:i})$ 。这里的 θ 代表模型的所有参数 (权重)。

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^m -\log p_\theta(x_{i+1} | x_{1:i})$$

$\ell(\theta; D)$ 就是这个损失函数。我们来把它拆解一下:

- $|D|$ 是数据集中序列的数量。
- m 是我们考虑的序列长度。
- $\sum_{x \in D} \sum_{i=1}^m$ 这个双重求和符号代表我们要计算数据集中每一条序列里, 从第 1 个位置到第 m 个位置的每一次预测的损失, 然后把它们全部加起来。
- $-\log p_\theta(x_{i+1}|x_{1:i})$ 这是整个损失函数的核心。
- $p_\theta(\dots)$ 是模型预测的、下一个词恰好是正确答案 x_{i+1} 的概率。
- **我们的目标是让这个概率尽可能高, 理想情况下趋近于 1。**
- $\log$ 函数在 $(0, 1]$ 区间内是单调递增的, 所以让 p 最大化等同于让 $\log(p)$ 最大化。
- 加上一个负号 $-$, 最大化 $\log(p)$ 就变成了最小化 $-\log(p)$ 。这正是机器学习中梯度下降的目标: 最小化损失函数。

- $\frac{1}{|D|m}$ ：最后除以总的预测次数（序列数 × 序列长），这是为了计算平均损失，避免数据集大小对损失值产生影响。

模型在内部并不会直接输出概率，而是先输出一个叫做**logits**的原始数值向量 o_i 。这个向量的维度等于你的词汇表大小（vocab_size），其中每个元素可以看作是对应单词的“得分”或“置信度”，这个得分可以是任意实数（可正可负）。

为了把这些原始得分 o_i 转换成一个合法的概率分布（即所有值都在 0 到 1 之间，且加起来等于 1），我们使用**Softmax**函数。

$$p(x_{i+1}|x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])}$$

这个公式展示了如何计算正确词 x_{i+1} 的概率。

- 分子 $\exp(o_i[x_{i+1}])$ ：对正确词的 logit 分数取指数，确保其为正数。
- 分母 $\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])$ ：对词汇表中所有词的 logit 分数取指数然后求和，这是一个归一化项。
- 两者相除，就得到了正确词 x_{i+1} 的最终概率。

总结一下整个流程：

对于训练数据中的一个序列，在每一个位置 i ：

- Transformer 模型接收前面的序列 $x_{1:i}$ 作为输入。
- 模型输出一个大小为 vocab_size 的 logits 向量 o_i 。
- 通过 Softmax 函数将 o_i 转换为概率分布。
- 取出真实下一个词 x_{i+1} 对应的概率，取负对数得到该位置的损失 $-\log(p)$ 。
- 将所有位置、所有序列的损失加起来求平均，得到最终的总损失，然后通过反向传播来更新模型参数 θ 以最小化这个损失。

4.2.6. Perplexity(困惑度)

Perplexity(困惑度) 是衡量语言模型预测能力的指标，它反映了模型对测试数据的不确定性。**Perplexity 越低，说明模型对数据的预测越准确；Perplexity 越高，说明模型对数据的预测越不确定。**

Perplexity 的数学定义

Perplexity(PP) 与交叉熵 (Cross-Entropy) 密切相关，数学定义如下：

$$PP(W) = 2^{H(W)}$$

其中， $H(W)$ 是语言模型的交叉熵，计算公式如下：

$$H(W) = -\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i | w_1, w_2, \dots, w_{i-1})$$

- W ：一整段文本
- N ：文本中的单词总数
- $P(w_i | w_1, \dots, w_{i-1})$ ：模型对单词 w_i 在给定前面单词的条件下的预测概率

直观理解

1. 如果模型完美预测每个单词的概率都是 1, 那么 $PP = 1$, 即模型完全不困惑。
2. 如果模型随机猜测 (每个单词概率相等), 那么 PP 会很高, 表示模型的预测不确定。

Perplexity 的意义

衡量语言模型的好坏:

1. 低 Perplexity(PP 低) → 说明模型更擅长预测下一个单词, 效果更好。
2. 高 Perplexity(PP 高) → 说明模型对语言的理解较差, 需要优化。

用于对比不同模型的性能:

- 例如, GPT-3 的 Perplexity 低于 GPT-2, 说明 GPT-3 在相同数据集上的预测能力更强。

优化语言模型的目标:

- 训练过程中, 优化目标通常是**最小化交叉熵 (降低 Perplexity)**, 使模型更擅长语言理解和生成。

4.3. Optimizer: 优化器

参考资料 4.2

通俗易懂理解 (梯度下降) 优化算法: <https://blog.csdn.net/Invokar/article/details/86768571>

优化算法《动手学深度学习》: http://zh.gluon.ai/chapter_optimization/index.html

神经网络中 warmup 策略为什么有效: <https://www.zhihu.com/question/338066667>

Optimizer(优化器) 是深度学习中的一种核心算法, 其主要作用是根据模型的损失函数 (Loss Function) 计算出的梯度 (Gradient), 来指导并更新网络中的参数 (如权重和偏置), 目标是让损失函数的值越来越小, 从而使模型的预测结果越来越准确。

常见的 Optimizer 包括: **SGD、Momentum、Adagrad、RMSprop、Adam** 等。

4.3.1. SGD (随机梯度下降)

SGD 是一般的梯度下降法的近似版本, 传统的梯度下降法是针对**整个训练集**进行求梯度, 而 SGD(mini SGD) 是对训练集的**一小批量**进行求梯度并更新。

公式 (和梯度下降法一样):

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$$

其中:

- θ_t : 第 t 步的参数 (或者权重);

- α : 学习率;
- $\nabla_{\theta} J(\theta_t)$: 损失函数对参数的梯度。 $\nabla_{\theta} J(\theta_t) = \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} f(x_i, \theta)$ 其中 $\mathcal{B}_t$ 代表这个批次

相较于 GD, SGD **计算损耗更少**, 但梯度的估计存在噪声, 导致收敛过程会发生震荡, 并且**不保证能收敛到全局最优点**。

4.3.2. 动量法 (Momentum)

直接利用 SGD 的话, 可能出现相邻两个时刻梯度差距过大, 来回**震荡**, 动量法利用了上一时刻的梯度值, 在 SGD 的基础上引入“动量”, 即**历史梯度的线性组合(加权平均)**, 以缓解震荡、加速收敛。

Keynote 4.1

为什么动量法要叫动量法？

这是从山坡上最快降到谷底这一物理过程上类比得出来的, 实际上叫“惯性法”更加合适。对于梯度法, 它没有考虑历史时刻的速度也就是惯性, 而动量法通过将当前梯度与上一时刻的速度进行加权平均的方式利用了惯性。打个比方, 一个小球从山坡上降到谷底, 梯度法不考虑它的速度, 没到一个地方就计算这个地方的梯度, 那么在接近谷底的陡峭点 (梯度高), 那么小球就直接冲到山谷另一侧了, 而如果我们保留了历史的速度, 在这个位置冲到另一侧时就会由于动量的存在冲的没那么厉害, 减少震荡。

公式:

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

其中:

- v_t : 动量变量, 表示历史梯度的指数加权平均;
- β : 动量衰减系数 (惯性系数), 常取 0.9。

动量法的两大好处

1. 在梯度方向一致时加速:

在山谷的一侧坡上, 梯度方向基本不变 (一直指向谷底)。每一时刻的梯度 (加速度) $J(\theta_t)$ 都会和上一时刻的速度 v_{t-1} 方向**相同**, 速度 v_t 就会不断累积, 变得越来越大, 从而让参数 的更新步伐越来越大, 实现**加速**效果。

2. 在梯度方向震荡时抑制:

在山谷的陡峭两侧, 梯度方向会来回变化 ($J(\theta_t)$ 和 $J(\theta_{t-1})$ 方向相反)。这时, $J(\theta_t)$ 会和 v_{t-1} 中的震荡方向分量相互**抵消**, 使得 v_t 在这个方向上的值变小。这就起到了**抑制震荡**、平滑更新路径的作用。

但相较于 GD、SGD 来说, 动量法需要**额外维护一个与模型参数量等大的动量变量** v_{t-1} , 其空间复杂度为 $O(N)$, N 为模型参数数量。

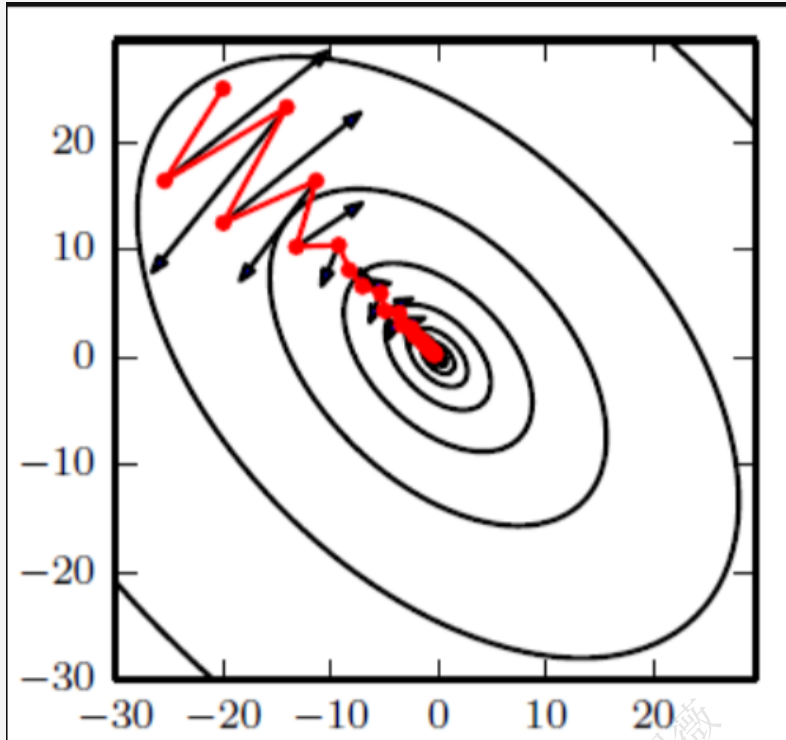


图 4.3: 动量法示意图

4.3.3. AdaGrad

核心思想: 为每一个参数分配**不同的自适应学习率**

像 GD、SGD 这些算法的学习率 α 一般都是固定的,AdaGrad 认为有些参数更新频繁,有些很少更新,不同的参数应根据其**历史梯度**有不同学习率

公式:

$$G_t = G_{t-1} + g_t g_t$$

$$x_t = x_{t-1} - \frac{\alpha}{\sqrt{G_t}} g_t$$

其中:

- g_t : 第 t 步的梯度;
- G_t : 从第 1 步到第 t 步, 梯度平方的累加;
- 表示**逐元素相乘**

e.g.:

假设在二维平面上的梯度 $g=[4,9]$, 初始时 $G=0$, 则 $G_1=[16,81]$. 我们对第一维度进行更新时, 采用 $x_t^{(1)} = x_{t-1}^{(1)} - \frac{\eta}{\sqrt{16}} \times 4$

对第二维度进行更新时, 采用 $x_t^{(2)} = x_{t-1}^{(2)} - \frac{\eta}{\sqrt{81}} \times 9$

从上面我们就可以看出, 当梯度的某一个方向上比较大时, 分母会约束其变小, 而当梯度的某一个方向上比较小时, 分母会令其相对变大。从中其实可以体现出归一化的效果, 有点类似于下图的转换结果:

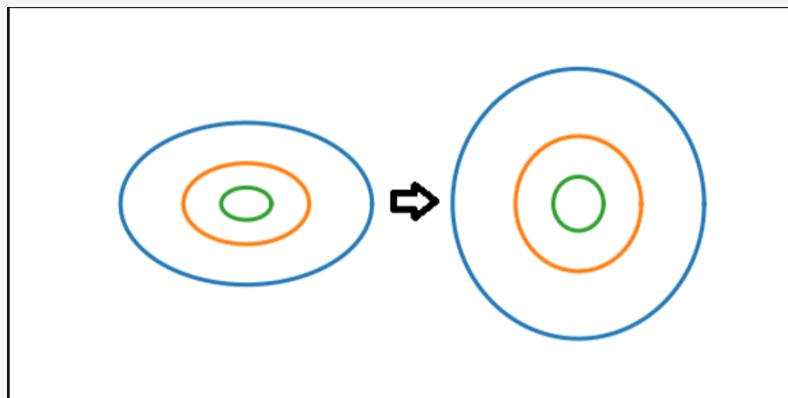


图 4.4: AdaGrad 示意图

AdaGrad 的一个重大缺点是 G_t 是**无上限累计**的, 这样到后期学习率基本上单调递减到 0 了, 就不会训练了。

4.3.4. RMSProp

为了克服 AdaGrad 的缺点,RMSProp 不是直接累加所有历史梯度, 而是使用**滑动窗口式的加权平均**, 只“记住**最近**的梯度行为”。

设梯度为 g_t , 均方梯度为 $E[g^2]_t$:

1. 估计梯度平方的指数平均:

$$E[g^2]_t = \beta \cdot E[g^2]_{t-1} + (1 - \beta) \cdot g_t^2$$

2. 更新参数:

$$\theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{E[g^2]_t + \epsilon}} \cdot g_t$$

其中:

- g_t^2 : 表示梯度按元素平方的值

$$E[g^2]_1 = (1 - \gamma)g_1^2$$

$$E[g^2]_2 = \gamma E[g^2]_1 + (1 - \gamma)g_2^2 = \gamma(1 - \gamma)g_1^2 + (1 - \gamma)g_2^2$$

$$E[g^2]_3 = \gamma E[g^2]_2 + (1 - \gamma)g_3^2$$

$$= \gamma^2(1 - \gamma)g_1^2 + \gamma(1 - \gamma)g_2^2 + (1 - \gamma)g_3^2$$

...

$$E[g^2]_t = (1 - \gamma) \sum_{i=1}^t \gamma^{t-i} g_i^2$$

最近的梯度平方权重**大**,**越早之前的梯度影响越小**(指数级衰减)。

直观理解

1. RMSProp 旨在估计一个“近似**局部**梯度方差”;
2. 如果梯度波动很大(即平方值变化大), 那 $E[g^2]_t$ 就会大;

3. RMSProp 用这个值来**对当前梯度进行缩放**, 让不同参数维度的学习率自适应:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{E[g^2]_t + \epsilon}} \cdot g_t \text{ 这意味着:}$$

- 如果某个参数维度的梯度在近期波动很大 (不稳定), 我们就**减少它的学习率**;
- 如果某个维度的梯度变化小 (较稳定), 就**保持或稍大更新步幅**。

超参数的取值:

- β 取值:
 - 接近 1(如 0.99): 长记忆 (依赖更长的历史)
 - 接近 0(如 0.5): 短记忆 (几步就忘)
 - 通常设置为 0.9: 平衡短期与长期趋势

4.3.5. Adam

Adam 是把之前的优化器的特点都融合起来了, 不仅**看当前位置的梯度**(就像普通 SGD 那样), 还**考虑过去梯度的平均值**(动量) 和**梯度平方的平均值**(像 RMSProp 那样), 并进行**自适应的学习率调整**, 从而在不同参数维度上使用不同的更新步长。

假设我们在训练中第 t 步, 梯度为 g_t 。Adam 的更新公式如下:

1. **一阶矩估计 (动量):**

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

2. **二阶矩估计 (RMS):**

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

3. **偏差校正:**

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

4. **参数更新:**

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

其中:

- θ 为当前模型参数 - g_t 为当前梯度 (损失对参数的导数) - m_t 为梯度的指数加权平均 (动量) - v_t 为梯度平方的指数加权平均 - β_1 控制动量的衰减率, 常设为 0.9 - β_2 控制梯度平方平均的衰减率, 常设为 0.999 - α 代表学习率, 默认 0.001 - ϵ 是防止除以 0 的微小常数, 通常设为 10^{-8}

Keynote 4.2

为什么要进行偏差校正？ 由于 v_t 是从 $v_0 = 0$ 开始的，所以在前几步迭代时， v_t 的值明显偏小（因为它平均值主要还在积累阶段）。例如第 1 步： $v_1 = (1 - \beta)g_1$ 这个值太小了，因为历史还没积累。于是，我们用一个“修正系数”将它还原为更合理的期望值： $\hat{v}_t = \frac{v_t}{1 - \beta^t}$

其中：

- v_t 是带偏的动量估计；
- $\hat{v}_t$ 是**无偏估计**(bias-corrected)；
- $1 - \beta^t$ 是第 t 步的修正系数。

随着 t 增大， β^t 趋近于 0，修正项变为 1，所以只在初期有显著影响。

Keynote 4.3

因为 $v_0 = 0$ 不是一个真实的估计，它肯定比现实值要小，所以 v_t 是被低估了。而由上面的推导我们知道 $E[g^2]_t = (1 - \gamma) \sum_{i=1}^t \gamma^{t-i} E[g_i^2]$ ，假设每一步期望采样独立同分布，令 $E[g_i^2] = \mu$ 因此有 $E[v_t] = (1 - \gamma) \sum_{i=1}^t \gamma^{t-i} \mu = (1 - \gamma) \mu \sum_{i=1}^t \gamma^{t-i} = (1 - \gamma) \mu \frac{1 - \gamma^t}{1 - \gamma} = \mu (1 - \gamma^t)$ 所以有 $E[v_t] = E[g^2](1 - \gamma^t)$ 因此误差估计的修正项就是 $\frac{1}{1 - \beta^t}$ 最终的 $\hat{v}_t$ 是无偏估计

4.3.6. AdamW

在原始的 Adam 中，我们常常使用 **L2 正则项** 来防止过拟合，形式是： $L(\theta) + \frac{\lambda}{2} \|\theta\|^2$ 这就会带来一个额外的梯度项 $\lambda \cdot \theta$ ，被加入到梯度更新中。

所以，很多框架（如 PyTorch）默认会把它**加到梯度上**：

$$g_t^{\text{new}} = g_t + \lambda \cdot \theta$$

然后再执行 Adam 的更新逻辑，这样附加项会对原有正常结果产生影响。

AdamW 就是希望将正则项“从梯度更新中剥离”，变成一个独立的**权重衰减项**。

即：

$$\theta_{t+1} = \theta_t - \alpha \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \cdot \theta_t \right)$$

Keynote 4.4

注意：weight decay(L2 衰减) 是直接对参数 θ_t 进行缩小，而不是对梯度进行修改！

4.4. 自适应学习率调整（学习率调度器）

学习率是调整梯度下降步长的超参数，学习率过大过小都会导致模型无法收敛。在实际训练中，我们常常会根据训练的轮数来调整学习率，这就是学习率调整。

Keynote 4.5

为什么需要学习率调度器？

以一个“一个盲人下山”为例子。

- **山**: 代表整个复杂的损失函数 (*Loss Landscape*)。山的海拔高度就是损失函数的值 (*Loss*)。
- **盲人**: 代表我们的模型。
- **盲人的位置**: 代表模型当前的一组参数 (权重和偏置)。
- **目标**: 走到山谷的最低点, 也就是让损失函数的值最小。
- **拐杖**: 用来感知脚下地面的坡度, 这个“坡度”就是梯度 (*Gradient*)。梯度会指向当前位置最陡峭的上升方向, 那么它的反方向就是下降最快的方向。
- **如何走路 (策略)**: 这就是 *Optimizer* (优化器)。它决定了盲人听从拐杖的建议后, 下一步该往哪里走, 走多大一步。
- **步子的大小**: 这就是学习率 (*Learning Rate*)。步子太大可能会直接跨过最低点 (“扯着蛋”), 步子太小则下山速度太慢。

如果整个下山过程都用固定的步长 (固定的学习率), 会遇到两个问题:

- **步子太大**: 在山坡上 (训练初期) 走得很快, 但快到山谷 (最优点) 附近时, 因为步子太大, 很容易一步跨过最低点, 然后在谷底来回“震荡”, 始终无法精确到达谷底中心。
- **步子太小**: 从一开始就走得很慢, 虽然最终能精确到达谷底, 但整个下山过程会非常非常耗时 (训练时间过长)。

一个聪明的盲人会怎么做? 他会先迈大步, 快速接近谷底的大致区域; 感觉脚下的路越来越平坦时, 就放慢脚步, 小心翼翼地小步探索, 找到最低的那个点。这就是学习率调度器的核心思想。

学习率调度器就是用来调整学习率的, 它通常是一个函数, 输入是当前的迭代次数, 输出是当前的学习率。

常见的学习率调度器包括: **StepLR**、**CosineAnnealingLR**、**ReduceLROnPlateau** 等。

4.4.1. StepLR (阶梯式下降)

StepLR 是一种简单的学习率调度器, 它根据训练的轮数来调整学习率, 通常是**阶梯式下降**。

- **公式**:
 $\alpha_t = \alpha_0 \cdot \gamma^t$ 其中 α_0 是初始学习率, γ 是衰减因子, t 是当前的迭代次数。
- **特点**: 学习率的变化是跳跃式的, 像下楼梯一样, 每隔一段时间就下降一个台阶。简单有效, 但不够平滑。

4.4.2. CosineAnnealingLR (余弦退火)

余弦退火的方式就是预先**设定一个初始学习率**，然后逐步衰减。然后余弦的**周期性**让学习率在降到最低后再到最高值。

公式：

(Warm-up) If $t < T_w$, then $\alpha_t = \frac{t}{T_w} \alpha_{max}$.

(Cosine annealing) If $T_w \leq t \leq T_c$, then $\alpha_t = \alpha_{min} + \frac{1}{2} \left(1 + \cos \left(\frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{max} - \alpha_{min})$.

(Post-annealing) If $t > T_c$, then $\alpha_t = \alpha_{min}$.

其中 T_w 是预热轮数, T_c 是余弦退火轮数, α_{max} 是初始学习率, α_{min} 是最小学习率。

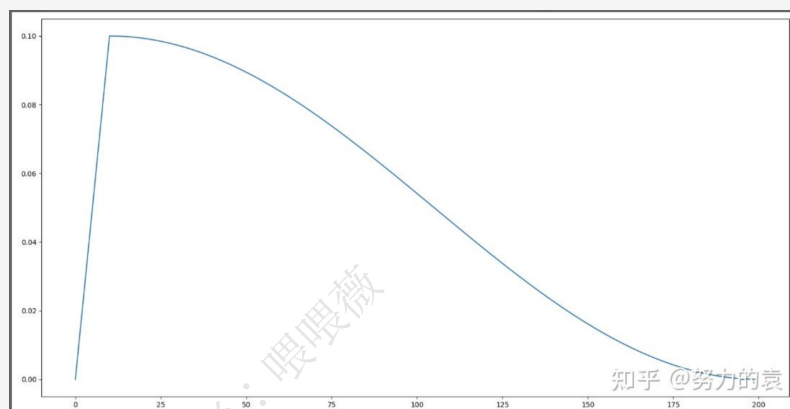


图 4.5: 余弦退火示意图

```
class CosineSchedule:
    def __init__(self, max_learning_rate, min_learning_rate, warmup_iters,
        cosine_cycle_iters):
        self.max_learning_rate = max_learning_rate
        self.min_learning_rate = min_learning_rate
        self.warmup_iters = warmup_iters
        self.cosine_cycle_iters = cosine_cycle_iters

    def __call__(self, it):
        if it < self.warmup_iters:
            return self.max_learning_rate * it / self.warmup_iters
        elif it > self.cosine_cycle_iters:
            return self.min_learning_rate
        else:
            return self.min_learning_rate + (self.max_learning_rate -
        self.min_learning_rate) * (1 + math.cos(math.pi * (it -
        self.warmup_iters) / (self.cosine_cycle_iters - self.warmup_iters))) / 2
```

4.4.3. ReduceLROnPlateau (遇到平台期就减速)

ReduceLROnPlateau (Reduce Learning Rate On Plateau) 是一种“自适应”学习率调度策略。它通过监控一个指定的性能指标 (通常是**验证集损失 val_loss**), 当这个指标在一定时期内 (称为**“耐心值” patience**) 不再出现明显改善时, 它就会自动将当前学习率降低一个固定的比例 (factor), 帮助模型“小步慢走”, 从而跳出平台

期,更精细地寻找最优解。

解释:

我们可以把这个过程想象成“一位有耐心的老师在辅导学生”。

- **学生**: 我们的模型。
- **考试分数**: 被监控的指标 (比如验证集损失 **val_loss**)。
- **学习方法/强度**: 学习率。
- **老师**: ReduceLROnPlateau 调度器。

这位老师的辅导策略是:

- **观察 (Monitor)**: 每次考完试 (每个 epoch 结束后), 老师都会记录下学生的分数 (val_loss)。
- **保持耐心 (Patience)**: 老师不会因为学生一次没考好就立刻改变策略。他会设定一个**耐心值**, 比如 `patience=5`, 意味着他会连续观察 5 次考试。
- **判断平台期 (Plateau)**: 如果在这连续的 5 次考试中, 学生的最高分都没有被刷新 (val_loss 没有降到更低), 老师就认为学生遇到了“学习瓶颈”或“**平台期**”。
- **调整策略 (Reduce LR)**: 一旦确认学生进入平台期, 老师就会调整辅导策略, 说: “看来之前的方法太激进了, 我们放慢点, 把知识点弄得再细一些。” —— **这就是将学习率乘以一个衰减因子 (factor), 比如 0.1, 让学习率骤减。**
- **冷静期 (Cooldown)**: 调整策略后, 老师会进入一个“**冷静期**”, 比如 `cooldown=2`。在这两次考试内, 即使学生分数还没起色, 老师也不会再调整策略, 而是给学生时间去适应新的、更慢的学习节奏。

工作流程示例:

假设我们设置如下:

`monitor='val_loss', mode='min', patience=3', factor=0.5, lr=0.01`

Epoch	val_loss	说明	Patience 计数	学习率 (LR)
1	1.0	初始最佳值	0	0.01
2	0.9	改善, 更新最佳值为 0.9	重置为 0	0.01
3	0.8	改善, 更新最佳值为 0.8	重置为 0	0.01
4	0.82	未改善	1	0.01
5	0.81	未改善	2	0.01
6	0.83	未改善	3	0.01
触发条件:Patience 达到 3,LR 需要降低				
7	0.75	改善, 更新最佳值为 0.75	重置为 0	0.005
8	0.76	未改善	1	0.005

表 4.1: ReduceLROnPlateau 工作流程示例

优缺点总结:

1. 优点:

- **自适应和直观:**它的逻辑非常符合直觉, 能够根据模型的真实表现来调整学习率, 而不是依赖一个固定的、需要预先猜测的时间表。
- **对于精调非常有效:**在训练后期, 当模型性能提升变得困难时, 这种“自动减速”机制对于找到更深、更优的最小值点非常有帮助。

2. 缺点:

- **依赖高质量的验证集:**如果验证集的表现有很大的随机性或噪声, 可能会导致调度器过早或错误地降低学习率。
- **可能反应较慢:**如果 patience 设置得过高, 模型可能会在平台期浪费很多个 epoch, 拖慢整体训练进程。

4.4.4. Warmup (预热)

Warmup (预热) 是一种在深度学习模型训练初期, 将学习率从一个**非常小的值 (甚至为 0)**逐步、平滑地增加到预设的初始学习率的策略。其核心目的是为了在模型训练最开始的、最不稳定的阶段, 给予模型一个“热身”或“适应”的过程, 防止因初始学习率过大而导致的训练不稳定甚至崩溃。它通常作为其他学习率调度器 (如余弦退火) 的前奏部分。

为什么初期需要预热:

在训练刚开始时, 由于

1. **参数是完全随机的:**模型对输入数据一无所知, 其内部的权重和偏置都是随机初始化的“胡乱猜测”。
2. **初始损失和梯度巨大:**因为参数是随机的, 模型第一次进行前向传播时, 其预测结果会与真实标签相差十万八千里。这会导致计算出的损失函数的值 (Loss) 通常非常大。
3. **梯度方向极其不确定:**巨大的损失会反向传播, 计算出的梯度 (Gradient) 也会异常巨大且方向不稳定。这个梯度仅仅反映了从一个完全随机的点该如何“逃离”, 这个方向对于全局最优解来说, 参考价值很低。

而在真正进入到第一轮训练的时候, 每个数据点对模型来说都是新的, 模型会很快地进行数据分布修正, 如果这时候学习率就很大, 极有可能导致开始的时候就对该数据**“过拟合”**, 后面要通过多轮训练才能拉回来, 浪费时间。当训练了一段时间 (比如两轮、三轮) 后, 模型已经对每个数据点看过几遍了, 或者说对当前的 batch 而言有了一些正确的先验, 较大的学习率就不那么容易会使模型学偏, 所以可以适当调大学习率。这个过程就可以看做是 warmup。

那么为什么之后还要 decay 呢? 当模型训到一定阶段后 (比如十个 epoch), 模型的分布就已经比较固定了, 或者说能学到的新东西就比较少了。如果还沿用较大的学习率, 就会破坏这种稳定性, 用我们通常的话说, 就是已经接近 loss 的 local optimal 了, 为了靠近这个 point, 我们就要慢慢来。

Warmup 与其他调度器的关系:

非常重要的一点是：Warmup 几乎总是与我们上面提到的其他学习率调度器（如 CosineAnnealingLR, StepLR）配合使用。

完整的训练过程的学习率策略被分为两个阶段：

- **第一阶段 (预热期):**学习率从 0 线性增长到 initial_lr。
- **第二阶段 (正常调度期):**学习率从 initial_lr 开始, 按照你选择的主调度器 (如余弦退火) 的规则进行衰减。

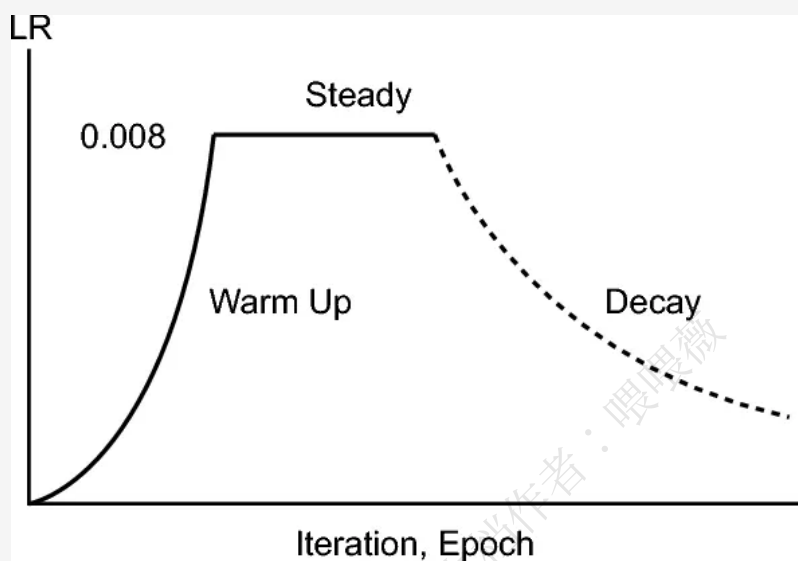


图 4.6: Warmup 与其他调度器配合使用示意图

微言大义 4.1 超参数的配置不仅在这篇教程中，在所有的 Training 里面都是很重要的，往后大部分科研或者工程上的工作可能实际上就是在调参。

4.5. 梯度裁剪

梯度裁剪是防止梯度爆炸的一种方法，通过将梯度限制在一个范围内，从而防止梯度爆炸。

公式：

$$g_t = \min(g_t, \text{clip_value})$$

其中 *clip_value* 是裁剪阈值。

```
class GradientClip:
    def __init__(self, parameters, max_l2_norm, epsilon=1e-6):
        self.parameters = parameters
        self.max_l2_norm = max_l2_norm
        self.epsilon = epsilon

    def __call__(self):
```

```
grads = [p.grad for p in self.parameters if p.grad is not None] # 我们求 l2 范数
↪ 是对所有元素求的，所以要把所有元素给 flatten
all_grads = torch.cat([grad.flatten() for grad in grads])
grad_l2 = torch.norm(all_grads, 2)
if grad_l2 > self.max_l2_norm:
    clip_coeff = self.max_l2_norm / (grad_l2 + self.epsilon)
    for grad in grads:
        grad.mul_(clip_coeff)
```

5

泛深度学习训练流程

一般的深度学习训练流程就包括:

1. 数据集加载 (DataLoader)

代表**输入环节**, 在数据集进入模型之前, 需要对数据集进行处理, 比如归一化, 分批, 打乱等。

2. 模型保存 (Checkpoint)

代表**输出环节**, 在模型训练过程中, 需要定期保存模型参数, 以便避免在训练过程中因为各种原因导致模型参数丢失。

3. 训练循环 (TrainLoop)

这是**最终的脚本**。它负责初始化所有组件, 并按照正确的顺序组织整个训练流程, 包括数据加载、模型计算、反向传播、参数更新、性能记录和模型保存等。

5.1. 数据集加载 (DataLoader)

参考资料 5.1

```
https://www.runoob.com/pytorch/pytorch-dataset-dataloader.html?utm_source=chatgpt.com
https://docs.pytorch.org/tutorials/beginner/basics/data_tutorial.html?utm_source=chatgpt.com
```

PyTorch 提供了两个数据原语: `torch.utils.data.DataLoader` 和 `torch.utils.data.Dataset`。它们允许你使用预加载的数据集和你自己的数据。Dataset 存储了样本及其相应的标签, DataLoader 在 Dataset 周围包装了一个可迭代对象, 以便于访问样本。AI 为你提供母语级高精翻译

一个完整、符合 PyTorch 习惯的流程图

原始数据 --> Dataset (读取样本 + 数据增强 transforms) --> DataLoader (batching, shuffle, num_workers, pin_memory...) --> 训练循环 for batch in DataLoader

5.1.1. 数据增强的两种方式

在线增强 (Online Augmentation)

放在 Dataset 里, 训练过程中每次取样本时随机增强
特点

5.1 数据集加载 (DataLoader)	71
5.1.1 数据增强的两种方式	71
5.1.2 自定义 Dataset	73
5.1.3 使用 DataLoader 加载数据	73
5.1.4 如果数据集过大无法载入内存怎么办?	74
5.1.5 内存映射	74
5.2 模型保存 (Checkpoint)	76
5.2.1 为什么 Checkpoint 至关重要?	76
5.2.2 一个 Checkpoint 文件里通常包含什么?	77
5.3 脚本化训练	78
5.4 消融实验 Ablation Studies	79

- 每次训练加载一个样本时才进行随机增强 (如随机裁剪、翻转), 不需要提前生成大量数据副本
- 每个 epoch 都可能看新的增强结, 提升模型泛化
- 训练期间使用 CPU 对每个样本执行 transform, DataLoader 多进程可**并行加速**

```
class MyDataset(Dataset):
    def __init__(self, data_paths, transform=None):
        self.paths = data_paths
        self.transform = transform # 数据增强在这里
    def __getitem__(self, idx):
        img = read_image(self.paths[idx])
        if self.transform:
            img = self.transform(img) # → 在 Dataset 做 augmentation
        return img
```

离线增强 (Offline Augmentation)

提前对原始数据增强好

特点

- Dataset 简单、**读取速度快**
- 没有训练时增强的 CPU 开销 → **DataLoader 更轻松、GPU 利用率更高**
- 非常适合:
 - 大规模数据
 - 训练频繁、增强重复多次
 - 多机训练、分布式训练 (offline augmentation 避免重复增强)
 - 图像增强开销特别大的情况 (如大图、复杂变换)

实际工程中通常会采用:

混合模式 (推荐)

- 离线做一些“耗时重、固定不变”的增强
 - 图像标准化
 - 统一尺寸 resize
 - 去噪、直方图均衡 (如果需要)
 - 硬件加速无法轻易做的操作
- 在线做一些“轻量、随机”的增强
 - RandomCrop
 - RandomFlip
 - ColorJitter
 - RandomRotation

5.1.2. 自定义 Dataset

`torch.utils.data.Dataset` 是一个抽象类, 允许你从自己的数据源中创建数据集。

我们需要继承该类并实现以下两个方法:

- ‘`__len__`’(self): 返回数据集中的样本数量。
- ‘`__getitem__`’(self, idx): 通过索引返回一个样本。(有了这个函数才可以使用索引, 比如 `dataset[6]`)

假设我们有一个简单的 CSV 文件或一些列表数据, 我们可以通过继承 `Dataset` 类来创建自己的数据集。

```
import torch
from torch.utils.data import Dataset

# 自定义数据集类
class MyDataset(Dataset):
    def __init__(self, data_paths, transform=None):
        """ 初始化数据集,path 是数据集的路径,transform 代表数据增强的手段"""
        self.paths = data_paths
        self.transform = transform # 数据增强在这里
        self.X_data = read(path) # 读取数据集
        self.Y_data = ...

    def __len__(self):
        """ 返回数据集的大小"""
        return len(self.X_data)

    def __getitem__(self, idx):
        """ 返回指定索引的数据"""
        x = torch.tensor(self.X_data[idx], dtype=torch.float32) # 转换为 Tensor
        y = torch.tensor(self.Y_data[idx], dtype=torch.float32)
        return x, y
```

5.1.3. 使用 DataLoader 加载数据

`Dataloader` 是对 `Dataset` 的进一步封装, 它主要是为了提升训练效率, 常用的手段有 **batch 打包 (batch_size)**、**打乱顺序 (shuffle)**、**多进程加速 (num_workers)**、**内存优化 (pin_memory)**、**collate 规则 (collate_fn)**、**textbfdrop_last**: 如果数据集中的样本数不能被 ‘batch_size’ 整除, 设置为 ‘True’ 时, 丢弃最后一个不完整的 batch。其中前两个最常用。

```
dataloader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True,
    num_workers=4,
    pin_memory=True
)

# 打印加载的数据
```

```
for epoch in range(1):
    for batch_idx, (inputs, labels) in enumerate(dataloader):
        print(f'Batch {batch_idx + 1}:')
        print(f'Inputs: {inputs}')
        print(f'Labels: {labels}')
```

5.1.4. 如果数据集过大无法载入内存怎么办？

我们可以使用名为 `mmap` 的 Unix 系统调用，它能将磁盘文件映射到虚拟内存，并在访问该内存位置时惰性加载文件内容。这样就能“假装”整个数据集已载入内存。

NumPy 通过 `np.memmap` (或使用 `np.load` 时设置 `mmap_mode='r'` 标志，前提是数组最初通过 `np.save` 保存) 实现该功能，它会返回一个类似 numpy 数组的对象，在访问时按需加载数据项。在训练期间从数据集 (即 numpy 数组) 采样时，务必以内存映射模式加载数据集 (通过 `np.memmap` 或 `np.load` 的 `mmap_mode='r'` 标志，具体取决于数组的保存方式)。

5.1.5. 内存映射

数据并不是在磁盘上被直接访问的。CPU 的核心原则是：**它只能直接处理位于物理内存 (RAM) 中的数据。任何在磁盘上的数据，如果想被 CPU 计算、读取或修改，最终都必须被加载到物理内存中。**

内存映射的快体现在它**极大地优化了数据从磁盘进入物理内存，并最终呈现给用户进程的这个过程。**

传统 I/O (`read()`) 的数据流: 两次 copy

假设程序需要从磁盘读取 1GB 的文件到一块内存中。

1. 第一次拷贝: 从 **磁盘 -> 内核缓冲区**

- 你的程序发起 '`read()`' 系统调用，导致程序从**用户态**切换到**内核态**。
- 内核向磁盘控制器发出指令。
- 磁盘控制器通过 **DMA(Direct Memory Access)**，将文件数据从磁盘直接拷贝到内核地址空间的一块缓冲区里。这个缓冲区通常被称为**页缓存 (Page Cache)**。
- **关键:** 到目前为止，数据在内核里，你的用户程序还访问不到它。DMA 完成了这次拷贝，CPU 没有参与搬运数据，但它需要等待。

2. 第二次拷贝: 从 **内核缓冲区 -> 用户缓冲区**

- 现在数据已经在内核的页缓存里了，'`read()`' 系统调用的下一步，就是把这些数据从**内核空间**拷贝到你程序在**用户空间**指定的缓冲区 (就是你传给 '`read()`' 函数的那个 `buffer` 指针)。
- **这是性能瓶颈所在:** 这次拷贝是由 **CPU** 亲自执行的。对于 1GB 的数据，CPU 需要执行大量的 '`mov`' 指令，逐字节地把数据从一块内存搬到另一块内存。这会消耗大量的 CPU 周期，并且会严重污染 CPU 的缓存 (Cache)。

- 拷贝完成后, `read()` 系统调用返回, 程序从内核态切换回用户态。现在你的程序终于可以访问这 1GB 数据了。

总结传统 I/O: 数据走了 ‘**磁盘 -> 内核内存 -> 用户内存**’ 的路径。发生了两次拷贝, 其中一次是由 CPU 完成的、非常昂贵的内存拷贝。

内存映射 (Memory Mapping),

就是操作系统提供的一种机制, 它允许一个进程将自己的**虚拟地址空间**中的一部分, 直接与一个文件或者设备进行 “**链接**” 或 “**映射**”。

核心工作原理

1. 建立映射 (`mmap` 系统调用)

- 当一个进程调用 `mmap()` 系统调用请求建立内存映射时, 内核执行以下操作:
- 在进程的虚拟地址空间中找到一段**连续的、未被使用**的区域。
- 为这段地址区域在进程的内存描述符 (如 Linux 中的 `vm_area_struct`) 中创建一个新的**条目**, 记录下映射的起始地址、长度、权限 (读/写/执行) 以及映射所关联的文件和偏移量。
- **关键点:** 此时, 内核**并不会**立即从磁盘加载任何文件数据到物理内存中。它仅仅是建立了逻辑上的关联, 即配置了相关的内核数据结构。这是一个**非常轻量级**的操作。

2. 首次访问与缺页中断 (Page Fault)

- 当进程首次尝试访问 (读或写) 这段被映射的虚拟内存地址时, 会发生以下一系列事件:
- **MMU(内存管理单元)** 硬件在转换虚拟地址时, 会查询进程的页表。由于数据尚未加载, 对应的 PTE 会标记为 “不存在” (Absent)。
- MMU 无法完成地址翻译, 于是触发一个硬件异常, 即**缺页中断**, 并将控制权交回给操作系统内核。
- 内核的缺页中断处理程序被激活。它会检查导致中断的虚拟地址, 并查询进程的内存描述符, 确认该地址属于一个合法的内存映射区域。

3. 数据加载与页表更新

- 确认是合法访问后, 内核会执行真正的 I/O 操作:
- 在物理内存 (RAM) 中分配一个**空闲的物理页帧**。
- 根据 `mmap` 时记录的文件信息和偏移量, 计算出需要从磁盘加载的数据块位置。
- 通过**磁盘驱动程序**, 将对应的一个页 (通常为 4KB) 的数据从文件加载到刚刚分配的物理页帧中。
- 更新进程的**页表**, 将触发中断的那个虚拟页的 PTE 指向这个新加载的物理页帧, 并设置好相应的权限位 (如 “存在”、“可读”、“可写” 等)。
- 中断处理结束, 控制权返回给用户进程。

4. 透明的后续访问

- 当中断处理程序返回后, 导致中断的那条指令会被重新执行。这一次, **MMU**能够成功地通过**页表**将虚拟地址翻译为物理地址, 进程便可以毫无知觉地访问到数据, 仿佛它从一开始就在内存里一样。后续对同一页内其他地址的访问将直接通过 **MMU** 快速完成, 不再触发中断。

总结内存映射: 数据走了‘**磁盘 -> 内核内存 (页缓存)**’的路径。然后, 你的程序被“授权”直接访问这块内核内存。从始至终, 数据只从磁盘到内存拷贝了**一次**, 而且**CPU 没有参与任何批量数据的搬运工作**。这就是为什么它有时被称为“零拷贝”技术 (这里的“零”指的是没有发生 CPU 参与的内核态到用户态的拷贝)。

所以实际上内存映射的本质, 就是通过虚拟内存机制, 让用户进程能够安全、直接地访问到本属于内核空间的那块内存 (页缓存), 从而消除了数据从内核空间到用户空间的冗余拷贝。

Keynote 5.1

内存映射这么牛逼, 这么省资源, 那么干脆所有的读取方式都改成内存映射不就好了?

也不是, 内存映射省资源主要省在一次 CPU 对文件的拷贝上, 对付大文件比较合适, 但对付小文件, 拷贝省的资源可能还不够**建立映射的开销**和**缺页中断的开销**这两部分大, 所以也是一个 *trade-off* 的问题。另外这个也会消耗虚拟地址的空间。

5.2. 模型保存 (Checkpoint)

参考资料 5.2

<https://blog.csdn.net/PolarisRisingWar/article/details/145529106>

<https://juejin.cn/post/7435680251878015016>

试想一下, 假如我们在某次深度学习的训练中大概要训练个 1000 个 epoch, 得知训练预计完成要花费将近 3 天时间, 我们好不容易训练了两天 23 个小时了, 这时候突然停电了, 服务器/电脑关机了, 一切努力付诸东流, 这下要气死了。

这个时候 checkpoint 的重要性就体现出来了。

checkpoint 可以看作深度学习训练里面的“**存档点**”, 它保存了深度学习某一时刻的模型权重。如果我们定时每 100 个 epoch 就保存一次模型权重, 那么即使训练到第 999 个 epoch 的时候就停电了, 那么我们起码还可以从第 900 个 epoch 上重新开始续训, 而不用从 0 开始。

5.2.1. 为什么 Checkpoint 至关重要?

Checkpoint 的重要性体现在以下几个关键场景:

- **训练中断与恢复**

- 场景: 深度学习训练非常耗时, 服务器可能会意外重启、程序可能崩溃、GPU 资源可能被抢占。
- 作用: 如果没有 Checkpoint, 你需要从头开始训练, 浪费大量时间和计算资源。有了 Checkpoint, 你可以从最近的保存点继续训练, 就像从游戏存档点复活一样, 几乎无缝衔接。

● 保存最佳模型

- 场景: 模型在训练过程中性能会波动。通常, 我们关心的是在**验证集 (Validation Set)**上表现最好的那个模型, 而不是训练到最后时刻的模型 (后者可能已经开始过拟合)。
- 作用: 通过在每个 Epoch(或固定步数) 结束后评估模型在验证集上的性能 (如准确率、损失值), 我们可以只保存那个性能指标最好的模型状态。这是实际部署时最常用的策略。

● 迁移学习与微调

- 场景: 你想在一个新的、数据量较小的任务上训练模型。从零开始训练一个大模型 (如 ResNet, BERT) 既困难又低效。
- 作用: 我们可以加载一个在大型数据集 (如 ImageNet, Wikipedia) 上预训练好的模型 Checkpoint, 然后在这个基础上针对我们的新任务进行**微调**。这极大地加速了收敛速度并提升了模型性能。

● 模型部署与推理

- 场景: 模型训练完成后, 你需要将它部署到生产环境中提供服务 (例如, 用于图像识别、文本生成)。
- 作用: 部署时加载的就是最终选定的最佳模型的 Checkpoint 文件, 用它的权重来进行预测和推理。

5.2.2. 一个 Checkpoint 文件里通常包含什么?

一个完备的 Checkpoint 文件不仅仅是模型的权重, 它通常包含一个字典或对象, 其中含有:

- **模型参数或权重**: 这是最核心的部分, 即模型学习到的所有权重 (weights) 和偏置 (biases)。在 PyTorch 中, 这通常是 `'model.state_dict()'`。
- **优化器状态**: 这对于恢复训练至关重要。像 Adam、SGD with Momentum 这类优化器, 它们内部会维护一些状态 (如动量、学习率适应性调整的梯度平方均值等)。如果不保存优化器状态, 恢复训练时优化器会重置, 相当于重新开始, 会影响训练的连续性。在 PyTorch 中, 这是 `'optimizer.state_dict()'`。
- **训练元数据**:
 - **当前轮次 (Epoch)**: 记录训练进行到了第几轮。
 - **迭代步数**: 记录总共迭代了多少步。
 - **损失值**: 当前或历史最佳的训练/验证损失。

● **其他指标**: 如准确率、F1 分数等。

如果说训练之后的结果只需要用来**推理 (inference) 或部署**, 而不再做“从某个中断点继续训练”的话, 那么只保存模型的 **state_dict**(即参数权重) 就够了。例如:

```
torch.save(model.state_dict(), "model_weights.pth")
```

然后在部署或加载时:

```
model = MyModel()
model.load_state_dict(torch.load("model_weights.pth"))
model.eval()
```

但如果担心停电导致前功尽弃, 需要断电续训的话, 就最好把模型权重 (**model.state_dict()**)、优化器状态 (**optimizer.state_dict()**) ——包括动量、Adam 的 m/v、学习率状态等以及当前训练轮次或步数 (epoch 或 global_step) 都保存下来。

```
checkpoint = {
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'loss': some_loss_value,
    # 可能也保存 scheduler 和其他东西
}
torch.save(checkpoint, "checkpoint.pth")
```

加载恢复:

```
checkpoint = torch.load("checkpoint.pth")
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
start_epoch = checkpoint['epoch'] + 1
```

5.3. 脚本化训练

等到把所有的模块都整理完毕了, 到最后我们进行实验时的变量其实主要就是超参数 + 不同数据集了, 这个时候使用脚本化的训练能大大方便我们的实验。

因此我们的目标: **一个 train.py, 改少量参数就能换模型 / 换数据 / 换实验**。

我的经验一般是一个 config 文件 (可以是 **yaml 或者 json 格式** 用来保存所有的超参数, 方便修改, 而不是在代码里面写死导致修改极不方便。然后用 **argparse** 配合直接在输入命令的时候就能控制变量:

比如

```
def get_args():
    parser = argparse.ArgumentParser()

    # 基本配置
```

```
parser.add_argument('--exp_name', type=str, default='baseline')
parser.add_argument('--seed', type=int, default=42)
parser.add_argument('--epochs', type=int, default=20)
parser.add_argument('--batch_size', type=int, default=64)
parser.add_argument('--lr', type=float, default=1e-3)
parser.add_argument('--num_workers', type=int, default=4)

# 数据 & 模型
parser.add_argument('--data_root', type=str, default='./data')
parser.add_argument('--num_classes', type=int, default=10)
parser.add_argument('--model', type=str, default='mlp') # 预留给后面多模型选择

# 设备 & 保存
parser.add_argument('--device', type=str, default='cuda')
parser.add_argument('--output_dir', type=str, default='./runs')
parser.add_argument('--resume', type=str, default='') # 断点续训

args = parser.parse_args()
return args
```

5.4. 消融实验 Ablation Studies

消融实验是一种非常常用的实验方法, 通过逐步移除模型中的某些组件或修改某些超参数, 来研究这些组件或超参数对模型性能的影响。

消融实验的目的是为了证明模型中某个组件或超参数的重要性, 或者证明模型中某个组件或超参数的无效性。绝大部分有关修改模型组件的文章里面都会有一章关于消融实验的讨论, 以此来证明自己修改的组件或超参数是有用的。

一个严谨的消融实验通常遵循以下步骤:

1. 第一步: 建立一个强大的基线 (Baseline)

- 首先, 你需要有一个完整、性能表现最好的模型。这个模型包含了你想要研究的所有组件和技术。你需要在一个标准的评估数据集上测试它, 并记录下关键性能指标 (如准确率、精确率、召回率、mAP 等), 这将作为后续所有比较的基准。

2. 第二步: 确定要“消融”的组件

- 列出你想要研究其贡献度的所有组件。这些组件可以是:
 - 网络架构的一部分: 如注意力模块、一个特定的卷积层块 (Bottleneck Block)、残差连接 (Skip Connection)。
 - 数据增强技术: 如 Mixup, CutMix, Mosaic 等。
 - 损失函数的某个部分: 如在一个复合损失函数中去掉某个特定的损失项。
 - 特定的训练策略: 如学习率预热 (Warm-up)。
 - 预训练模型的应用: 比如比较“使用预训练权重”和“从零开始训练”的性能差异。

3. 第三步: 系统性地实验

- 从基线模型开始，每次只移除或替换一个组件，然后重新训练模型，并在相同的测试集上评估性能。

4. 第四步：结果分析与呈现

- 将所有实验结果整理成一个清晰的表格，这是学术论文中最常见的形式。表格会直观地展示移除不同组件后性能的下降情况。

在我们这个作业里，它要求的消融实验是 3 部分：

1. 消融实验 1: 层归一化

通常认为层归一化对 Transformer 训练的稳定性至关重要。讲义要求从每个 Transformer 模块中移除 RMSNorm 看看会发生什么。

另一个关于层归一化的实验是前归一化与后归一化的对比，讲义要求实现前归一化与后归一化的 Transformer 模块，并比较两者的性能。

2. 消融实验 2: 位置嵌入对模型性能的影响

我们接下来将研究位置嵌入对模型性能的影响。具体而言，我们将对比基础模型（使用 RoPE）与完全不使用位置嵌入（NoPE）的模型。研究发现，仅使用解码器的 Transformer 模型（即我们实现的因果掩码模型）理论上无需显式提供位置嵌入信息，即可推断相对或绝对位置信息。接下来我们将通过实证测试 NoPE 与 RoPE 的性能差异。

3. 消融实验 3: SwiGLU 与 SiLU 对比

讲义要求通过比较使用 SiLU 激活函数但未采用门控线性单元（GLU）的前馈网络与使用 SwiGLU 激活函数但未采用门控线性单元的前馈网络的性能，验证门控机制在前馈网络中的重要性。需要说明的是，在我们的 SwiGLU 实现中，将内部前馈层的维度设定为约 $d_{ff} = \frac{3}{8}d_{model}$ （同时确保 d_{ff} 模 64=0，以便充分利用 GPU 张量核心）。在您的 FFNSiLU 实现中，应将 d_{ff} 设置为 $4 \times d_{model}$ ，以大致匹配 SwiGLU 前馈网络的参数数量（该网络具有三个而非两个权重矩阵）。

模型训练之后就是我们最关心也是最常用的推理部分，这一部分主要是将训练好的模型加载出来，然后根据输入的一段文本，用模型来输出一段文本。训练部分可以理解为对输入文本的各种各样的**编码**，让模型能够学习。而推理部分就是将学习后的模型后对输入的文本再进行**解码**，得到最终的输出。

6.1 Softmax 层与解码 . . . 81

6.2 温度缩放 (Temperature Scaling) 82

6.3 Top-p 采样 83

6.1. Softmax 层与解码

正如我们之前所说的，模型最后有一个线性层，它将输出一个维度等于模型的词汇表大小 (Vocabulary Size) 的向量，这个输出的、长度为词汇表大小的向量，就是 **Logits 向量**。这个 Logits 向量中的每一个元素，都对应词汇表中的一个词。例如：logits[10] 可能是词“pig” 的分数，logits[25] 可能是词“cat” 的分数，logits[100] 可能是词“dog” 的分数。

假设我们得到的 Logits 向量中，与“pig” 对应的分数值是 10.2，与“cat” 对应的分数值是 5.1，而与“dog” 对应的分数值是 -3.5。这直观地表明，模型非常有信心地认为下一个词是“pig”，对“cat” 有一些信心，但几乎完全排除了“dog”。

但 Logits 本身是原始分数，不直观，也不方便计算损失函数。因此，我们需要将它们转换成标准的概率分布（所有项的概率在 0 到 1 之间，且总和为 1）。这个转换步骤通过 Softmax 函数来完成。经过 Softmax 处理后，我们就得到了一个概率分布向量。在上面的例子中：“pig” 的概率可能变成了 0.95% (95%)，“cat” 的概率可能变成了 0.04% (4%)，“dog” 的概率可能变成了 0.0001% (接近 0%)。所有词汇表的概率之和为 1。

用公式表示即： $P(x_{t+1} = i \mid x_{1...t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)}$ $v = \text{TransformerLM}(x_{1...t})_t \in \mathbb{R}^{\text{vocab_size}}$

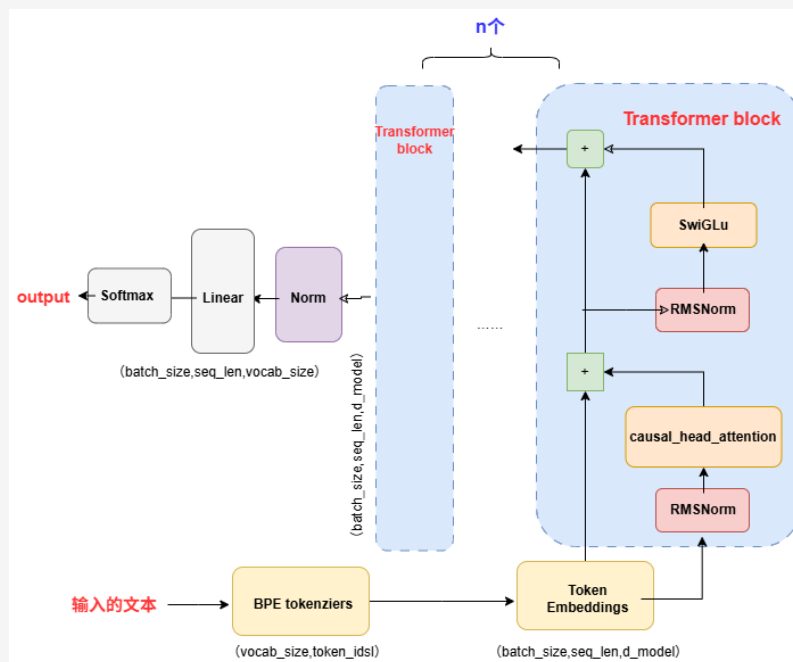


图 6.1: Transformer 架构流程图

因此总结下来，整个解码过程就是输入一段文本 (或者说提示词 Prompt), 模型就会根据提示词生成下一个词的概率分布，模型进行采样得到下一个词，自回归特性将重复这个过程，直到生成序列结束标记 <endoftext> (或达到我们指定的最大生成数)。

微言大义 6.1 解码部分相对容易，很好理解。

6.2. 温度缩放 (Temperature Scaling)

我们刚才提到了 Transformer 解码器在每步生成时会输出 logits (未归一化的分数)，通过 Softmax 转化为概率，然后决定下一个 token。

即

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^{\text{vocab-size}} \exp(v_j)}. \quad (6.1)$$

温度缩放就是先把 logits 除以一个 ‘T’ 值：

即

$$\text{softmax}(v, \tau)_i = \frac{\exp(\frac{v_i}{\tau})}{\sum_{j=1}^{\text{vocab-size}} \exp(\frac{v_j}{\tau})}. \quad (6.2)$$

- 当 $T < 1$ 时：会 **放大高概率 token** 的优势，使分布更尖锐，对概率最高的词更偏向于生成，文本更**确定和保守**。
- 当 $T > 1$ 时：会 **平滑 logits**，使高低概率 token 差异减小，低概率 token 出现的机会增多，从而生成更具**创造性**但可能更**不稳定或混乱**的内容。

当 T 趋于无穷大时，输出概率分布将趋于**均匀分布**，概率为 $1/K$ ，此时**信息熵是最大的**。反过来， T 趋于 0 时，正确类别的概率接近 1，输出结果就是确定的，信息熵为 0。如果 T 太低，生成倾向高频词，输出趋于一致； T 太高，输出虽多样但可能跑题、不连贯。

Keynote 6.1

注意：温度缩放是 离线 操作，在生成时一次性应用，不会影响模型训练。
这里的温度 *temperature* 和 Gemini 等大模型的 *temperature* 设置道理一致。

6.3. Top-p 采样

Top-p 方法是动态选取那些累计概率高于门限 p 的 token 集合，然后从中随机抽样：

1. 对所有 token 按原始概率降序排序；
2. 选择前面的 token，直到累计概率 $\geq p$ ；
3. 丢弃剩余 token，重新归一化所选集合的概率；
4. 从中随机选取下一个 token。

候选集大小根据实际分布变化， p 较低时生成更保守、 p 较高时生成更保留创造力。

文档作者：喂喂薇

7

Assignment 2 系统篇:Systems and Parallelism

7.1. Assignment 2 Overview

7.1 Assignment 2 Overview 85

实验目标: 学习如何从**系统底层**优化提升**单 GPU 的训练速度**以及如何将训练扩展到**多 GPU**。

每一部分的实验任务

1. 基准测试与性能分析框架
2. Flash Attention 2 Triton 内核编写
3. 分布式数据并行训练
4. 优化器状态分片

文档作者: 喂喂微

文档作者：喂喂薇

8

性能分析与基准测试 Profiling and Benchmarking

在我们进行优化之前,我们首先就得清楚程序或模型哪个地方还不够好,哪个地方还有可提升的空间。一般来讲可优化的点就是时间和空间(对应我们老生常谈的时间复杂度和空间复杂度)。那么我们该如何观察到某一个部分的时间和空间的消耗程度呢?

讲义提供了三种性能评估方式:

1. 使用 Python 标准库进行简单的**端到端基准测试**,以测量前向和后向传递的时间;
2. 使用**NVIDIA Nsight Systems**工具进行计算分析,以了解这些时间在 CPU 和 GPU 操作上的分布;
3. 分析**显存使用情况**。

8.1. 端到端简单性能评估测试

参考资料 8.1

- https://blog.csdn.net/gitblog_00002/article/details/148993434
- https://blog.csdn.net/weixin_40198079/article/details/129726302

为了找到程序的瓶颈点,实现快速迭代优化。我们最好以**脚本化**的方式进行性能评估测试,例如我们可以把需要调节的超参数设置成命令行参数,然后通过脚本自动化地进行性能评估测试。讲义强烈推荐使用 Sbatch 或 Slurm 平台上的**submitit**工具来实现批量扫描。

8.1.1. Slurm 平台上的 submitit 工具简要介绍

submitit 工具也是一种批量调度脚本化的工具,和使用 shell 脚本不同,submitit 和 python 绑定程度更高,它甚至是直接无缝衔接在 python 代码中的。

举个例子,比方说我们要实现一个简单的加法函数。

```
import sys
import time
def add(a, b):
    time.sleep(10) # 模拟耗时
```

8.1 端到端简单性能评估测试	87
8.1.1 Slurm 平台上的 submitit 工具简要介绍	87
8.1.2 前向和反向传播性能评估	89
8.2 Nsight Systems Profiler	90
8.2.1 nsys 简介	90
8.2.2 nvtx	90
8.3 混合精度训练	93
8.3.1 浮点数知识回顾	94
8.3.2 混合精度训练的实现	96
8.4 显存分析	97

```

    return a + b
if __name__ == '__main__':
    # 从命令行读取参数
    a = int(sys.argv[1])
    b = int(sys.argv[2])

    result = add(a, b)

    # 打印结果，Slurm 会把这个输出保存到文件中
    print(f"计算结果是: {result}")

```

之后我们配合 shell 脚本实现。但如果我们使用了 submitit 工具，我们就可以直接在 python 代码中实现。

```

import submitit
import time
# 我们的“菜谱”函数，和之前完全一样，甚至更纯粹
def add(a, b):
    time.sleep(10) # 模拟耗时
    return a + b
# 1. 实例化一个“助理” (Executor)
# AutoExecutor 会自动检测到你正在 Slurm 环境下，并进行配置
executor = submitit.AutoExecutor(folder="slurm_logs")
# 2. 设置“厨房”要求（等同于 #SBATCH 参数）
executor.update_parameters(
    timeout_min=5, # 任务最长运行 5 分钟
    cpus_per_task=1,
    mem_gb=1
)
print("准备向集群提交任务...")
# 3. 直接让“助理”把你的 Python 函数和参数送去做
job = executor.submit(add, 5, 7)
print(f"任务已提交，任务 ID 是: {job.job_id}")
# 4. 直接在 Python 里等待并获取结果
result = job.result() # 这行代码会等待任务完成，然后把返回值给你
print(f"从集群拿回了结果: {result}")

```

在需要做**大量重复实验**时（比如机器学习调参）。

假设我们要计算三组不同的加法：(5,7), (10,20), (100,200)。

```

params = [(5, 7), (10, 20), (100, 200)] # 准备多组参数
# 一句话提交一个任务数组！
jobs = executor.map_array(add, *zip(*params)) # zip(*params) 会把 [(5,7),...] 变成
↔ ([5,10,100], [7,20,200])
print(f"一次性提交了 {len(jobs)} 个任务！")
# 等待所有任务完成，并一次性取回所有结果
results = [job.result() for job in jobs]
print(f"所有任务都完成了，结果是: {results}")
# 输出将会是：所有任务都完成了，结果是: [12, 30, 300]

```

submitit 的一大核心优势是**控制器与工作任务的解耦**，当执行 `job = executor.submit(...)` 时，发生了两件独立的事情：

- **控制器（你的本地脚本）**：它的任务是打包你的函数和参数，生成一个 Slurm 脚本，通过 `sbatch` 命令把它发送给 Slurm 调度器。一旦 Slurm 确认收到（返回一个任务 ID），控制器的主要任务就完成了。之后它做的 `job.result()` 只是一个可选的“等待和接收”动作。
- **工作任务（集群上的脚本）**：Slurm 收到请求后，会在某个计算节点上启动一个全新的、独立的进程。这个进程负责执行你的深度学习函数。它的生死只由 Slurm 和它自己决定，和你的“控制器”脚本没有任何关系。

8.1.2. 前向和反向传播性能评估

首先，通过计时前向和反向传递来对模型进行最基础的性能分析。由于仅需测量时间和内存消耗，因此将采用随机权重和数据。

Keynote 8.1

CUDA 异步调用

当 GPU 去进行矩阵计算的时候，由于 CUDA 默认是**异步执行**，CPU 也会继续执行下面的任务，不会等待 GPU 执行完毕之后再去。这样做充分利用了计算资源，但在测量的时候由于是异步，所以不能够通过 `time.time()` 这种方式直接测量 GPU 计算时间（因为 CPU 会直接跳过 GPU 运算的步骤），要测量需要 `torch.cuda.synchronize()` 强制同步。

评估流程：

- 给定**超参数**（例如层数），初始化模型。
- 生成随机数据批次。
- 运行 w 个**预热步骤**（在开始测量时间之前），然后记录 n 个步骤的执行时间（根据参数选择仅前向或前向和反向传递）。

Keynote 8.2

端到端基准测试里面的预热并非学习率预热，而是希望机器在训练了几个步骤达到稳定之后再去测试时间，否则由于刚开始的随机权重和数据，导致损失值非常大，梯度也非常大，导致训练不稳定，时间测量不准确。

- 对于计时，可以使用 Python 的 `timeit` 模块（例如使用 `timeit` 函数，或者使用 `timeit.default_timer()`，它提供系统最高分辨率的时钟，因此比 `time()` 更适合作为基准测试的默认选项）。
- 每个步骤后调用 `torch.cuda.synchronize()`。

8.2. Nsight Systems Profiler

参考资料 8.2

```
https://github.com/NVIDIA/NVTX  
https://nvidia.github.io/NVTX/python/reference.html#nvtx.start\_range
```

端到端基准测试无法揭示模型在前向传播和反向传播过程中具体消耗时间与内存的环节，因而难以发现特定组件的优化空间。为准确掌握程序**各组件（如函数）**的运行耗时，可采用性能分析工具。执行分析器通过在函数开始和结束时插入监控点来检测代码，从而提供函数级别的详细执行统计（包括调用次数、该函数累计耗时等指标）。

像 CProfile 这样的 Python 性能分析器无法对 CUDA 内核进行性能分析，因为 CUDA 内核在 GPU 上是异步执行的。因此我们将利用 NVIDIA 提供的一个可以通过命令行工具 nsys 使用的性能分析器。

8.2.1. nsys 简介

nsys 全称 **NVIDIA Nsight Systems**，是一个专门记录 CPU、GPU 具体工作的工具，输出的结果类似甘特图、时序图这种，会配合 nvtx 使用清楚标注每一个事件的起始时间。

nsys 的工作流程 (The Workflow)

1. Step 1: 命令行分析 (Profile on the Command Line)

讲义中给出的命令是：

```
uv run nsys profile -o result.nsys.rep python benchmark.py
```

- nsys profile: 这是核心命令，告诉 Nsight Systems “开始录制！”。
- -o result.nsys.rep: 这个参数非常重要。-o 代表 output。它告诉 nsys 将所有录制下来的性能数据保存到一个名为 result.nsys.rep 的文件中。.rep 是报告 (report) 文件的后缀。
- python benchmark.py: 这是你想要分析的目标程序。

2. Step 2: 查看报告文件 (The .nsys-rep File)

3. Step 3: 图形界面分析 (Analyze in the GUI)

8.2.2. nvtx

NVTX API 本身是一套“空接口”或“占位符”。默认情况下，在你的代码中调用 NVTX 函数**不会产生任何实际操作，也不会带来性能开销**。

它的作用只在当你的程序被一个专门的“**开发者工具（如性能分析器）**”启动时才会被激活。这时，这些 NVTX 调用就会被该工具拦截，并转交（重定向）给工具内部的相应功能来处理。

比如一个运动员在跑道上跑步，而教练在赛道上某个位置（比如 100 米、400 米）打个标记方便后续分析，但这些标记不会对运动员跑步产生直接影响。

由于 NVTX 只定义了调用的规范（比如“标记一个时间范围的开始”），而没有规定具体如何响应，因此不同的开发者工具可以根据自己的需要，自由决定如何利用这些调用信息。例如，有的工具可能用它来计时，有的则用它来触发日志记录。

一些常见工具配合 NVTX 使用的例子

- **打印一条消息到控制台**
- **工具精确记录下每一次 NVTX 调用发生的具体时间点**。当程序运行结束后，工具会将所有这些带有时间戳的事件收集起来，并以时间轴的形式图形化地展示出来。这样可以直观地看到代码中不同标记阶段的执行顺序、持续时间以及它们之间是否存在重叠。
- **统计 NVTX 记录下的时间**
- **作为开关，在 NVTX 调用限定的范围内启用/禁用工具功能**
- **作为一个中转站，将数据转发到其他工具出处理**

NVTX 提供的注解类型

● Markers

基础注解类型，主要传递消息，提供了一些选择**颜色、类别**的参数(在 Ranges 里也有这些可选参数)，配合 Nsight System 这种可视化的性能分析工具使用。

```
nvtx.**mark**(  
    message: str | None = None,  
    color: str | int | None = 'blue',  
    domain: str | None = None,  
    category: str | int | None = None,  
    payload: int | float | None = None,)
```

● Ranges

Ranges 用于标记程序在**一段时间内**的活动，就像一对相关联的标记点（一个开始，一个结束）。它主要用来衡量一个函数或一段代码块的执行耗时。

这也是 cs336 希望我们使用的。

NVTX 提供了两种不同机制的范围，以适应不同的应用场景：

1. 推入/弹出式范围 (Push/Pop Ranges):

- **工作机制**：这种范围像一个**栈 (Stack)** 一样工作，遵循“后进先出” (Last-In, First-Out) 的原则。当你 ‘Push’ 一个新范围时，它就被推入栈顶；当你 ‘Pop’ 时，最顶端的范围被弹出。
- **核心特点**：它们必须是**严格嵌套**的，不能交叉重叠。‘Pop’ 操作总是自动与同一线程上最近一次的 ‘Push’ 操作配对。
- **适用场景**：非常适合标记结构清晰、层层调用的函数或代码块。例如，‘main’ 函数调用 ‘function_A’，‘function_A’ 又调用 ‘function_B’。

```

nvtx.push_range("数据处理总流程")
// 步骤 1: 加载数据
nvtx.push_range("加载数据")
...加载数据的代码...
nvtx.pop_range() // "加载数据"范围结束

// 步骤 2: 处理数据
nvtx.push_range("处理数据")
...处理数据的代码...
nvtx.pop_range() // "处理数据"范围结束

// 步骤 3: 保存结果
nvtx.push_range("保存结果")
...保存结果的代码...
nvtx.pop_range() // "保存结果"范围结束
nvtx.pop_range() // "数据处理总流程"范围结束
主程序结束

```

在 Nsight system 上的结果就是:

- 一个最长的、名为“数据处理总流程”的范围。- 在它**内部**，依次排列着三个互不重叠的短范围：“加载数据”、“处理数据”和“保存结果”。- 你永远不会看到“加载数据”还没结束，“处理数据”就开始的情况。这就是严格嵌套。

2. **开始/结束式范围 (Start/End Ranges):** - **工作机制:** ‘Start’ 操作会返回一个唯一的**句柄 (Handle)**，这个句柄就像一个凭证，唯一对应。你必须在未来的某个时刻调用 ‘End’，并将这个凭证传递给它，才能正确地关闭对应的范围。- **核心特点:** 它们可以**任意重叠**，并且一个范围的开始和结束可以发生在**不同的线程**上。- **适用场景:** 用于标记复杂的、异步的或并行的操作，这些操作的生命周期不是简单的嵌套关系。

主程序线程:

```

...
// 步骤 1: 加载数据
// 步骤 2: 并行处理数据
// 为工作线程 1 的任务创建一个范围，并拿到凭证 handle1
handle1 = nvtx.start_range("并行任务 1")
// 启动工作线程 1，把 handle1 交给它

// 为工作线程 2 的任务创建一个范围，并拿到凭证 handle2
handle2 = nvtx.start_range("并行任务 2")
// 启动工作线程 2，把 handle2 交给它

// 主线程可以继续做其他事，或者等待线程结束...

- ----- 时间流逝 -----

工作线程 1 (在另一个 CPU 核心上):
...执行任务 1 的代码...
// 任务完成，用凭证 handle1 关闭对应的范围
nvtx.end_range(handle1)

```



```
工作线程 2（在另一个 CPU 核心上）：  
...执行任务 2 的代码...  
// 任务完成，用凭证 handle2 关闭对应的范围  
nvtx.end_range(handle2)
```

在性能分析工具的时间轴上，你会看到：

- “并行任务 1”和“并行任务 2”这两个范围是在主线程上**几乎同时开始**的。
- 它们的执行过程在时间上是**重叠**的。
- 它们的结束点发生在各自的工作线程上，并且结束时间**可能不同**。

3. Resources

资源命名：

对于新的线程，在 Nsight 里面分析的结果默认显示 python (TID: 54321) 这种线程名，资源命名就是给这个线程起一个名字方便观测处理。

资源追踪：

是命名的扩展，对于一些标准的比如加锁、解锁的过程，NVTX 只需命名，其他工具可以自动识别并给予、释放资源；

对于一些不标准的，比如自己写的自旋锁，NVTX 除了命名之外，还需要明确指出哪里加锁了，哪里解锁了。

我的结果示例

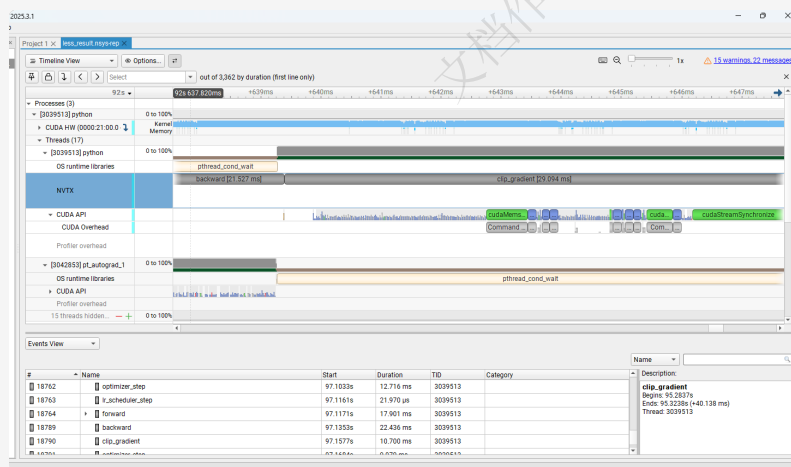


图 8.1: nvtx 结果示例

从结果上来看，没想到裁剪梯度这一步居然这么耗时。。。

8.3. 混合精度训练

到目前为止，在这个任务中，我们一直在使用 FP32 精度——所有模型参数和激活值都具有 torch.float32 数据类型。然而，现代 NVIDIA GPU 包含专门的 GPU 核心（张量核心），用于以较低精度加速矩阵乘法。例如，NVIDIA A100 规格表显示，其在 FP32 下的最大吞吐量为 19.5 TFLOP /秒，而使用 FP16（半精度浮点）或

BF16（脑浮点）时，最大吞吐量显著提高至 312 TFLOP/秒。因此，使用较低精度的数据类型有助于加快训练和推理速度。

然而，若简单地将我们的模型转换为低精度格式，可能会导致模型准确度降低。在此情况下，混合精度训练（Mixed Precision Training）提供了一种解决方案。它允许我们使用较低精度（如 FP16 或 BF16）来计算模型参数和激活值，同时保持 FP32 精度来计算梯度。这不仅提高了训练速度，还避免了精度丢失问题。

总结混合精度训练的特点：

朴素的训练默认使用 32 位浮点数（FP32），而混合精度训练引入了 16 位浮点数（FP16 或 BF16），主要带来两大好处：

1. **减少显存占用**：模型参数、梯度和优化器状态占用的显存几乎减半。这意味着可以用同样的显存训练更大的模型，或者使用更大的批量（batch size）。
2. **加快训练速度**：现代 NVIDIA GPU 内置了 Tensor Cores，专门用于加速 FP16/BF16 的矩阵运算，其理论吞吐量远高于 FP32。使用低精度计算能充分利用这部分硬件性能，带来显著的训练加速（通常能提升 1.5 倍至 3 倍）。

但是，混合精度训练也存在两大挑战：

1. **数据溢出**：直接把所有东西都变成 FP16 是行不通的，因为它的数值范围（约 $6e-5$ 到 65504）太小，容易出现**上溢（Overflow）**变成‘inf’和**下溢（Underflow）**变成‘0’，导致训练崩溃。
2. **下溢（Underflow）**：指一个数因为**太小**（过于接近零），超出了当前数据类型能表示的最小精度范围，结果被强制舍入为**零**。这就像用一把最小刻度是“毫米”的尺子去测量一粒灰尘，由于灰尘太小，你只能记录下它的长度是“0 毫米”。
3. **上溢（Overflow）**：指一个数因为**太大**，超出了当前数据类型能表示的最大范围，结果变成了一个特殊值——**无穷大（‘inf’）**。这就像用一把 30 厘米的尺子去测量一张 1 米长的桌子，你只能说它的长度“超出了尺子的范围”。
4. **舍入误差**：当网络模型的反向梯度很小时，一些 FP32 能够表示的数值可能不能满足 FP16 精度下的表示范围，导致被强行舍入，带来误差。比如 $0.66\dots6$ （32 位）被舍入成 $0.66\dots7$ （16 位）。

Keynote 8.3

溢出是范围问题，它是指数值超出了 FP16 表示的范围；而舍入误差是精度问题，它发生在数值可以被 FP16 表示范围呢，但精度不能够被 FP16 表示。

8.3.1. 浮点数知识回顾

定点数表示小数点位置是固定的，而浮点数则是用科学计数法来表示数值。

IEEE 浮点表示

$$V = (-1)^s \times M \times 2^E$$

- 符号 s : $s=1$ 代表负数, $s=0$ 代表正数
- 尾数 M : 尾数是浮点数的有效精度部分;
e.g. $6.75 = 1.6875 \times 2^2$, 1.6875 就是尾数 M 。尾数隐含以 1 开头, $M=1+f$, 实际存的只有小数点后面的数
- 阶码 E : 决定数值的范围;
值得注意的是在规格化下: 指数位会存在一个偏移, 偏移的好处是为了方便数值大小的比较和避免 0 的不唯一表示。偏移量 $bias = 2^{k-1} - 1$; 比如 8 位指数的偏移量 $bias = 127$ 。
- 最后存储的指数 $E = e() + bias$

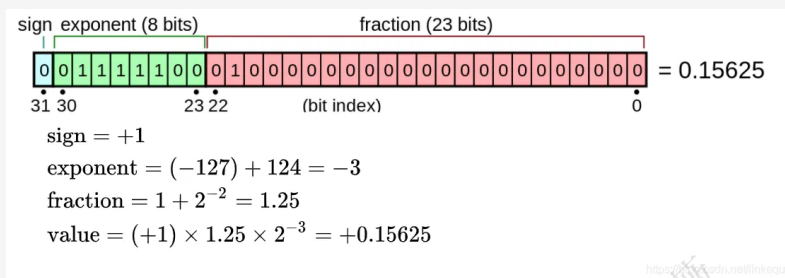


图 8.2: IEEE 浮点表示

半精度浮点数、单精度浮点数和双精度浮点数

- 半精度浮点数 (FP16) 包括 1 位符号位, 5 位指数, 10 位尾数。
- 单精度浮点数 (FP32) 包括 1 位符号位, 8 位指数, 23 位尾数。
- 双精度浮点数 (FP64) 包括 1 位符号位, 11 位指数, 52 位尾数。

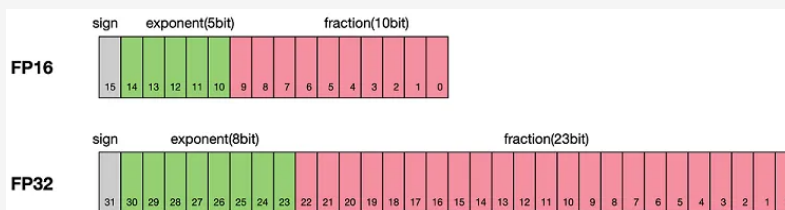


图 8.3: 半精度浮点数、单精度浮点数和双精度浮点数

规格化、非规格化和特殊值

规格化: 指数 E 既不是全 0, 也不是全 1; $E = e - bias$, 对于单精度全 0 $\rightarrow -126$, 全 1 $\rightarrow 127$. $bias = 2^{k-1} - 1$; 尾数部分隐含 1 开头的情况; 大部分都是这种情况。

非规格化: 指数 E 全 0; 阶码 $E = 1 - bias$; 尾数不全 0, 没有隐含的 1, $0.xxxxx \times 2^{-126}$ 为范围 (单精度)。

特殊值: 尾数和指数全为 0 时, 代表 ± 0

指数全为 1, 尾数全为 0 时, 为 $\pm\infty$

当指数全为 1, 尾数非全 0, 为 NaN

8.3.2. 混合精度训练的实现

梯度缩放 (Loss Scaling): 解决数据溢出问题

举个例子，在模型前向传播、计算损失使用 FP16 精度计算完了之后，某个权重计算得到的真实梯度是 ‘1e-6’，如果不使用缩放还使用 FP16 去存储的话，那么因为 ‘1e-6’ 已经不能被 FP16 半精度所表示，它就自动被视为 0，这样就成了梯度消失，学习无效。

而缩放就是在计算得到梯度之后给它乘一个缩放因子，比如 65536，乘了之后的结果就是 FP16 可以表示的了。

Keynote 8.4

在深度学习训练中，尤其是在模型后期趋于收敛时，计算出的梯度值经常会变得非常非常小，比如 ‘1e-6’，‘1e-7’ 等，所以一般缩放因子都比较大，只针对下溢而非上溢出。

权重备份 (Weight Backup): 解决舍入误差

假设你有一个 FP16 格式的权重，值是 ‘0.9824’。现在计算出一个梯度更新量，在 FP32 下是 ‘0.0010001’。但在 FP16 下，这个更新量可能被舍入成 ‘0.001000’。所以 ‘0.9824 + 0.001000’ 在 FP16 下可能是 ‘0.9834’。而最后一位这个小更新就永久丢失了。一次两次没问题，成千上万次迭代后，大量的小更新都丢失了，模型就学偏了或不收敛了。

权重备份的思路就是把权重备份一个 FP32 精度的，每次计算梯度更新权重的时候就用 FP32 的来计算，得到新结果再转换成 FP16，方便下一次计算梯度、激活等操作。

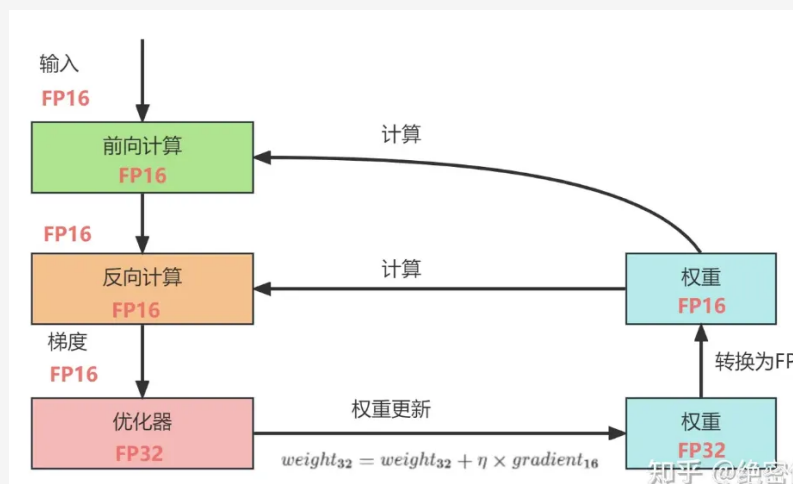


图 8.4: 权重备份

精度累加 (precision accumulated): 解决舍入误差

与权重备份类似，精度累加也是利用了 FP32 精度作为中介来解决舍入误差的问题，区别在于精度累加的过程是在**计算矩阵乘法运算**的时候，计算的结果用 FP32 保存减少舍入误差，然后再转成 FP16 格式进行下一步计算。



图 8.5: 精度累加

Keynote 8.5

事实上，在解决舍入误差的问题上，更新后的 FP32 主权重转换回 FP16 用于下一次计算时，会引入一次新的舍入误差。但这个误差是一次性的、非累积的。真正关键在于，所有微小的梯度更新信息已经被精确地、永久地累积到了 FP32 的主权重上。我们避免了最致命的累积误差问题。

8.4. 显存分析

参考资料 8.3

https://hugging-face.cn/blog/train_memory

此前我们主要探讨了计算的时间性能，接下来将聚焦于**显存**这一语言模型训练与推理中的关键资源。PyTorch 自带强大的内存分析工具 Memory Profiler，可实时追踪内存分配动态。

对于训练和推理大语言模型 (LLM) 来说，显存 (GPU Memory) 和计算的时间性能一样，都是极其宝贵的资源。显存瓶颈常常会导致 “Out of Memory” (OOM) 错误，限制了我们能使用的模型大小和批量大小 (batch size)。因此，精确地分析显存都用在了哪里，是性能优化的关键一步。

Memory Profiler 的核心流程是：

1. 通过代码启动内存历史记录，运行需要分析的目标代码，

```
torch.cuda.memory._record_memory_history(max_entries=1000000)
```

2. 然后将记录到的内存分配快照 (snapshot) 保存成一个 ‘pickle’ 文件。

```
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")
```

3. 停止记录。

```
torch.cuda.memory._record_memory_history(enabled=None)
```

4. 可视化工具分析。

- 使用官方工具：访问图片中提到的在线工具 ‘https://pytorch.org/memory_viz’。
- 加载快照文件：将你本地生成的 ‘memory_snapshot.pickle’ 文件拖拽到这个网页上。
- 解读分析结果：
 - 内存时间线 (Timeline)：你会看到一个图表，显示了总的预留显存 (Reserved Memory) 和已使用显存 (Allocated Memory) 随时间的变化。这可以帮助你快速定位显存峰值出现在哪个阶段。
 - 内存分配详情 (Allocations)：工具会列出每一次具体的内存分配事件。最关键的是，它会提供每一次分配的**大小 (Size)** 和**代码堆栈追踪 (Stack Trace)**。通过堆栈追踪，你可以精确地知道是你的代码中的哪一行（例如，‘model.forward()’ 里的某个特定操作）导致了这次内存分配，从而实现精确优化。

示例代码：

```
import torch
import torch.nn as nn

# 确认你的环境支持 CUDA

if not torch.cuda.is_available():
    print("CUDA is not available. This script requires a GPU.")
else:
    device = torch.device("cuda")

# 1. 定义一个简单的模型
class SimpleModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(in_features=1024, out_features=2048)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(in_features=2048, out_features=512)

    def forward(self, x):
        return self.layer2(self.relu(self.layer1(x)))

# 2. 实例化模型和输入数据，并移动到 GPU
model = SimpleModel().to(device)
# 创建一个需要梯度的输入张量
input_tensor = torch.randn(256, 1024, device=device, requires_grad=True)

# 3. 开始记录内存历史
print("===== 开始内存性能分析 =====")
```



```
torch.cuda.memory._record_memory_history(max_entries=1000000)

# --- 我们想要分析的核心代码段 ---
# 执行一次前向传播
output = model(input_tensor)
# 计算一个标量损失
loss = output.sum()
# 执行一次反向传播
loss.backward()
# -----

# 4. 保存内存快照到文件
snapshot_filename = "simple_model_snapshot.pickle"
torch.cuda.memory._dump_snapshot(snapshot_filename)

# 5. 停止记录
torch.cuda.memory._record_memory_history(enabled=None)

print(f"===== 性能分析结束，快照已保存至 '{snapshot_filename}' =====")
print("请访问 <https://pytorch.org/memory\_viz> 并上传该文件进行分析。")
```

微言大义 8.1

注意这里分析的是 GPU 的显存而非主机的内存。

分析计算性能（对应时间）和显存使用（对应空间）是非常重要的优化手段。

它让我们从**完成**跨越到**完美**。

文档作者：喂喂薇

在上一节利用 Nsight Systems 等工具对模型进行计算分析, 可以看到限制模型内存和计算资源的主要瓶颈主要还是在 Attention 注意力计算上。因此, 我们优化的目标也会集中在此。仔细分析过后可以发现, Attention 计算的瓶颈主要在于 IO 感知上, FlashAttention 的优化方法应运而生。

9.1. GPU 内存架构

参考资料 9.1

- <https://www.cnblogs.com/ArsenalFanInECNU/p/18021724>

9.1.1. 物理内存架构: 片上内存与片下内存

从硬件位置来看, GPU 内存主要分为两类: **片上 (on-chip) 内存** 和 **片下 (off-chip) 内存**。

- **片上内存 (On-chip Memory):** 位于 GPU 芯片内部, 主要用作高速缓存 (Cache)、共享内存 (Shared Memory) 和寄存器 (Register)。

- **特点:** 速度极快, 但存储空间非常小。
- **硬件类型:** 通常是 **SRAM** (静态随机存取存储器), 其优点是访问速度快, 无需刷新即可保存数据。

- **片下内存 (Off-chip Memory):** 位于 GPU 芯片外部, 主要用作全局内存 (Global Memory), 也就是我们常说的“显存”。

- **特点:** 容量大, 但速度相对较慢。
- **硬件类型:** 通常是 **HBM** (高带宽内存), 它通过将多个 DDR 芯片堆叠并与 GPU 封装在一起, 以实现大容量和高位宽。HBM 属于 **DRAM** (动态随机存取存储器), 其成本较低、密度高, 但访问速度慢于 SRAM。

9.1 GPU 内存架构 . . .	101
9.1.1 物理内存架构: 片上内存与片下内存	101
9.1.2 逻辑内存层次与功能划分	102
9.2 FlashAttention . . .	103
9.2.1 GPU 的 FLOPs 计算能力与内存吞吐能力	104
9.2.2 I/O 感知	104
9.2.3 FlashAttention 如何充分考虑 GPU 的内存层级结构?	104
9.2.4 实现细节	106
9.2.5 Flash Attention 的计算量分析	109
9.3 FlashAttention2 与 FlashAttention1 的区别	110
9.3.1 FlashAttentionv3	112
9.4 Triton	112
9.4.1 Triton 的核心思想: 并行化与分块 (Parallelism and Tiling)	113
9.4.2 Triton 的“分块”特性 . . .	114
9.4.3 Triton 和 C/C++ 的编译方式的联系和区别	115
9.5 用 Triton 优化 FlashAttention . . .	116
9.5.1 前向传播	116
9.5.2 反向传播	118

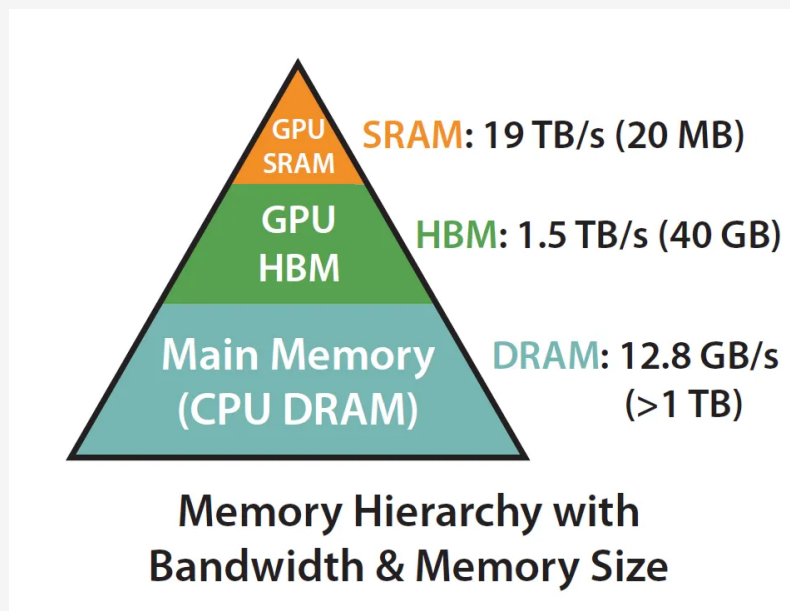


图 9.1: GPU 内存架构

9.1.2. 逻辑内存层次与功能划分

在 CUDA 编程模型中, GPU 内存根据其作用域、生命周期和访问特性被划分为不同的逻辑类型。其速度和容量关系通常是: **寄存器 > 共享内存/L1 缓存 > L2 缓存 > 全局内存**。

以下是各类内存的详细介绍:

1. 寄存器 (Register)

- **位置与速度**: 位于片上, 是 GPU 中速度最快的内存空间。
- **作用域**: 每个线程独享的私有资源, 用于存储线程内频繁使用的临时变量。
- **生命周期**: 与核函数 (Kernel) 的执行周期一致, 核函数运行结束即被释放。
- **特点**: 容量非常有限。如果一个线程需要的变量过多, 寄存器会发生“溢出”, 多出的变量将被存放到速度慢得多的本地内存中, 严重影响性能。

2. 本地内存 (Local Memory)

- **位置与速度**: 物理上它与全局内存位于同一区域, 即片下的 HBM/DRAM 中, 因此访问延迟高、速度慢。
- **作用域**: 和寄存器一样, 是每个线程的私有内存。
- **用途**: 主要用于存放两种数据: □ 编译器无法确定索引的本地数组; □ 因体积过大或数量过多而无法放入寄存器的变量 (即寄存器溢出)。

3. 共享内存 (Shared Memory)

- **位置与速度**: 位于片上 (on-chip), 是可编程的内存, 访问速度非常快, 几乎和寄存器一样。
- **作用域**: 被一个线程块 (Block) 内的所有线程共享, 可用于块内线程间的高效通信。不同线程块之间无法通过共享内存通信。
- **生命周期**: 与线程块的生命周期一致, 线程块执行开始时分配, 执行结束时释放。

4. 全局内存 (Global Memory)

- **位置与速度**: 位于片下 (off-chip) 的 DRAM/HBM 中, 是 GPU 上容量最大但访问速度最慢的内存空间。通过 'cudaMalloc' 等函数分配的内存就在这里。
- **作用域**: 所有线程都可以访问, 是实现不同线程块之间数据通信的唯一途径。
- **生命周期**: 与整个应用程序的生命周期相同, 除非被显式释放。
- **缓存**: 对全局内存的访问会经过 L1 和 L2 缓存来提速。

5. 常量内存 (Constant Memory)

- **位置与速度**: 物理上驻留在片下的设备内存中, 但每个 SM 都有一个专用的只读常量缓存 (Constant Cache), 因此读取速度很快。
- **作用域**: 对所有线程可见, 但它是只读的。
- **用途**: 主要用于存储在核函数执行期间不会改变的数据。当一个 Warp (一组线程) 中的多个线程需要访问同一个常量数据时, 常量缓存可以实现广播 (broadcast), 一次性满足所有请求, 避免了串行访问, 从而提高效率。

6. L1/L2 缓存 (Cache)

- **位置**: L1 缓存位于每个 SM 内部, 被该 SM 内的 CUDA 核心共享。L2 缓存则被 GPU 上所有的 SM 共享。
- **硬件类型**: 它们都是片上 SRAM。
- **作用**: 由系统自动控制, 对程序员不完全透明。它们主要用于缓存对慢速内存 (如全局内存和本地内存) 的访问, 以减少访问延迟。FlashAttention 这类算法的核心思想就是尽可能利用 L1/L2 缓存来减少对 HBM 的读写。

9.2. FlashAttention

参考资料 9.2

- **参考**: [FlashAttention-猛猿-知乎](#)
- **flashAttention 动画演示**-[bilibili](#)

开宗明义: FlashAttention 的背景是在当前原始的 Attention 在 GPU 中, 存写速度是逊色于计算速度的, 也就是说存写相比于计算时拖了后腿的。为了达到最佳效率, 我们需要减少多次存写, 哪怕以稍微降低计算速度的代价。

9.2.1. GPU 的 FLOPs 计算能力与内存吞吐能力

多年来，GPU 的计算能力（FLOPS）的增长速度比增加内存吞吐量（TB/s）更快。

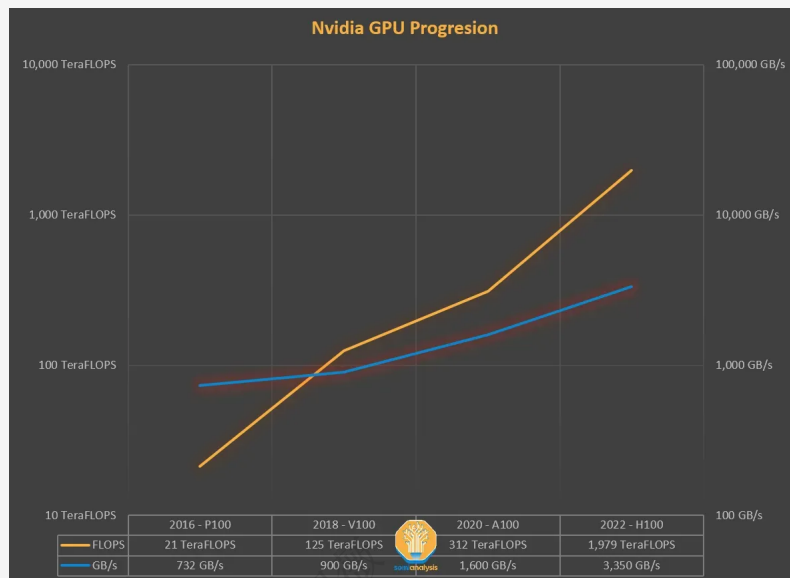


图 9.2: GPU 的 FLOPs 计算能力与内存吞吐能力

如果没有数据需要处理，那么额外的 FLOPS 的计算能力是没有意义的。总而言之，只有**存算**合理配合，才能达到最佳效率。

9.2.2. I/O 感知

IO 感知（IO-awareness）是一种在设计算法时充分考虑**硬件输入/输出（I/O）开销**的优化思想，特别是在 GPU 这类**计算速度远超内存访问速度**的设备上尤为重要。

在现代 GPU 中，性能瓶颈往往不是浮点运算（FLOPs）的速度，而是从**显存（HBM）读写数据的速度**。因此，IO 感知的算法旨在通过优化内存访问模式，特别是**减少对速度较慢、但容量较大的高带宽内存（HBM）的读写次数**，来减少整体的运行时钟时间（wall-clock time）。

简单来说，IO 感知的核心就是：**尽可能减少与慢速内存的数据交换，让计算尽可能在快速内存中完成。**

9.2.3. FlashAttention 如何充分考虑 GPU 的内存层级结构？

标准注意力算法的缺点

标准注意力算法没有感知到 IO 的成本，它会频繁地读写 HBM。例如，它需要计算并存储一个巨大的 $N \times N$ 的中间注意力矩阵（S 和 P）到 HBM 中，然后再从 HBM 中读出进行下一步计算。这些冗余的 HBM 读写操作（IO）占据了大量的计算时间，成为了性能瓶颈。

Transformer 的计算瓶颈不在运算能力，而在读写速度上。

Keynote 9.1

- **FLOPS**: 等同于 $FLOP/s$ ，表示 **Floating Point Operations Per Second**，即每秒执行的浮点数操作次数，用于衡量硬件计算性能。
- **FLOPs**: 表示 **Floating Point Operations**，表示某个算法的总计算量（即总浮点运算次数），用于衡量一个算法的复杂度。

FlashAttention 的性能提升方式

1. **利用 SRAM 进行分块计算 (Tiling)**: 为了避免对 HBM 的反复读写, FlashAttention 的核心思路是将多个操作融合成一个单一的 GPU 核 (Kernel)，并利用速度快 10 倍左右的 SRAM 进行计算。

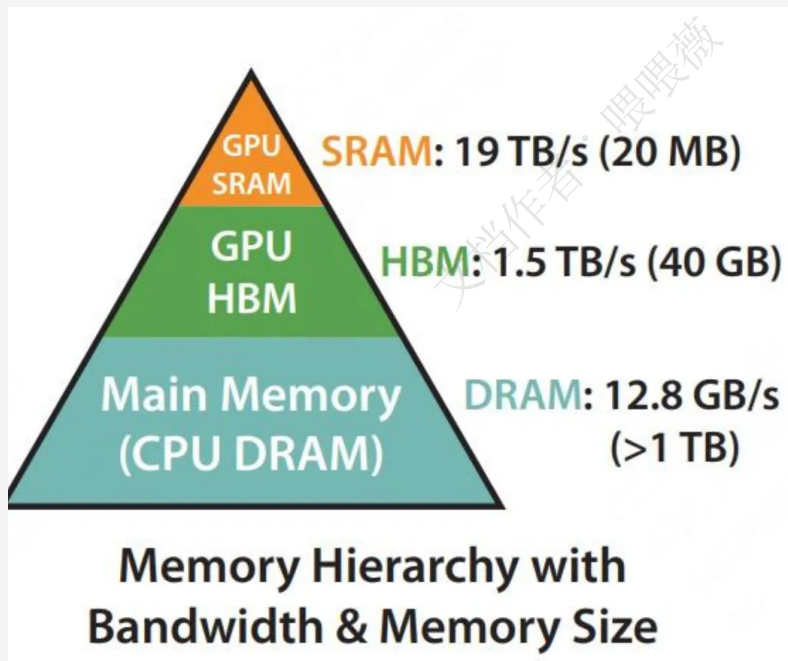


图 9.3: GPU 体系架构

- 它将输入的查询 (Q)、键 (K)、值 (V) 矩阵从 HBM 中切分成块 (Blocks/Tiles)。
- 然后，它逐块将这些数据加载到高速的 SRAM 中。所有的中间计算步骤，包括矩阵乘法 and Softmax，都在 SRAM 内部完成，完全不产生将巨大的 'N×N' 中间矩阵写回 HBM 的操作。
- 最后，只将计算完成的最终输出结果从 SRAM 一次性写回 HBM。

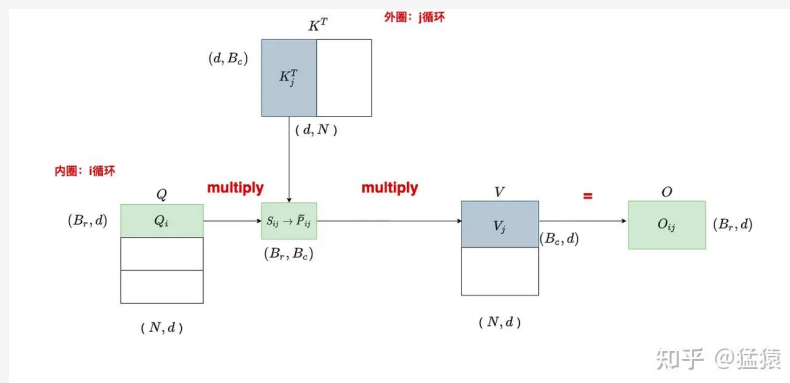


图 9.4: FlashAttention v1 内外循环

2. **通过重计算 (Recomputation) 减少内存占用**: 为了进一步优化内存, FlashAttention 在前向传播时不会保存用于反向传播的中间注意力矩阵 (S 和 P)。在反向传播时, 它会利用保存在 SRAM 上的输入块以及少量统计数据, 快速地重新计算出这些中间值。这种“以时间换空间”的策略极大地节省了 HBM 的占用。

9.2.4. 实现细节

原始 Attention 计算

$$\text{Attention}(Q, K, V) = \text{softmax}((Q * K^T) / d_k) * V$$

缩放部分 $\sqrt{d_k}$ 的计算不占额外存储, 可忽略掉。主要考量 $Q * K^T \text{softmax}() * V$ 这三个运算

FlashAttention 的分块计算流程

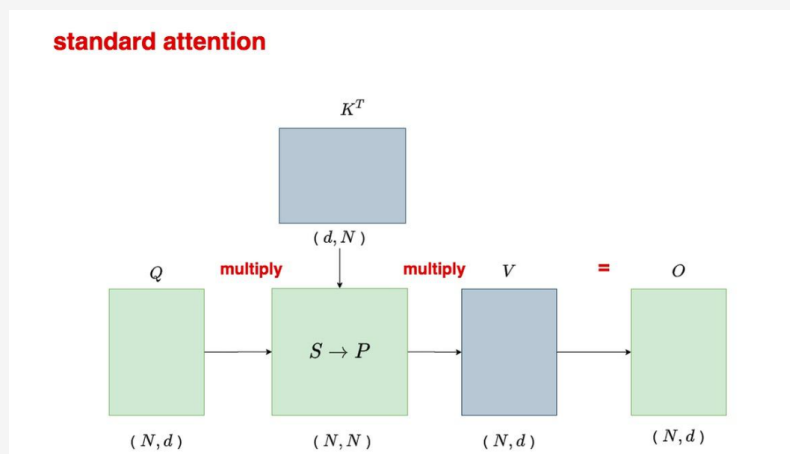


图 9.5: 分块计算

1. 首先, 将 Q 矩阵 (形状为 (N, d)) 按照“行”切为 T_r 块 (block), 每块的长度为 B_r 。用 Q_i 来表示切完后的某块矩阵, 则 Q_i 的维度为 (B_r, d) 。不难

理解， Q_i 中存储着某 B_r 个 token 的 query 信息。

2. 然后，将 K^T 矩阵(形状为 (d, N))按照“列”切为 T_c 块，每块的长度为 B_c 。用 K_j^T 表示切完后的某块矩阵，则 K_j^T 的维度为 (d, B_c) 。易知 K_j^T 中存储着某 B_c 个 token 的 key 信息。
3. 同样，将 V 矩阵也按照“行”切为 T_c 块，每块长度为 B_c 。用 V_j 表示切完后的某块矩阵，则 V_j 的维度为 (B_c, d) 。易知 V_j 中存储着某 B_c 个 token 的 value 信息。

计算初始 attention 分数：

$$S_{ij} = Q_i * K_j^T = (B_r, d) * (d, B_c) = (B_r, B_c)$$

图中的 S_{ij} 表示前 B_r 个 token 和前 B_c 个 token 间的原始相关性分数。

具体循环中的操作示意：

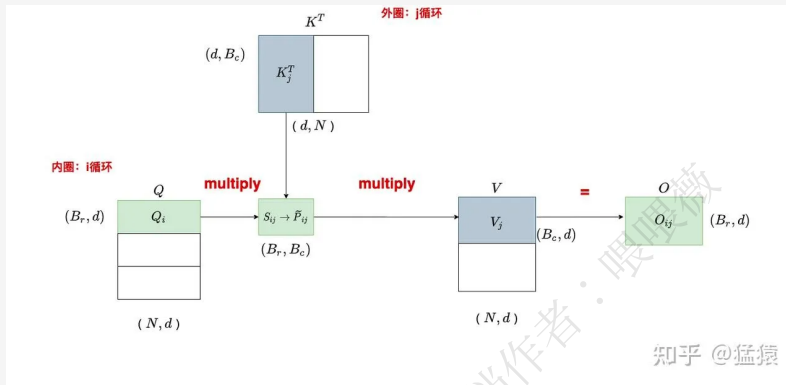


图 9.6: FlashAttention v1 内外循环

softmax 计算操作

原始 softmax 计算： $\text{softmax}(x_i) = \exp(x_i - m) / \sum_j \exp(x_j - m)$ 其中 m 是某一行列的最大值，减去它是为了数值稳定性。

定义：

- $m(x)$ ：标准场景下，该行的全局最大值
- $m(x^{(1)})$ ：分块 1 的全局最大值
- $m(x^{(2)})$ ：分块 2 的全局最大值

那么易知： $m(x) = m([x^{(1)}, x^{(2)}]) = \max(m(x^{(1)}), m(x^{(2)}))$

- $f(x)$ ：标准场景下， $e^{x-m(x)}$ 的结果
- $f(x^{(1)})$ ：分块场景下， $e^{x^{(1)}-m(x^{(1)})}$ 的结果
- $f(x^{(2)})$ ：分块场景下， $e^{x^{(2)}-m(x^{(2)})}$ 的结果

那么易知： $f(x) = [e^{m(x^{(1)})-m(x)} f(x^{(1)}), e^{m(x^{(2)})-m(x)} f(x^{(2)})]$

- $l(x)$ ：标准场景下， $\text{rowsum}[f(x)]$ 的结果

● $l(x^{(1)})$: 分块场景下, $\text{rowsum}[f(x^{(1)})]$ 的结果

● $l(x^{(2)})$: 分块场景下, $\text{rowsum}[f(x^{(2)})]$ 的结果

那么易知: $l(x) = [l(x^{(1)}), l(x^{(2)})]$ 。

$$\text{softmax}(x) = \frac{f(x)}{l(x)} = \frac{[e^{m(x^{(1)})-m(x)} f(x^{(1)}), e^{m(x^{(2)})-m(x)} f(x^{(2)})]}{e^{m(x^{(1)})-m(x)} l(x^{(1)}) + e^{m(x^{(2)})-m(x)} l(x^{(2)})}$$

分块计算操作:

$$m(x) = \max_i (x_i)$$

$$f(x) = [e^{x_1-m(x)}, \dots, e^{x_n-m(x)}]$$

$$l(x) = \sum_i f(x)_i$$

$$\text{softmax}(x) = \frac{f(x)}{l(x)}$$

FlashAttention 的在线 Softmax 算法

FlashAttention 的核心思想是: 我们可以**迭代地**更新 Softmax 的计算结果。每当一个新的数据块到来时, 我们用它来**修正**之前基于旧数据块计算出的 (不完整的) 结果。

让我们把注意力计算中的一行 (例如, Q_i 和所有 K_j 的点积) 看作是要计算 Softmax 的输入向量 x 。这个向量 x 被分成了多个块 $x_1, x_2, \dots, x_{T_c}$ 。

算法为这一行维护三个核心变量:

- O : 当前的输出 (对 V 的加权和)。
- m : 到目前为止见过的所有 x 元素中的最大值 (**running maximum**)。
- l : 到目前为止 Softmax 分母的累加和 (**running denominator**)。

现在, 我们模拟算法的迭代过程:

假设我们已经处理了第 1 到 $j-1$ 个块, 得到了当前的统计量 m_{old} 和 l_{old} , 以及当前的输出 O_{old} 。

现在, 我们处理第 j 个块, x_j 。

1. 计算当前块的局部统计量

我们只看当前块 x_j , 可以计算出它的:

- 局部最大值: $m_j = \max(x_j)$
- 局部 Softmax 分子: $P_{tilde_j} = e^{x_j - m_j}$ (注意, 减去的是局部最大值)
- 局部 Softmax 分母: $l_j = \text{sum}(P_{tilde_j})$

2. 合并统计量, 更新全局最大

新的全局最大值, 一定是旧的全局最大值和当前块最大值中的较大者: $m_{new} = \max(m_{old}, m_j)$

3. 重新缩放 (Re-scaling) 并更新分母 l

这是最关键的数学技巧。我们之前的 l_{old} 是基于 m_{old} 计算的, 而 l_j 是基于 m_j 计算的。现在我们有了新的全局最大值 m_{new} , 我们需要把这两部分都统一到新的基准上再相加。

$$l_{new} = l_{old} * e^{m_{old}-m_{new}} + l_j * e^{m_j-m_{new}}$$

理解这个公式:

- l_{old} 最初是 $e^{x_{old}-m_{old}}$ 。乘以 $e^{m_{old}-m_{new}}$ 后，它就变成了 $e^{x_{old}-m_{new}}$ 。
- l_j 最初是 $e^{x_j-m_j}$ 。乘以 $e^{m_j-m_{new}}$ 后，它就变成了 $e^{x_j-m_{new}}$ 。
- 两者相加，就得到了处理完 ‘j’ 块数据后，基于新的全局最大值 m_{new} 的正确的分母总和！

4. 重新缩放并更新输出 O

输出 O 是对值矩阵 V 的加权和。它的更新逻辑和分母 l 完全一样，也需要重新缩放。

$$O_{new} = (O_{old} * l_{old} * e^{(m_{old} - m_{new})} + (P_{tilde_j} * V_j) * e^{(m_j - m_{new})}) / l_{new}$$

理解这个公式:

- 分子部分 (...) 是计算了未归一化的加权和，并将其统一到 m_{new} 基准上。
- $P_{tilde_j} * V_j$ 是当前块 x_j 对 V_j 的加权和。
- 最后，除以新的、正确的总分母 l_{new} ，就得到了更新后的、正确的输出 O 。

在论文的算法图中，这个公式被写作： $O_i \text{diag}(l_i^{new})^{-1} (\text{diag}(l_i) * e^{(m_i - m_i^{new})} * O_i + e^{(m_i - m_i^{new})} * P_{tilde_{ij}} * V_j)$

这只是上述逻辑的矩阵化写法。 $\text{diag}(l)$ 乘以 O 是为了还原出未归一化的加权和。

在反向传播中，因为中间计算的结果没有存储，所以要重新计算一下。

9.2.5. Flash Attention 的计算量分析

主要设计两部分矩阵计算：

对于 $S_{ij} = Q_i K_j^T$ ，其中 $Q_i \in \mathbb{R}^{B_r \times d}$, $K_j^T \in \mathbb{R}^{d \times B_c}$ 。根据前置知识，求 S_{ij} 的计算量为 $O(B_r B_c d)$ 。

对于 $\tilde{P}_{ij} V_j$ ，其中 $\tilde{P}_{ij} \in \mathbb{R}^{B_r \times B_c}$, $V_j \in \mathbb{R}^{B_c \times d}$ 。则这里的计算量同样为 $O(B_r B_c d)$ 。

接下来我们看一共计算了多少次 (1) 和 (2)，也就是执行了多少次内循环：

$$T_c T_r = \frac{N}{B_c} \frac{N}{B_r}$$

综合以上三点，flash attention 的 forward 计算量为： $O\left(\frac{N^2}{B_c B_r} B_r B_c d\right) = O(N^2 d)$

注意，因为计算量是用大 O 阶表示的，所以这里我们把常数项都省略了。

IO 复杂度分析

操作 (Operation)	标准注意力 (Standard Attention)	FlashAttention	备注 (Remarks)
HBM -> SRAM (读操作)			
读 Q, K, V	$O(Nd)$	$O(Nd)$	这是无法避免的初始加载。
读中间矩阵 S 或 P ($N \times N$)	$O(N^2)$	$\emptyset$ (不读取)	核心区别: 标准注意力需 S/P 写回 HBM 再读出, FlashAttention 完全避免这一步。
SRAM -> HBM (写操作)			
写中间矩阵 S 或 P ($N \times N$)	$O(N^2)$	$\emptyset$ (不写入)	核心区别: FlashAttention 中间计算结果保留在 SRAM 中, 用完即弃。
写最终输出 O	$O(Nd)$	$O(Nd)$	最终结果必须写回。
总 I/O 复杂度	$O(N^2 + Nd)$	$O(Nd)$	当 $N \gg d$ 时, N^2 项成为不可逾越的瓶颈。

9.3. FlashAttention2 与 FlashAttention1 的区别

v2 主要提升有两点:

1. 引入多线程提升了并行的能力
2. 继续优化了细节, 比如延迟归一化使得内部的除法运算进一步减少。

v2 最主要的改进: 原 v1 在进行 QK^TV 的分块计算时, 是把 K^T 放到最外层循环的, QV 放到内层循环。v2 将 Q 放到最外层, K^TV 放到了内层。

从图里面可以看到, 对于 v1, K^T 是外层循环, 这时候每次和 Q 相乘的内循环得到的中间结果是红色圈中的“列”, 这个列是没办法直接和 V 相乘得到完整的结果的, 它需要等到下一次 K^T 的循环得到下一个列与 V 相乘得到的结果求和才是最终的 O 。

而对于 v2, Q 是外层循环, Q 的每一行是每次循环的变量, 这时候这一行可以和所有的内循环的 K^T 列相乘得到中间结果的行, 即图中画蓝、红、紫的圈, 这几个行可以单独与 V 也进行相乘, 得到最终结果 O_{00} 、 O_{10} 等单个元素, 不需要等待下一次循环。这样做好处就是可以把 Q 按照行进行各自独立的分割, 每一行之间互不影响, 就可以放到多个线程里面并行优化。

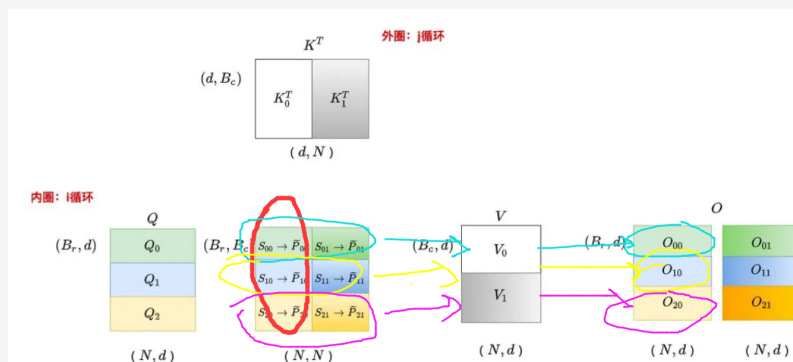


图 9.7: FlashAttention v2 内外循环

并行策略的进一步增强

- **FlashAttention-1**: 其并行化主要在 **批次 (batch size)** 和 **头 (head)** 这两个维度上进行。每个注意力头分配一个线程块。当序列很长而批次和头数较少时，可并行的线程块数量可能远少于 GPU 上的流多处理器 (SM) 数量，导致 GPU 利用率低下。
- **FlashAttention-2**: 在 V1 的基础上，增加了在 **序列长度 (sequence length)** 维度上的并行化。
 - **前向传播**: 将 Q 矩阵按行切分，不同的 **行块 (row blocks)** 分配给不同的线程块并行计算。
 - **反向传播**: 由于梯度计算的依赖关系不同 (dK 和 dV 需要按行累加)，为了优化，反向传播是按 **列块 (column blocks)** 分配给不同的线程块并行计算的。

Warp 间工作分区的优化：减少内部通信

即使在单个线程块 (Thread Block) 内部，V2 也优化了工作分配方式。一个线程块由多个 Warp (通常每个包含 32 个线程) 组成。

- **FlashAttention-1**: 采用了一种称为 **“Split-K”** 的策略。它将 K 和 V 矩阵沿着序列维度切分给不同的 Warp，而 Q 块对所有 Warp 可见。这种方式的缺点是，每个 Warp 计算出的是部分结果，为了得到最终的行输出，必须将这些中间结果写入共享内存 (SRAM)，进行 Warp 间的同步和聚合 (相加)，这引入了不必要的通信和共享内存读写开销，成为性能瓶颈。
- **FlashAttention-2**: 转而采用 **“Split-Q”** 策略。它将 Q 矩阵切分给不同的 Warp，而 K 和 V 块对所有 Warp 可见。因为不同 Q 的计算是独立的，每个 Warp 在计算完自己的 $(Q_{slice} * K^T) * V$ 后，直接得到最终输出 O 的一个分片，**无需与其他 Warp 进行通信或聚合**。这极大地减少了共享内存的读写和同步开销，是 V2 速度提升的关键之一。

算法层面的优化

FlashAttention-2 在算法层面进行了一些精简，以减少开销较高的非矩阵乘法浮点运算（non-matmul FLOPs）。

- **延迟归一化 (Rescaling)**: 在 V1 中，每一次内循环迭代，都需要用当前的局部 softmax 分母 l 来重新缩放（rescale）累积的输出 O ，这涉及到除法运算。FlashAttention-2 优化了这一点，在内循环中，它只更新 O 的“未归一化”版本，将最终的除法操作**延迟到外循环结束时才执行一次**。这个调整减少了中间步骤的除法运算次数，更充分地利用了为矩阵乘法优化的 Tensor Cores。
- **反向传播存储优化**: V1 为了反向传播时的重计算，需要从 HBM 中读取并保存两个统计量 m (最大值) 和 l (exp 分数总和)。V2 则将它们**合并，只存储一个 $L = m + \log(l)$ (即 log-sum-exp)**，在反向传播时由此恢复所需信息。这进一步减少了一次 HBM 的 I/O 操作，节省了内存带宽。

9.3.1. FlashAttentionv3

相比于 FlashAttentionv1、v2，v3 没有太多算法上的创新，主要有两个性能提升的点：

1. **矩阵计算 (GEMM) 和 softmax 计算流水线化**，交错并行，不严格顺序执行，而是会有时间的重叠。
2. **硬件加速的 FP8 低精度计算与精度保持**。

9.4. Triton

参考资料 9.3

- Triton 官方中文文档: <https://triton-lang.cn/main/index.html>

Triton 是一个由 OpenAI 开发的、基于 **Python** 的编程语言的编译器，可以让我们用类似 Python 的语法为 GPU 编写高效的**计算内核 (Kernel)**。这样就不用学习复杂的 CUDA C++ 编程，就能写出优化 CUDA 代码性能的**自定义算子**，降低了 GPU 编程的门槛。

我们以实现 Triton 的向量加法为例子：

假设我们想实现一个很简单的“向量 + 向量 $\rightarrow$ 向量”的加法计算。绝大多数情况我们都不会考虑底层优化，而是直接在 PyTorch 上直接 **`c = a + b`** 了，这样做当然可以，而且 Pytorch 也会有它自己的底层优化机制。但如果你想做一个 PyTorch 没有的、非标准的、或者多个操作想融合在一起的操作（比如 **`c = a * b + \sin(a)`**），PyTorch 就需要依次调用多个内核，达不到最佳性能。

这个时候如果我们还想实现最佳的性能，方式就是写一个最佳的优化算子，一套“组合拳”直接实现 **`a * b + \sin(a)`** 计算。传统的方式就是手动写 CUDA C++，直接与 GPU 底层交互，不过需要我们自己管理线程块（Thread Blocks）、线程（Threads）、共享内存（Shared Memory）、内存对齐和合并访问（Coalescing）等一系列复杂的底层细节。

Extension 9.1 算子融合

算子融合是将多个独立的小操作（如乘法、加法、Mask 操作）合并成一个单一的、更高效的“大”操作（称为 ****Kernel****）。这大大减少了从内存中读取和写入数据的次数，也减少了启动 GPU 计算任务的开销。

这个时候 Triton 的作用就得以体现，它让我们用类似 Python 的较为简单的语法来实现底层 CUDA 优化的性能。

9.4.1. Triton 的核心思想: 并行化与分块 (Parallelism and Tiling)

Triton 编程的核心是**并行化**。我们不会写一个程序来处理整个矩阵‘**X**’，而是会写一个**程序模板（Triton Kernel）**，GPU 会同时启动成百上千个这个程序的**实例（Program Instance）**，每个实例只负责一小部分工作。

还是以我们之前说的向量加法为例子：

```
import torch
import triton
import triton.language as tl

@triton.jit # Triton 内核函数装饰器，只有装饰器内的函数才会被 Triton 编译
def add_kernel(
    x_ptr, # 指向输入向量 a 的指针，就和 C 语言的指针差不多，直接和显存地址交互
    y_ptr, # 指向输入向量 b 的指针
    output_ptr, # 指向输出向量 c 的指针
    n_elements, # 向量中的元素总数
    BLOCK_SIZE: tl.constexpr, # 块大小，一个编译时常量
):
    # --- 内核的执行逻辑 ---
    # GPU 通过大规模并行来获得高性能。Triton 会启动很多个 add_kernel 的程序实例来同
    # 时处理数据。pid 就是当前这个实例的唯一编号（从 0 开始）。
    # 计算当前这个程序实例的 ID
    pid = tl.program_id(axis=0)
    # 计算当前程序实例要处理的数据块的偏移量
    # tl.arange 生成一个 [0, 1, 2, ..., BLOCK_SIZE-1] 的数组
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE) # 偏移量 它定义了当前这个 pid
    # 对应的程序实例负责处理哪一段数据。

    # 创建一个 mask，防止内存越界
    # 比如向量长度是 1000，BLOCK_SIZE 是 1024，最后一个块会访问到不存在的元素
    mask = offsets < n_elements

    # 从全局内存（HBM）加载数据块
    # mask=mask 保证了只加载有效的数据
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)

    # 执行计算
    output = x + y

    # 将计算结果写回全局内存
```

```
tl.store(output_ptr + offsets, output, mask=mask)
```

接下来我们将在 CPU 上调用这段函数

```
size = 98432 # 创建输入数据
a = torch.randn((size,), device='cuda')
b = torch.randn((size,), device='cuda')
output = torch.empty_like(a) # 准备一个空的输出 tensor
# 定义执行配置
BLOCK_SIZE = 1024
# grid 这个元组定义了我们启动多少个 add_kernel 的程序实例。
grid = (triton.cdiv(size, BLOCK_SIZE),) # cdiv 是向上取整的除法
# 方括号里传入 grid 配置 像调用普通函数一样传入参数
add_kernel[grid](a, b, output, size, BLOCK_SIZE=BLOCK_SIZE)
# 验证结果
print(torch.allclose(output, a + b)) # 输出 True
```

总结下来有如下几个要点：

- Triton 内核函数一定要加 `@triton.jit` 修饰符
- 每个内核函数在被实际调用的时候都只是众多实例中的一个，它只负责部分计算，所以要确保它的计算在正确的地址上（对应**输入的指针 + 单位偏移量 offset**）
- 有了指针地址后才能从 GPU 上加载到数据，执行完我们想要的运算之后在写回去。
- 一些细节比如**mask**是考虑到了数据块的大小和真实输入的大小。

9.4.2. Triton 的”分块”特性

其实 Triton 的分块性能更好，刚才向量加法的例子没能体现，接下来我们讲解讲义上矩阵乘法的例子来充分证明这一点。

对于矩阵乘法 `'C = A @ B'`，我们不直接计算 C，而是把 C**分块 (tiling)**，我们利用 Triton 启动很多个实例，每个实例就去计算其中的一个小块，对于每一个小块的计算，执行以下步骤：

- 从输入矩阵 'A' 中加载一**行块**，从矩阵 'B' 中加载一**列块**。
- 执行 `'tl.dot'`（点积）运算，得到一个中间结果。
- 将 'A' 的指针向右移动一个块的距离，'B' 的指针向下移动一个块的距离，加载下一对块。
- 执行 `'tl.dot'` 并将结果**累加**到上一步的结果上。
- 重复 c, d 步，直到遍历完 'A' 的所有列和 'B' 的所有行。

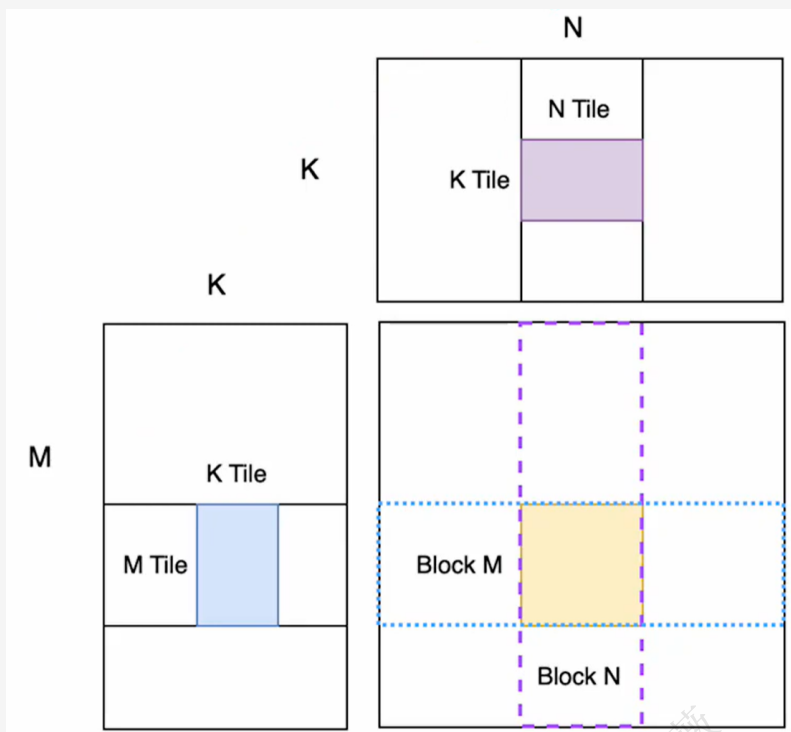


图 9.8: Triton 矩阵乘法

除此之外，我们会使用 `tl.make_block_ptr()` 这一智能指针来替代我们之前的手写的方式

```
a_block_ptr = tl.make_block_ptr(    # 创建一个描述 A 矩阵上数据块的“智能指针”
    base=a_ptr,                      # 矩阵的基地址
    shape=(M, K),                    # 整个矩阵的形状
    strides=(stride_am, stride_ak),  # 矩阵的步长
    offsets=(pid_m * BLOCK_SIZE_M, 0), # 当前块的起始偏移（行，列）
    block_shape=(BLOCK_SIZE_M, BLOCK_SIZE_K), # 定义要操作的块形状
    order=(1, 0)                     # 内存布局顺序，通常是（1，0），代表行主序，即先行
    ↪ 后列，（0，1）就反过来
)

# 在循环中，使用 tl.advance 来移动指针
a_block_ptr = tl.advance(a_block_ptr, (0, BLOCK_SIZE_K))
# 语义非常清晰：“在第 1 个维度（K 维）上前进 BLOCK_SIZE_K”

# 加载时，使用 boundary_check 自动处理边界，不用 mask 了
a = tl.load(a_block_ptr, boundary_check=(0, 1)) # 检查第 0 和第 1 个维度
```

9.4.3. Triton 和 C/C++ 的编译方式的区别和联系

C/C++ 编译器比较通用，负责将通用程序翻译成 CPU 或 GPU 代码，开发者需要手动处理大量底层细节。而 Triton 编译器**垂直性更强**，它专门针对 GPU 高性能计算，接收高度抽象的、以**数据块**为单位的指令，并**自动**完成最困难的性能优化，将开发者从复杂的 GPU 硬件细节中解放出来。

C/C++ (含 CUDA C++): 编程模型更底层。在为 GPU 编写 CUDA C++ 时，你需要手动管理：

- **线程层级**: 精确定义线程块 (Block) 和网格 (Grid) 的维度。
- **内存管理**: 手动管理共享内存的分配和同步，以实现线程间通信和数据重用。
- **内存访问**: 程序员需要自己精心设计内存访问模式，以确保“合并访问”，避免性能大幅下降。

Triton: 编程模型抽象层次更高。你不再直接操作单个线程，而是从**程序实例**的**视角**出发，操作**数据块**。

- **块级编程**: 你通过 ‘tl.program_id’ 获取当前程序实例的 ID，然后加载、计算和存储一整块数据 (例如 ‘tl.load(ptr + offsets)’)。
- **自动优化**: 你无需关心共享内存、合并访问或线程同步。Triton 编译器会分析你的块级操作，**自动生成**管理共享内存、保证合并访问的底层 PTX 代码。它为你处理了最复杂的部分。

9.5. 用 Triton 优化 FlashAttention

9.5.1. 前向传播

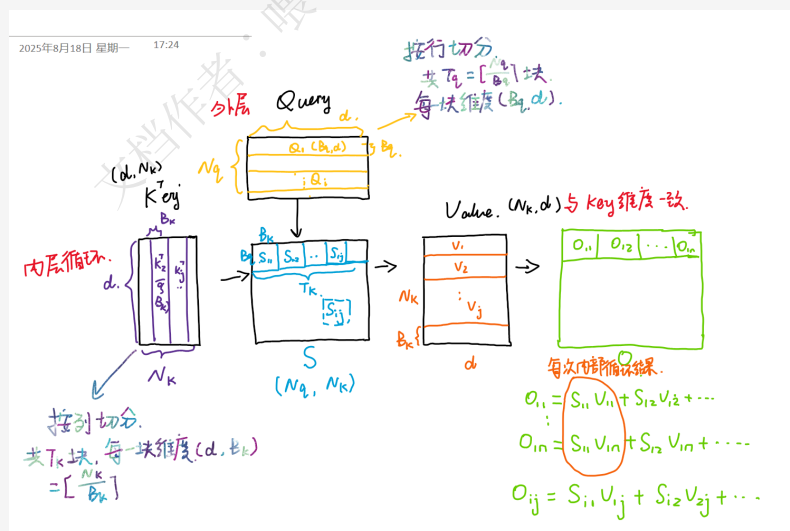


图 9.9: FlashAttention 计算示意图

就像我们之前提到的，我们想要实现性能的提高，就要尽可能地**减少对 HBM 的读写**，因为计算速度远大于通信速度。这也是我们利用 Triton 优化 FlashAttention 的核心思想。

在讲义中，我们 Triton 被要求设置的网格维度是 (Tq, batch_size)，也就是并行的两个维度。事实上，Triton 可以并行的数量不止是两个维度，并且一般 FlashAttention 并行的维度是三个，分别是 (Tq, batch_size, head)。讲义为了简化，只要求并行两个维度，取消了多头。

Extension 9.2 为什么在 FlashAttention 中 Triton 除了批次和多头之外只并行了 Q?

还是我们之前所说的，我们的核心思想是为了尽可能减少通信，用计算代替通信。如果把 K 也并行起来，那么每次一个小线程就是计算一小块 Q 和一小块 K 的点积，而最后要得到 softmax 的结果，需要一行所有的点积和，这就需要每个线程块之间进行通信，这就会大大降低性能。只并行 Q，那么一个线程就可以得到 S 一行的所有值，就不必再通信了。

在每一个线程的前向传播中， K^T 的小块是它的列， V 的小块是它的行。我们中间会得到 $S = \frac{QK^T}{\sqrt{d_k}}$ 的小块，用 M 来维护 S 每一行的最大值， L 来维护 S 每一行的 exp 分数总和 (随着循环计算的进行， M 和 L 会不断更新)。然后我们用 $P = \exp(S - M)$ 的小块来得到 softmax 的结果，用 $O = PV$ 的小块来得到输出。

薇言大义 9.1 这里我觉得很值得仔细想清楚的一点是后面的 O 矩阵的计算，它不是想前面 S 那样，每次一个线程就得到一块 S_{ij} ，而是每次一个线程就得到一整行 O_i ，然后随着循环不断叠加更新，最后得到最终的 O_i 。

```
Q_i = tl.load(Q_block_ptr) #(Q_TILE_SIZE, D)
O_i_acc = tl.zeros((Q_TILE_SIZE, D), dtype=tl.float32) #(Q_TILE_SIZE, D)
L_i_acc = tl.zeros((Q_TILE_SIZE, 1), dtype=tl.float32) #(Q_TILE_SIZE, 1)
M_i_acc = tl.full((Q_TILE_SIZE, 1), float('-inf'), dtype=tl.float32) #(Q_TILE_SIZE,
↪ 1)
# 外层循环：对 K/V 的序列长度进行分块
for j in range(tl.cdiv(N_KEYS, K_TILE_SIZE)):
    K_j = tl.load(K_block_ptr) #(K_TILE_SIZE, D)
    V_j = tl.load(V_block_ptr) #(K_TILE_SIZE, D)

    S_ij = tl.dot(Q_i, K_j.T) * scale #(Q_TILE_SIZE, K_TILE_SIZE)
    M_i_new = tl.max(S_ij, axis=1, keep_dims=True) #(Q_TILE_SIZE, 1)
    P_ij = tl.exp(S_ij - M_i_new) #(Q_TILE_SIZE, K_TILE_SIZE)
    L_i_new = tl.exp(M_i_acc - M_i_new) * L_i_acc + tl.sum(P_ij, axis=1,
↪ keep_dims=True) #(Q_TILE_SIZE, 1)
    # 数据对齐
    P_ij_cast = P_ij.to(V_block_ptr.type.element_ty) # 这代表从指针全局内存中读取类型，
↪ 用 V_j.dtype 理论上也行。
    O_i_new = tl.exp(M_i_acc - M_i_new) * O_i_acc + tl.dot(P_ij_cast, V_j)
    ↪ #(Q_TILE_SIZE, D)
    # 与 pytorch 代码不同，这里必须显式地更新 M_i_acc 和 O_i_acc，因为下一个循环不会记住
    ↪ 上一次的值
    M_i_acc = M_i_new
    O_i_acc = O_i_new
    L_i_acc = L_i_new

# 只有 K 和 V 指针在 K 维度上前进，遍历所有 K/V 分块
K_block_ptr = K_block_ptr.advance((K_TILE_SIZE, 0))
V_block_ptr = V_block_ptr.advance((K_TILE_SIZE, 0))
```

```

O_i = O_i_acc / L_i_acc # 全局归一化
L_i = M_i_acc + tl.log(L_i_acc) # 这个保存下来给反向传播用
tl.store(O_block_ptr, O_i)
tl.store(L_block_ptr, L_i)

```

9.5.2. 反向传播

反向传播计算公式：

$$S = \frac{QK^T}{\sqrt{d}}$$

$$P_{ij} = \exp(S_{ij} - L_i)$$

$$dV = P^T dO$$

$$dP = dO V^T$$

$$dS_{ij} = P_{ii} \circ (dP_{ii} - D_i)$$

$$dQ = \frac{dSK}{\sqrt{d}}$$

$$dK = \frac{dS^T Q}{\sqrt{d}}$$

正向计算公式：

$$S = \frac{QK^T}{\sqrt{d}}$$

$$P_{ij} = \text{softmax}_j(S)_{ij}$$

$$O = PV$$

反向传播：

根据链式法则：若 $Y = AX$ ，则 $\frac{\partial L}{\partial X} = A^T \frac{\partial L}{\partial Y}$

则 V 的梯度 dV (也就是 $\frac{\partial L}{\partial V}$ 损失)

$$\text{有 } dV = P^T dO$$

$$\text{而 } dP = V dO^T \Rightarrow dP = dO V^T$$

$$\Rightarrow \text{同理可求 } dQ = \frac{dSK}{\sqrt{d}}, dK = \frac{dS^T Q}{\sqrt{d}}$$

★ 对于 $P_{ij} = \text{softmax}(S)_{ij}$ 这一步的反向梯度推导：

$$\text{元素级: } P_j = \frac{e^{S_j}}{\sum_k e^{S_k}}$$

P_j 对应样本 S_j 。
存在样本 $S_k = S_j$ 和 $S_k \neq S_j$ 两种情况。

令分母 = Z

① 若 j 恰好与 k 相等

$$\text{则 } \frac{\partial P_j}{\partial S_k} = \frac{e^{S_j} Z - e^{S_j} \cdot e^{S_j}}{Z^2} = \frac{e^{S_j}}{Z} \left(1 - \frac{e^{S_j}}{Z}\right) = P_j (1 - P_j)$$

② 若 $j \neq k$ 。

$$\text{则 } \frac{\partial P_j}{\partial S_k} = \frac{0 - e^{S_j} \cdot e^{S_k}}{Z^2} = -P_j P_k$$

换算到矩阵中，只有 $j=k$ 也就是行列一致对角的时候值为 $P_j (1 - P_j)$ 。

其余为 $-P_j P_k$ 。

$$\Rightarrow \text{diag}(P) - PP^T$$

$$\therefore \text{原式 } dS_i = d\text{-softmax}(P_i) = [\text{diag}(P_i) - P_i P_i^T] dP_i$$

图 9.10: 反向传播数学公式推导

反向传播并不会直接利用前向传播的所有计算结果，而是需要**重新计算**，同样还是为了尽可能减少通信，用计算代替通信。

反向传播也要像前向传播那样按照循环来写，只是会更容易，因为参数里面会给 Q, K, V, O, L ，所以总体上只需要按照公式来

反向传播的逻辑（链式法则）：根据链式法则，一个变量的梯度等于所有流经它的路径上的梯度之和。

对于 dK 。 K 的第 j 个分块 K_j ，在前向传播中，它不仅和 Q 的第 0 个分块 Q_0 作用了，也和 $Q_1, Q_2, \dots, Q_{Tq-1}$ 都发生了作用。

当我们的外层循环 $i = 0$ 时，我们计算了 K_j 因为和 Q_0 相互作用而产生的梯度贡献。当外层循环 $i = 1$ 时，我们又计算了 K_j 因为和 Q_1 相互作用而产生的梯度贡献。... 以此类推。

所以， K_j 的**总梯度**，必须是它与**所有** Q_i 相互作用产生的梯度的**总和**。这就是为什么用 $+=$ 来进行累加。如果用 $=$ 直接赋值，那么当外层循环 i 走到下一个值时，上一次计算出的梯度贡献就会被覆盖和丢失，最后得到的 dK 是不完整的，只反映了 K 和最后一个 Q 分块 Q_{Tq-1} 交互的梯度。

dV 的计算也是完全相同的道理。

对于 dQ ，逻辑是类似的，只是循环的内外层反了过来。在计算 dQ 的第 i 个分块 dQ_i 时，它也需要累加来自所有 K_j 和 V_j 的梯度贡献，所以在我们的实现中， $dQ[:, i * Bq : (i + 1) * Bq, :] += dQ_{ij}$ 也是在累加。

与前向传播不同，反向传播是**把 K 放在了外层循环，把 Q 放在了内层循环**。这是为了让每一个并行的计算单元（线程块）都能独立、无冲突地完成一整块梯度（ dK 或 dV ）的计算，并将累加过程完全限制在各自高速的 SRAM 中。不过就算反过来也可以，只是会增加时延。

Extension 9.3 只从公式里面看，好像 dQ 和 dK 不都分别依赖了所有 K 和 Q 吗？那凭什么要把 K 放在外层循环，把 Q 放在内层循环？

这是一处非常重要的细节，把 K 放在外层是很重要的操作，这样做是因为如果把 Q 放在外层，需要很多线程块之间进行通信（下面会具体展开），而把 K 放在外层，就可以巧妙避免。

1. 已知: Q, K, V, O, L, dO ; 还可先求出 $D = \text{rowsum}(dO \circ O)$ (逐元素相乘)

2. 外层循环: 遍历 K 和 V 的块 j 。

● 加载 K 和 V 整个矩阵

3. 内层循环: 遍历 Q 、 O 、 dO 的块 i 。

● 加载 Q_i 、 O_i 、 dO_i dQ 整个矩阵

● 先计算 $S_{ij} = Q_i * K_j^T / \text{sqrt}(d)$ ，得到 S_{ij} 的小块

● 再计算 $P_{ij} = \exp(S_{ij} - L_i)$ ，得到 P_{ij} 的小块

● 之后可以累加得到 $dV_j += P_{ij}^T @ dO_i$ ，即 dV_j 的小块

● 然后求 $dS_{ij} = P_{ij} * (dP_{ij} - D_i)$ ，得到 dS_{ij} 的小块 ($dP_{ij} = dO_i @ V_j^T$)

- dS 有两大用处，一是求 $dK_j+ = dS_{ij}^T @ Q_i$ ，得到 dK_j 的小块，二是求 $dQ_i+ = dS_{ij} @ K_j$ ，得到 dQ_i 的小块

那么，如果把 Q 放在外层会怎么样？

1. 外层循环：遍历 Q 、 O 、 dO 的块 i 。

- 加载 Q_i 、 O_i 、 dO_i 整个矩阵

2. 内层循环：遍历 K 和 V 的块 j 。

- 加载 K 和 V dQ 整个矩阵
- 先计算 $S_{ij} = Q_i * K_j^T / \text{sqrt}(d)$ ，得到 S_{ij} 的小块
- 再计算 $P_{ij} = \exp(S_{ij} - L_i)$ ，得到 P_{ij} 的小块
- 之后可以累加得到 $dV_j+ = P_{ij}^T @ dO_i$ ，即 dV_j 的小块
- 然后求 $dS_{ij} = P_{ij} * (dP_{ij} - D_i)$ ，得到 dS_{ij} 的小块 ($dP_{ij} = dO_i @ V_j^T$)
- dS 有两大用处，一是求 $dK_j+ = dS_{ij}^T @ Q_i$ ，得到 dK_j 的小块，二是求 $dQ_i+ = dS_{ij} @ K_j$ ，得到 dQ_i 的小块

可以看到关键的差别就在 Q_i 、 O_i 、 dO_i 的读取和 K 和 V 的读取上，后者的大小远大于前者，经过这么多次循环后，性能就产生了较大的差距。

对比项	K/V 在外	Q 在外	性能影响
HBM 读取 (K, V)	整个矩阵只被读 1 遍	整个矩阵被读 T_q 遍	灾难性的
HBM 读取 (Q, O, dO)	整个矩阵被读 T_k 遍	整个矩阵只被读 1 遍	占优
核心权衡	T_k 通常远小于 T_q （例如序列长度 8k，块大小 128，则 $T_q=64$ ；如果 K/V 序列长度一样， T_k 也是 64。但在交叉注意力等场景下， T_k 可能很小）。官方方案让被读取次数更多的矩阵 (Q) 在内层循环，被读取次数更少的矩阵 (K/V) 在外层循环。但更关键的是，原子写比重复读更高效。	你的方案让 K 和 V 被反复从 HBM 读取，这是 GPU 计算中最大的性能瓶颈。	
最终评价	最优。最大化了数据在 SRAM 中的复用，将 HBM 的访问降到了最低。	性能极差。导致了对 HBM 的冗余、重复访问，完全违背了 IO 感知的算法设计原则。	

10

分布式并行训练 (Distributed Data Parallel Training)

参考资料 10.1

分布式并行教程 <https://github.com/LambdaLabsML/distributed-training-guide?tab=readme-ov-file>

PyTorch 分布式官方文档 https://docs.pytorch.org/tutorials/beginner/dist_overview.html

cs336 第二篇的主旋律是优化已经实现的训练模型程序的性能；在第一节里我们重点放在了程序本身的优化, 包括如何与硬件协同。

第二节我们优化性能的方式从直观理解上看更加简单粗暴:

我一个人做一件任务要花很长时间, 那么两个人并行分担任务, 那时间自然就降下来了 (但不见得时间就一定变为原来的一半)。这到我们训练的场景上就是先用一块 GPU 训练, 现在用很多块 GPU 一起训练以实现效率提升, 即**分布式并行 (distribute parallel)**。

Extension 10.1 补充知识: 线程 (Thread) 和进程 (Process) 的比较

比较项	进程 (Process)	线程 (Thread)
定义	操作系统下分配资源的基本单位	进程下分配资源的基本单位, 是 CPU 调度 和 执行 的基本单位
比喻	工厂	工人
资源分配	拥有独立的内存地址空间、文件句柄、IO 设备	共享所属进程的绝大部分资源
数据共享	进程之间数据共享困难	线程间通信非常高效
创建开销	创建新进程的资源开销较大	创建新线程的开销较小
稳定性	相互独立, 一个进程崩溃不影响其他进程	一个线程崩溃会影响整个进程及其他线程
依赖关系	进程包含线程, 至少包含一个线程才能执行	线程一定属于某个进程, 无法独立存在

10.1. 单机多进程

顾名思义, 单机多进程是在一个物理服务器 (节点) 上启动多个独立的操作系统进程, 共同协作完成一个计算任务。在现在主流的深度学习训练中, 一般就是每个进程独立的控制一块 GPU (比如 DDP), 但也有一个进程控制多个 GPU 的情况

10.1	单机多进程	121
10.2	AllReduce 算法 . .	123
10.2.1	分布式通信算法 . . .	126
10.3	DDP	130
10.3.1	DDP 和 DP 的区别 . .	130
10.3.2	DDP 算法优化	131
10.4	混 合 并 行 (4D 并 行)	135
10.4.1	每种并行技术简介 . .	135
10.5	混合数据并行 . .	143
10.5.1	DP+PP (数据并行 + 流 水线并行)	143
10.5.2	3D 并行 (DP+PP+TP) .	144
10.5.3	4D 并	146
10.5.4	专家并行/MOE 并行 .	148
10.6	CUDA 通信机制知 识补充	148
10.6.1	主 机 与 设 备 (Host-Device) 通信 . .	150
10.6.2	设备内	151

(比如 Model parallelism、Pipeline Parallelism)。

单机多进程是实现数据并行 (DP, Data Parallelism) 的前提, 其核心思想是“**模型复制, 数据分片**”, 每个进程独立完成各自的任务, 再通过**集合通信 (Collective Communication)**来同步梯度, 加速训练。

Extension 10.2 为什么不采用多线程?

规避 Python 的全局解释器锁 (GIL): Python 的 GIL 机制规定, 在同一进程中, 任意时刻只允许一个线程执行 Python 字节码。这使得 Python 的多线程在计算密集型任务中无法实现真正的并行。而多进程模型中, 每个进程都有自己独立的 Python 解释器和内存空间, 因此不受 GIL 的限制, 能够实现真正的并行计算。

下面以讲义上所说的一个简单的单机多进程代码作为示例

这个示例是在 CPU 上进行多进程, 代码的目标是启动 4 个工作进程, 每个进程都生成一个随机整数张量, 然后通过一个名为**all-reduce**的分布式操作, 将这 4 个张量相加, 并将最终的总和结果广播回每个进程。

1. 开局指定 IP 地址和端口号, 指定通信方式是 Gloo(Gloo 既可用于 CPU 也可用于 GPU 通信, 但 GPU 通信性能不如**Nccl**)
2. 每个进程创建一个张量, 进行如下操作:
 - (a). **Reduce (规约)**: 从所有进程收集 data 张量。
 - (b). **Op (操作)**: 对收集到的所有张量执行指定的操作, 这里是 SUM(求和)。
 - (c). **All**: 将计算出的最终结果 (总和) 分发回所有的进程, 并用该结果覆盖它们各自原来的 data。

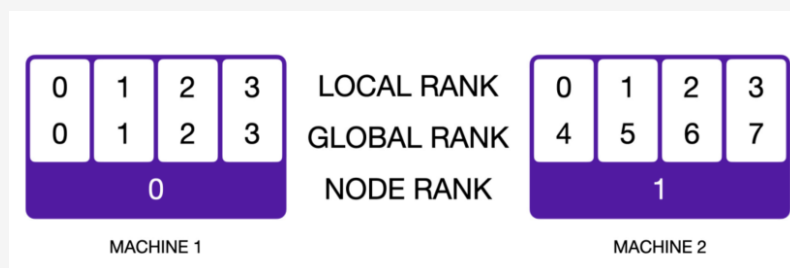


图 10.1: 各种 Rank 之间的关系

Keynote 10.1

区分 Node Rank、Local Rank 和 Global Rank

场景描述: 一个分布式应用运行在 2 台机器 (MACHINE 1 和 MACHINE 2) 上,world_size 为 8(因为 GLOBAL RANK 从 0 到 7)。

- **Node Rank (节点排名):** NODE RANK 为 0 和 1, 代表这是第 0 号和第 1 号机器。
- **Local Rank (本地排名):** 每台机器上都运行了 4 个进程, 因此在各自的机器内部, 它们的 LOCAL RANK 都是从 0 到 3。
- **Global Rank (全局排名):** GLOBAL RANK 是全局唯一的。MACHINE 1 上的 4 个进程拥有全局排名 0, 1, 2, 3; 而 MACHINE 2 上的 4 个进程则接着拥有全局排名 4, 5, 6, 7。

10.2. AllReduce 算法

参考资料 10.2

AllReduce 算法原理 <https://zhuanlan.zhihu.com/p/79030485>

AllReduce 算法原理 <https://juejin.cn/post/7084135971687497759>

我们理解计算机的算法都是基于一个一个函数操作组合在一起得到的, 那么我们在讲解分布式算法之前, 我们必须先了解一下组成这种算法所应用于硬件的函数操作——集合通信的基本概念,

Broadcast(广播): 将根服务器 (Root Rank) 上的数据分发广播给所有其他服务器 (Rank)

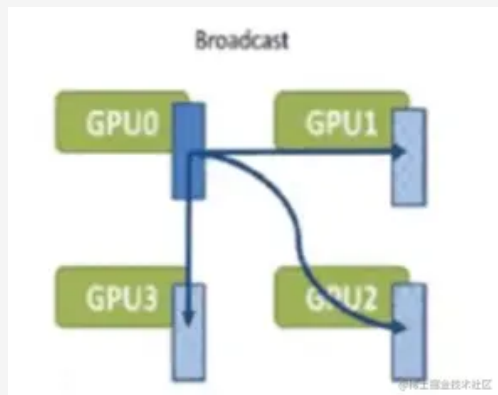


图 10.2: Broadcast

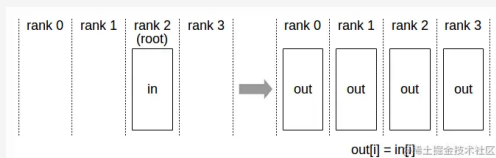


图 10.3: Broadcast

如图所示, 当一台服务器计算完成了自己部分的参数数据, 在分布式训练中想要把自己这部分数据同时发送给其他所有服务器, 那么这种操作方式就叫做广播 (broadcast)。

Scatter(散射): 将根服务器上的数据散射为**同等大小的数据块**, 每一个其他服务器得到一个数据块

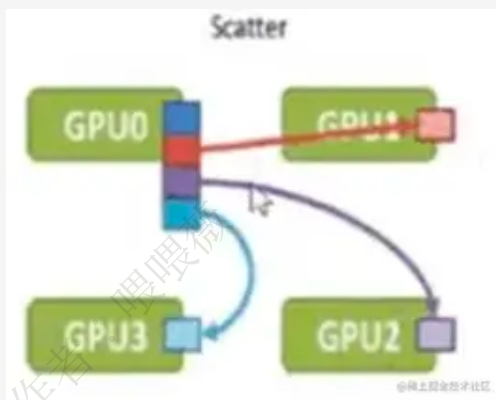


图 10.4: Scatter

如图所示, 当一台服务器计算完成自己部分的参数数据, 但是因为有时候服务器上全部的参数数据过大, 于是我们想要把这台服务器上的数据切分成几个同等大小的数据块 (buffer), 再按照序列 (rank index) 向其他服务器发送其中的一个数据块, 这就叫做散射 (Scatter)。

Gather(聚集): 将其他服务器上的数据块直接**拼接到一起**, 根服务器 (Root Rank) 获取这些数据

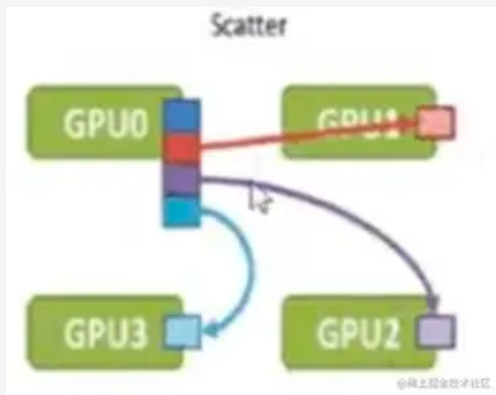


图 10.5: Gather

如图所示, 当服务器都做了散射之后, 每个服务器获得了其他服务器的一个数据块, 我们将一台服务器获得的数据块拼接在一起的操作就叫做聚集 (Gather)。

AllGather(全聚集): 所有的服务器都做上述 Gather 的操作, 于是所有服务器都获得了全部服务器上的数据

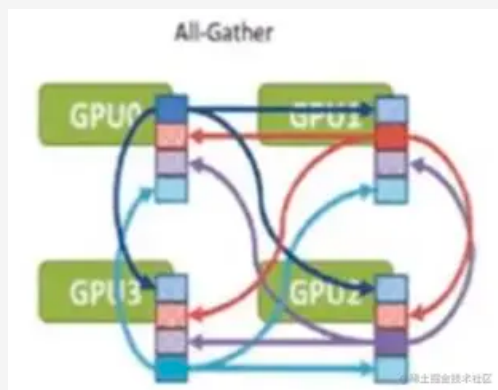


图 10.6: AllGather

如图所示, 所有的服务器都将自己收到的数据块拼接在一起 (都做聚集的操作), 那么就是全聚集 (AllGather)。

Reduce(规约): 对所有服务器上的数据做一个规约操作 (如最大值、求和), 再将数据写入根服务器

如图所示, 当所有服务器都做广播或散射的时候, 我们作为接收方的服务器收到各服务器发来的数据, 我们将这些收到的数据进行某种规约的操作 (常见如求和, 求最大值) 后再存入自己服务器内存中, 那么这就叫规约 (Reduce)。

AllReduce(全规约): 对所有服务器上的数据做一个规约操作 (如最大值、求和), 再将数据写入根服务器

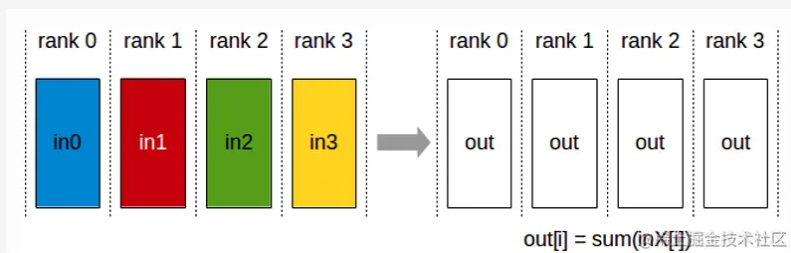


图 10.7: Reduce

如图所示, 同样**每一个服务器都完成上述的规约操作, 那么就是全规约**。这也就是分布式训练最基础的框架, 将所有数据通过规约操作集成到各个服务器中, 各个服务器也就获得了完全一致的、包含原本所有服务器上计算参数的规约数据。

ReduceScatter(散射规约): 服务器将自己的数据分为同等大小的数据块, 每个服务器将根据 index 得到的数据做一个规约操作即, 即先做 Scatter 再做 Reduce。

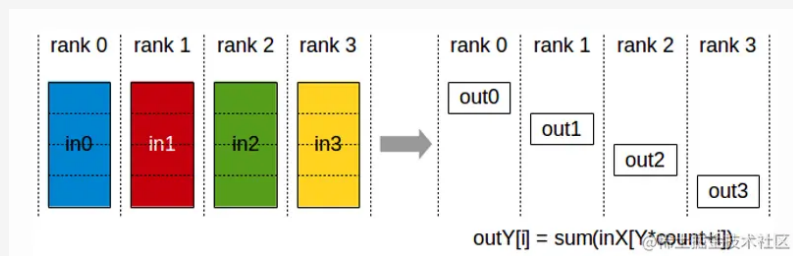


图 10.8: ReduceScatter

简单来讲, 就是先做散射 (Scatter), 将服务器中数据切分成同等大小的数据块, 再按照序列 (Rank Index), 每一个服务器所获得的参数数据做规约 (Reduce)。这就类似于全聚集, 只不过我们将数据不是简单拼接到一起而是做了规约操作 (求和或最大值等操作)。

10.2.1. 分布式通信算法

PS 算法 (Parameter Server, 参数服务器)

PS 算法是一种**中心化**的分布式训练架构。它将计算节点分为两种角色:

- 参数服务器/根服务器 (Parameter Server, PS) 负责**存储**和**更新**模型的全部参数;
- 工作节点 (Worker) 负责使用本地数据**计算梯度**, 并将梯度推送给 PS 做累积 (Reduce), 然后再从 PS 拉取最新的参数。

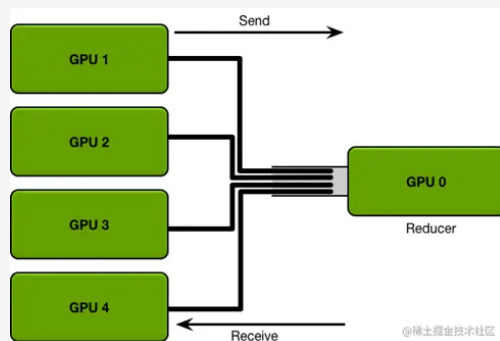


图 10.9: PS 算法示意图

缺点:

- 每一轮的训练迭代都需要所有卡都将数据同步完做一次 Reduce 才算结束, 并行的卡很多的时候,**木桶效应**就会很严重, 一旦有一张卡速度较慢会拖慢整个集群的速度, 计算效率低。
- **Reducer 服务器任务过重**, 成为瓶颈, 所有的节点需要和 Reducer 进行数据、梯度和参数的通信, 当模型较大或者数据较大的时候, 通信开销很大, 根节点收到巨量的数据, 从而形成瓶颈。

Halving and Doubling (HD) 算法 (Recursive Doubling)

阶段一: 规约-散射 (Reduce-Scatter):

1. 在 ' $\log_2(N)$ ' 步中, 节点间的通信距离 (步长) 按 1, 2, 4, ..., $N/2$ 的规律加倍。
2. 第 1 步 (距离=1): 节点 ' i ' 与节点 ' $i+1$ ' 配对。它们交换自己数据数组的一半的**一半**, 并进行累加。例如, 节点 ' i ' 发送前半部分给 ' $i+1$ ', 接收 ' $i+1$ ' 的前半部分并累加; 同时发送后半部分, 接收后半部分并累加。完成后, 节点 ' i ' 保留累加的前半部分, 节点 ' $i+1$ ' 保留累加的后半部分。
3. 第 2 步 (距离=2): 节点 ' i ' 与节点 ' $i+2$ ' 配对。它们交换的数据块是上一步规约后的结果, 数据量仍然是数组的一半。
4. ... 依此类推...
5. 阶段结果: 经过 ' $\log_2(N)$ ' 步后, 每个节点都只拥有整个数组的 ' $1/N$ ', 但这 ' $1/N$ ' 的部分是所有节点对应部分的全局规约结果。

阶段二: 全收集 (All-Gather):

1. 这个阶段是第一阶段的**逆过程**。
2. 在 ' $\log_2(N)$ ' 步中, 节点间的通信距离按 $N/2$, ..., 4, 2, 1 的规律减半。
3. 第 1 步 (距离= $N/2$): 在第一阶段最后一步通信的节点对 (例如 ' i ' 和 ' $i+N/2$ ') 互相交换它们持有的 ' $1/N$ ' 的最终结果。现在它们各自拥有了 ' $2/N$ ' 的全局结果。
4. ... 依此类推...
5. 最终结果: 经过 ' $\log_2(N)$ ' 步的逆向操作, 每个节点都逐步从邻居那里收集到了所有的最终数据块, 最终恢复出完整的全局规约结果。

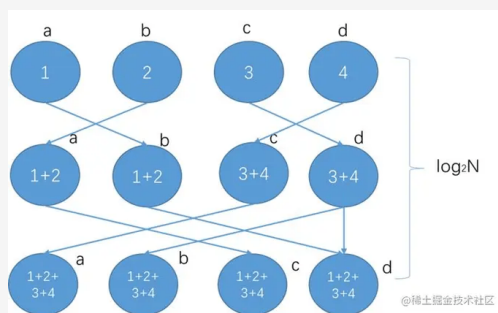


图 10.10: HD 算法示意图

Ring 算法 (Ring-AllReduce)

核心思想: 将数据分成 N 个块 (chunk), 通过两个阶段, 每个阶段 ' $N-1$ ' 步, 完成规约和分发。

工作流程:

阶段一: 散射-规约 (Scatter-Reduce):

1. 分块: 每个节点都将自己的数据数组分成 N 个小块。
2. 传递与累加: 在 ‘ $N-1$ ’ 步中, 进行循环传递。
3. 在第 ‘ k ’ 步, 每个节点 ‘ i ’ 会将自己的第 ‘ $(i-k+N)$ ’
4. 同时, 它会从上一个节点 ‘ $(i-1+N)$ ’
5. 阶段结果: 经过 ‘ $N-1$ ’ 步后, 每个数据块都完整地绕环一周, 并累加了所有节点上对应位置的值。此时, 每个节点 ‘ i ’ 都拥有一个最终的、全局规约后的数据块 (具体来说是第 ‘ $(i-1+N)$ ’)

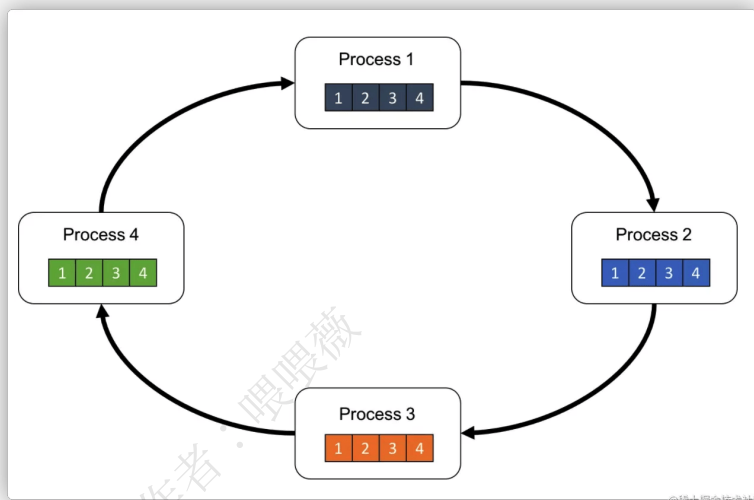


图 10.11: Ring 算法示意图 1

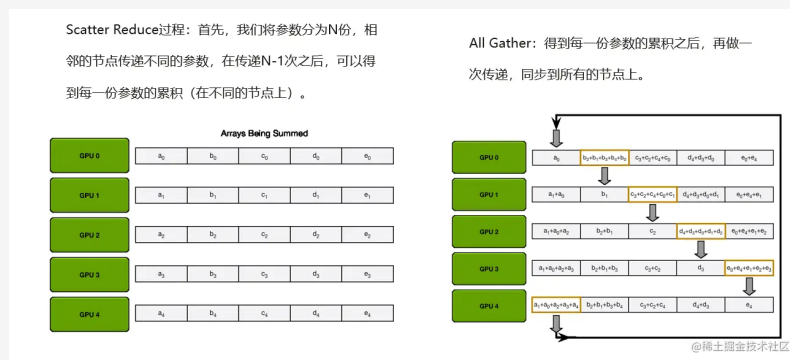


图 10.12: Ring 算法示意图 2

阶段二: 全收集 (All-Gather):

1. 再次传递: 现在每个节点都持有一个“最终版”的数据块, 目标是让所有节点都拥有所有 N 个“最终版”数据块。
2. 简单传递: 在接下来的 ‘ $N-1$ ’ 步中, 节点们只是单纯地传递它们手中的“最终版”数据块, 不再进行累加。

3. 最终结果: 经过 ' $N-1$ ' 步后, 所有最终块都在环上完整地跑了一圈, 每个节点都收集到了所有 N 个最终块, 从而得到了完整的全局规约结果。

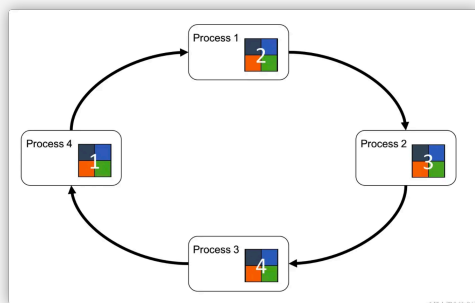


图 10.13: Ring 算法示意图 3

性能特点:

1. 延迟: 总共需要 ' $2 * (N-1)$ ' 次通信步骤, 延迟与节点数 ' N ' 成正比。
2. 带宽: 在任何时刻, 每个节点只发送和接收一小块数据 (大小为 ' M/N '). 算法可以使链路带宽持续被占满, 因此带宽利用效率非常高。它被认为是**带宽最优 (Bandwidth Optimal)**的。现代的 pytorch 的 DDP 并行的底层计算也是靠 Ring 算法。

10.3. DDP

参考资料 10.3

DDP 原理 https://blog.csdn.net/my_name_is_learn/article/details/146468992

桶排序算法 <https://www.runoob.com/w3cnote/bucket-sort.html>

10.3.1. DDP 和 DP 的区别

DDP(Distribute Data Parallel) 是在 DP(Data Paralle) 的基础上的进一步优化。两者的主要特性的对比:

特性	DP (DataParallel)	DDP (DistributedDataParallel)
底层实现	单进程多线程	多进程 (通常一个 GPU 一个进程)
模型复制	每个前向传播, 主 GPU 上的模型会被复制到所有辅助 GPU。	每个进程维护一个独立的模型副本。
数据处理	主 GPU 将输入数据分割, 并分发给所有 GPU。	每个进程独立地加载数据, 或从数据加载器获取其应处理的数据分片。
前向、反向传播	每次传播前, 主 GPU 将模型分给其他 GPU, 计算完了之后其他 GPU 再把结果返回给主 GPU, 不保存模型。	每个 GPU 都有保有独立的模型, 独立进行计算。
计算负载	不均衡 。主 GPU 承担额外的任务 (如收集输出、计算 Loss、计算梯度、更新参数)。	均衡 。每个进程独立完成前向、反向传播和参数更新。
梯度同步	反向传播在主 GPU 上完成, 主 GPU 更新模型。	每个进程计算本地梯度, 通过 All-Reduce 在所有进程间同步并平均梯度。
通信效率	效率较低, 模型和数据在主 GPU 和辅助 GPU 之间频繁传输。每个前向传播都需要复制模型。	效率较高, 只在反向传播时同步梯度 (相对较小的数据量)。利用高效的通信库。
GIL 影响	受 Python 全局解释器锁 (GIL) 的潜在影响。	由于是多进程, 不易受 GIL 影响。
适用场景	单机多卡, 代码简单, 快速验证小模型。	单机多卡、多机多卡, 大规模模型训练, 追求高性能和更好的可伸缩性。
pytorch 代码	<code>torch.nn.DataParallel</code>	<code>torch.nn.parallel.DistributedDataParallel</code>

DDP 的工作流程可以概括为以下几个步骤:

1. 初始化

- **多进程启动**:DDP 采用多进程模型, 为每个 GPU 分配一个独立的进程。这与 Python 的**全局解释器锁 (GIL)**完美契合, 因为不同进程拥有独立的内存空间和 Python 解释器, 可以实现真正的并行计算。
- **进程组建立**: 通过 ‘torch.distributed.ini_process_group’ 函数初始化一个进程组 (Process Group), 组内的所有进程共同参与训练。每个进程被分配一个唯一的排名 (rank), 从 0 到 N-1(N 是总 GPU 数)。通常,rank 0 被指定为主进程 (master)。

- **模型复制**: 使用广播集合通信操作 (broadcast) 将模型参数从 rank 0 的设备发送到所有其他设备。在训练开始时, 每个设备都持有相同的模型参数和优化器状态副本 (例如 Adam 优化器中累积的梯度统计量)。

2. 数据分发

- **数据分发**: 给定一个包含 n 个样本的批次时, 该批次会被分片处理, 每个设备接收 n/d 个互不重叠的样本 (其中 d 是用于数据并行训练的设备数量)。 n 必须能被 d 整除, 因为训练时间受限于最慢进程的速度 (如果不能整除, 那么就会有某一/几个设备的数量大于其他设备, 导致不同设备之间时间不一致, 产生**木桶效应**)

3. 计算与梯度同步 (DDP 的精髓)

- **前向传播**: 每个进程独立地在其分配到的数据批次上执行模型的前向传播, 计算损失。
- **反向传播与梯度同步**: 当调用 `loss.backward()` 时, 梯度从模型的输出层向输入层逐层计算 (这里会有**梯度分桶**、**计算通信重叠**等优化操作, 后面会细说), 每个反向传播进程计算得到的梯度都会保存。
- **All-Reduce 操作**: 这是去中心化的梯度同步算法。在 All-Reduce 操作中, 每个进程都会贡献出自己计算出的梯度, 并最终从操作中获得所有进程梯度的总和 (或平均值)。最常用的高效实现是**Ring-AllReduce**。

4. 模型更新

- **同步更新**: All-Reduce 操作完成后, 每个进程都拥有了完全相同的平均梯度。
- **本地更新**: 每个进程独立调用优化器 (如 `optimizer.step()`) 来更新其本地模型副本的权重。因为初始权重相同, 更新的梯度也相同, 所以更新后的模型权重在所有进程间依然保持严格一致。

10.3.2. DDP 算法优化

主要涉及了两种重要的方法,**梯度分桶**和**计算通信重叠**。

之前朴素实现 DDP 的性能瓶颈:

1. **通信开销大**: 它为模型中的每一个参数张量都单独执行一次 `all-reduce` (全局规约) 通信操作。频繁的通信调用会带来显著的性能开销。
2. **通信与计算未重叠**: 它需要等待整个反向传播过程计算完所有梯度后, 才开始进行通信, 这浪费了宝贵的计算资源和时间。

针对第一个瓶颈, 容易想到为了避免用循环每次都要全规约进行通信, 我们可以将循环里的所有的梯度张量先存起来, 等循环结束后再一起发送。这样做的好处是减少了通信的开销, 但缺点是没能充分发挥**“并行”**和**“异步”**的力量。

具体而言, 由于反向传播不止计算一个参数的梯度, 我们完全可以等一个参数梯度计算好了之后, 让它**立刻异步执行 all_reduce**, 然后再去算下一个参数的梯度, 这样类似流水线的做法充分发挥了**“并行”**的能力, 理论上更省时间。

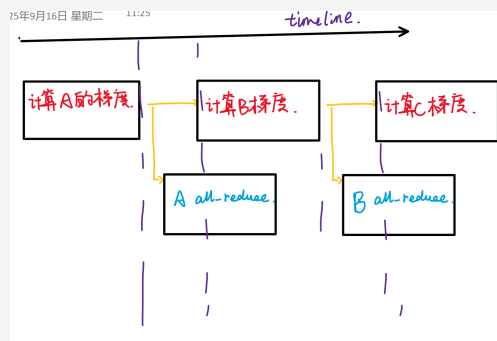


图 10.14: 计算通信重叠

在具体实现里面, 反向传播的代码 `loss.backward()` 比较固定, 但 `pytorch` 也配备了 `hook`(钩子) 操作。钩子是一种允许你“挂载”自定义代码到现有程序执行流程中的一种机制。它使得你可以在不修改程序或框架源代码的情况下, 对程序的行为进行监视、修改或增强。它本质上是一种回调 (`callback`) 思想的体现: 你预先注册一个函数, 然后由系统在某个特定事件发生时“回头调用”它。

因此, 我们可以在反向传播完一个参数之后利用 `hook` 操作异步进行 `all_reduce`, 从而实现计算和通信的流水线重叠。

```
for param in reversed(list(self.module.parameters())): # 这里由于是反向传播, 梯度从后往前计
    ↪ 算, 所以加一个 reverse
    if param.requires_grad:
        param.register_post_accumulate_grad_hook(self._create_hook(param))
```

接下来介绍分桶排序优化的方法, 它更好结合了两种方法缓解两个瓶颈。

相比于之前的每一个参数一计算完就进行通信的方式, 利用桶排序对计算好的参数分批进行通信的方式能显著减少通信次数, 从而减少网络压力, 而且利用桶的方式有更好地灵活性。比方说某个任务一定要追求极低的时延, 那就可以把桶设置成一个桶只装一个参数的形式, 这就变成了之前的这种形式, 如果某个任务所处的网络环境不好, 不方便经常通信, 那就把桶设置的大一些, 这样就减少了通信次数。

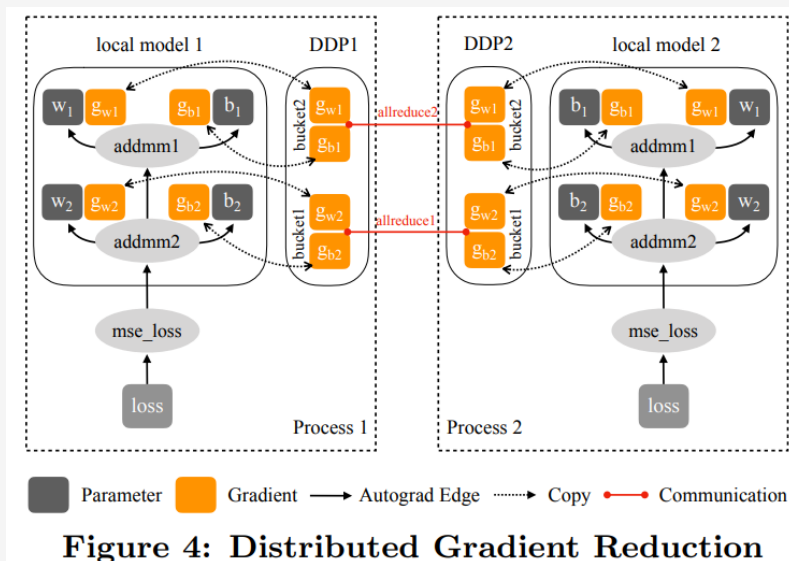


图 10.15: 桶排序流程

下面介绍整体的流程:

1. 初始化阶段: 桶的构建 (Bucket Construction)

- **参数排序**: DDP 首先会拿到模型的所有可训练参数 (`model.parameters()`)。它会按照这些参数在反向传播中梯度被计算的**逆序**来排列它们。(逆序至关重要, 因为反向传播是从模型的最后一层开始, 逐层向前计算梯度的。因此, 排在最前面的参数 (例如模型最后一层全连接层的权重和偏置) 是反向传播时最先准备好梯度的。)
- **参数入桶**: DDP 会遍历这个逆序的参数列表, 依次将参数装入桶中。每个桶有一个预设的**容量上限**, 由参数 `'bucket_cap_mb'` 控制 (默认为 25MB)。DDP 会持续向当前桶中添加参数, 直到加入下一个参数会导致桶的总大小超过容量上限为止。此时, 当前桶构建完毕, DDP 会新建一个空桶, 并将刚才那个放不下的参数作为新桶的第一个成员, 然后继续该过程, 直到所有参数都被分配到桶中。
- **结果**: 初始化完成后, DDP 内部就维护了一个桶的列表, 每个桶里包含了一组参数。例如, 对于一个典型的 Transformer 模型, 最后一个 `'Linear'` 层和 `'LayerNorm'` 层的参数可能在同一个桶里 (桶 0), 倒数第二个注意力块的参数可能在桶 1, 以此类推。

2. 反向传播阶段: Autograd Hook 的触发

- **注册钩子 (Registering Hooks)**: 在初始化阶段, DDP 会遍历模型的所有参数, 并为每一个参数的梯度 (`'grad'` 属性) 注册一个回调函数, 即 `'autograd'` 钩子。
- **触发机制**: 当你调用 `'loss.backward()'` 时, PyTorch 的 `'autograd'` 引擎会负责计算每个参数的梯度。每当一个参数的梯度被完全计算出来后, `'autograd'` 引

擎会**立即自动调用**之前注册在它上面的那个钩子函数。

3. 核心逻辑: 标记、检查与异步通信

- **标记就绪**: 钩子函数首先会找到该梯度所属的桶, 并在桶内部的计数器或状态中, 将这个梯度标记为“已就绪”(Ready)。
- **检查桶满**: 接下来, 钩子会检查这个桶内是否所有参数的梯度都已处于“已就绪”状态。
- **触发通信**:
 - 如果桶内还有其他参数的梯度没算好, 钩子函数不做任何事, 直接返回。继续进行下一轮的梯度计算。
 - 如果桶内所有参数的梯度都算好了, DDP 会执行以下操作:
 - **梯度合并 (Flatten)**: 为了提高通信效率, DDP 会将这个桶里所有零散的梯度张量复制并拼接成一个巨大、扁平且在内存上连续的单一张量。这被称为‘flatten’操作, 因为单次传输一个大 Tensor 远比多次传输多个小 Tensor 要快。
 - **启动异步 All-Reduce**: DDP 会立即对这个合并后的大 Tensor 调用‘torch.distributed.all_reduce(..., async_op=True)’。告诉 NCCL 开启异步通信, 这边不阻塞继续下一个桶的梯度计算。

4. 等待与梯度回写

- **等待与同步**: 在整个‘loss.backward()’的末尾, DDP 会确保所有桶的 All-Reduce 操作都已完成。它通常只需要等待最后一个被触发的桶的通信完成即可, 因为前面的通信已经和计算重叠了。
- **梯度解包与回写 (Unflatten)**: 当一个桶的 All-Reduce 操作完成后, 其对应的扁平化 Tensor 中存储的是该桶所有梯度拼接后的结果。DDP 会将其除以分布式环境的‘world_size’(即 GPU 总数)来得到平均梯度。然后, 再将这个扁平 Tensor 中的值“解包”, 按顺序写回到该桶内每个原始参数的‘grad’属性中。
- **优化器更新**: 至此, 所有参数的‘grad’属性都包含了同步后的全局平均梯度, 优化器(‘optimizer.step()’)便可以安全地使用它们来更新模型权重了。

10.4. 混合并行 (4D 并行)

参考资料 10.4

DeepSeed 原文 <https://www.microsoft.com/en-us/research/blog/deepspeed-extreme-scale-model-training-for-everyone/>

DeepSeed 文档 <https://www.deepspeed.ai/tutorials/pipeline/>

混合并行教程 https://blog.csdn.net/qq_25295605/article/details/143671968

混合并行教程 <https://huggingface.co/docs/transformers/v4.15.0/en/parallelism>

张量模型并行 <https://zhuanlan.zhihu.com/p/622212228>

混合并行技术是指同时使用多种并行技术，比如数据并行 (DP) 和张量并行 (TP)，或者数据并行和流水线并行 (PP)。

10.4.1. 每种并行技术简介

数据并行 (Data Parallelism, DP)

最常见的一种并行方式。它将同一个模型完整地复制到多块 GPU 上。然后，将一大批训练数据切分成多份小数据，每块 GPU 拿到一份小数据独立进行计算。在每个训练步骤结束时，所有 GPU 会同步一次计算结果 (通常是梯度)，以确保所有模型副本的参数保持一致。

DP 工作流程:

1. 数据切分:

- 在一个训练循环中，你从数据加载器中取出一个大的 ‘batch’。
- 主 GPU (通常是 ‘cuda:0’) 会将这个大 ‘batch’ 沿着批次维度进行切分，分成 N 个子批次，其中 N 是参与训练的 GPU 数量。
- 然后，主 GPU 将这些子批次分发到其他各个 GPU 上 (包括它自己)。

2. 模型复制:

- 在每次前向传播开始之前，主 GPU (‘cuda:0’) 上的最新模型参数会被复制并广播 (broadcast) 到所有其他的从属 GPU 上，确保每个 GPU 都使用完全相同的模型。

3. 并行计算:

- 每个 GPU 拿到自己的子批次数据和复制过来的模型。
- 所有 GPU **并行地、独立地** 执行前向传播，计算出各自的损失。
- 接着，每个 GPU 根据自己的损失，并行地、独立地执行后向传播，计算出模型参数的梯度。

4. 梯度规约 (Gather & Reduce):

- 所有从属 GPU 将它们计算出的梯度全部发送回主 GPU ('cuda:0')。
- 主 GPU 负责收集所有 GPU 传来的梯度, 并将它们进行 **Reduce(取平均)** 操作。

5. 模型参数更新:

- 主 GPU 使用规约后的总梯度, 通过优化器来更新其自身的模型参数。
- 请注意: **只有主 GPU 上的模型参数得到了更新**。其他 GPU 上的模型在此刻还是旧的。

6. 模型广播 (Broadcast):

- 为了开始下一次迭代, 主 GPU 会将刚刚更新完毕的、最新的模型参数再次广播给所有从属 GPU。
- 循环回到第 1 步, 处理下一个 'batch' 的数据。

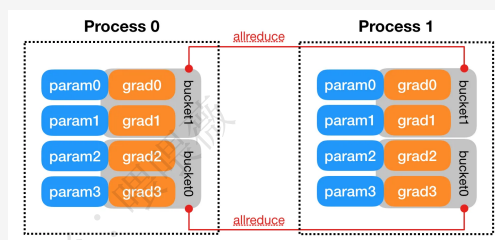


图 10.16: DP 并行示意图

张量并行 (TensorParallel, TP):

这种方式用于解决**模型本身过大、单块 GPU 无法容纳**的问题。它不对模型进行复制, 而是将模型中的某个大张量 (例如一个巨大的权重矩阵) 切分成多个小块 (Shard), 每一块分别存放在不同的 GPU 上。计算时, 每块 GPU 只处理自己负责的那一小块张量, 最后再将结果同步汇总。

举个例子, 对于一个神经网络计算单元: $Y = X * W$

- 'X' 是输入激活 (Input Activation)
- 'W' 是权重矩阵 (Weight Matrix)
- 'Y' 是输出激活 (Output Activation)

如果权重矩阵 'A' 太大, 无法放入单个 GPU, 我们就需要将 'A' 切分到多个 GPU 上, 并设计一种方法让这些 GPU 协同完成 $X * W$ 这个运算。

张量并行的具体实现方式:

对于张量的切分也是有讲究的, 如果随便切分很难实现 $X * W$ 的同步运算, 因此 'W' 的切分往往是按照行或者按照列

列并行 (Column Parallelism):

我们将权重矩阵 'W' **按列**切分。假设我们有 2 个 GPU:

$$W = [W_1 | W_2]$$

其中 ‘W₁’ 和 ‘W₂’ 分别是 ‘W’ 的左半部分和右半部分, 它们被分发到 GPU 1 和 GPU 2。

1. 前向传播 (Forward Pass):

输入 ‘X’ 被复制到两个 GPU 上。GPU 1 计算 ‘Y₁ = X * W₁’, GPU 2 计算 ‘Y₂ = X * W₂’。计算完成后, 我们将结果拼接起来: ‘Y = [Y₁ | Y₂]’。注意, 在这个过程中, ‘Y’ 的不同部分自然地分布在不同的 GPU 上。

2. 通信: 在这一步的前向传播中, **不需要**任何通信。每个 GPU 独立计算自己那一部分。

3. 反向传播 (Backward Pass):

- 在计算输入的梯度 ‘dL/dX’ 时, 根据链式法则, 它等于 ‘(dL/dY) * W^T’。
- 由于 ‘Y’ 和 ‘W’ 都是分片的, ‘dL/dX’ 的计算会变成 ‘(dL/dY₁) * W₁^T + (dL/dY₂) * W₂^T’。
- GPU 1 计算 ‘(dL/dY₁) * W₁^T’, GPU 2 计算 ‘(dL/dY₂) * W₂^T’。
- 为了得到最终完整的 ‘dL/dX’, 两个 GPU 需要将各自计算出的梯度相加。

4. 通信: 这里需要一次 **All-Reduce** 操作, 将所有 GPU 上的梯度累加, 并同步回每个 GPU。

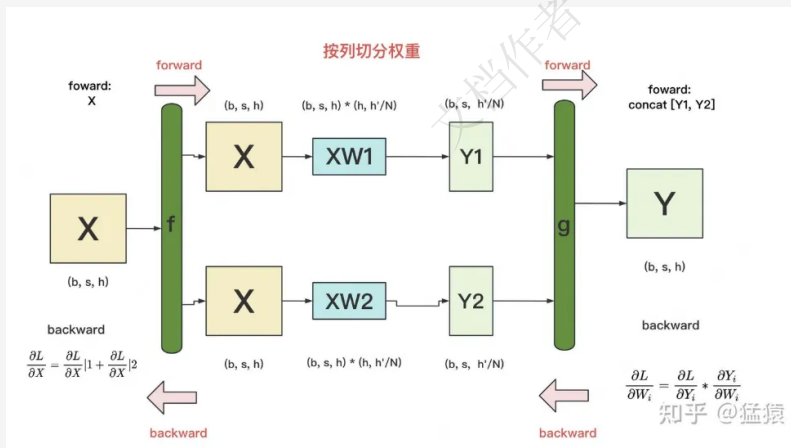


图 10.17: 列并行示意图

行并行 (Row Parallelism):

现在, 我们将权重矩阵 ‘W’ **按行**切分:

‘W = [W₁; W₂]’ (这里用分号表示按行堆叠)

‘W₁’ 和 ‘W₂’ 分别是 ‘W’ 的上半部分和下半部分, 被分发到 GPU 1 和 GPU 2。

1. 前向传播 (Forward Pass):

此时, 输入 ‘X’ 必须是已经被分片的 (例如, 它是上一个列并行层的输出)。假设 ‘X = [X₁ | X₂]’。

完整的计算是 $Y = X * W = [X_1 | X_2] * [W_1; W_2] = X_1 * W_1 + X_2 * W_2$ 。GPU 1 计算 $Y_1 = X_1 * W_1$ 。GPU 2 计算 $Y_2 = X_2 * W_2$ 。为了得到最终的 Y ，我们需要将 Y_1 和 Y_2 相加。

2. **通信**: 这里需要一次 **All-Reduce 操作**, 将各个 GPU 上的部分结果相加, 得到最终的完整输出 Y 。
3. **反向传播 (Backward Pass)**: 在计算输入的梯度 dL/dX 时, 它的一部分 dL/dX_1 等于 $(dL/dY) * W_1^T$, 这可以在 GPU 1 上独立完成。
4. **通信**: 在这一步的反向传播中, **不需要** All-Reduce 通信。

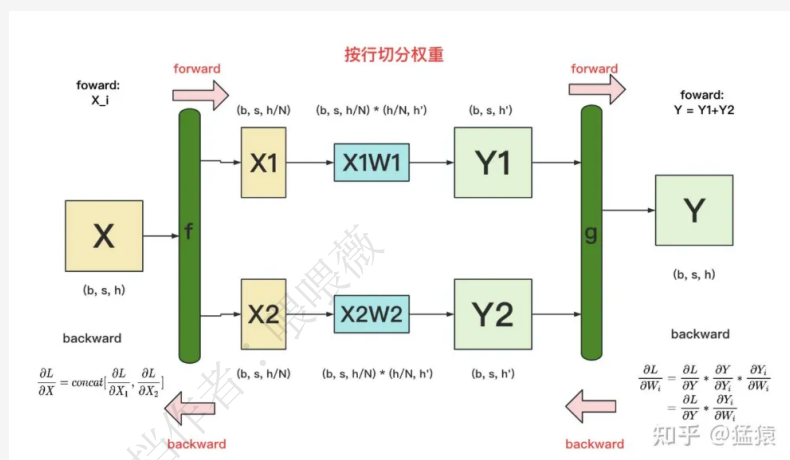


图 10.18: 行并行示意图

行列结合:

以 FFN 为例, 它通常由两个线性层和一个激活函数组成: $Y = \text{GELU}(X * A) * B$

1. **第一个线性层 ($X * A$):** 使用列并行。

- A 按列切分 $[A_1, A_2]$ 。
- 输出 $\text{GELU}(X * A)$ 也是按列切分的 $[Y_1, Y_2]$, 分布在不同 GPU 上。
- 前向传播**无通信**。反向传播需要一次 **All-Reduce**。

2. **第二个线性层 ($\dots * B$):** 使用行并行。

- B 按行切分 $[B_1; B_2]$ 。
- 它的输入 $[Y_1, Y_2]$ 恰好是上一步的分布式输出。
- 前向传播需要一次 **All-Reduce** 来合并最终结果。反向传播**无通信**

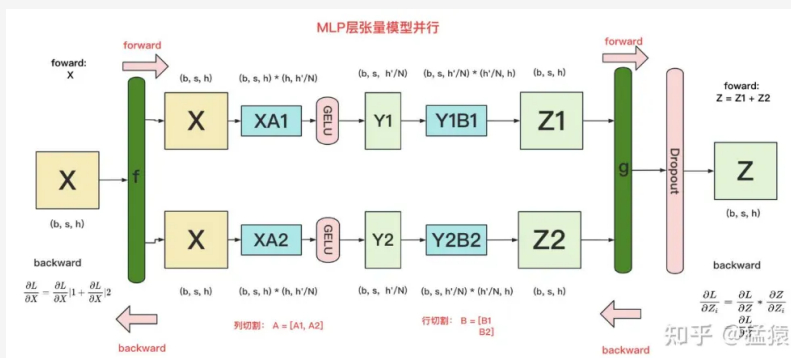


图 10.19: 行列结合示意图

流水线并行 (PipelineParallel, PP)

这种方式也是为了解决模型过大的问题, 但它的切分维度不同。它将整个模型**按层 (Layer) 进行切分**, 类似于工厂里的流水线。例如, 一个 30 层的模型, 可以把前 10 层放在 GPU 1 上, 中间 10 层放在 GPU 2 上, 最后 10 层放在 GPU 3 上。数据像在水流线上传送带一样, 依次流过这几块 GPU, 完成一次完整的计算。

基本原理:

假设我们将一个模型切分成 4 个部分, 分别放在 GPU 0, 1, 2, 3 上。

前向传播 (Forward Pass):

1. ‘Batch 1’ 进入 ‘GPU 0’ 进行计算。
2. ‘GPU 0’ 计算完成后, 将其输出 (称为激活值) 发送给 ‘GPU 1’。
3. ‘GPU 1’ 收到后开始计算, 计算完成后将激活值发送给 ‘GPU 2’。
4. ... 以此类推, 直到 ‘GPU 3’ 完成计算。

后向传播 (Backward Pass):

1. ‘GPU 3’ 首先计算梯度。
2. ‘GPU 3’ 将其计算的梯度发送回 ‘GPU 2’。
3. ‘GPU 2’ 接收到梯度后, 计算自己的梯度, 再发送回 ‘GPU 1’。
4. ... 以此类推, 直到 ‘GPU 0’ 完成梯度计算。

Keynote 10.2

流水线并行的缺陷——巨大的“气泡” (Bubble)

在上述流程中, 当 ‘GPU 1’ 在工作时, ‘GPU 0’ 在做什么? 它在空闲等待。当 ‘GPU 2’ 工作时, ‘GPU 0’ 和 ‘GPU 1’ 都在空闲等待。在任何一个时间点, 只有一个 GPU 在忙碌, 其他的 GPU 都在闲置。这种 GPU 利用率的巨大浪费, 我们称之为流水线气泡 (Pipeline Bubble)。

优化: 微批次流水线

为了解决“气泡”问题，现代流水线并行引入了一项关键技术：**将一个大的训练批次 (mini-batch) 进一步切分成若干个更小的微批次 (micro-batch)**。

通过这种方式，虽然在开始和结束阶段仍有无法避免的“气泡”，但中间大部分时间 GPU 的利用率被极大地提高了。微批次数量越多，稳态阶段占比就越大，“气泡”的相对开销就越小。

梯度累积：在后向传播阶段，每个 GPU 会计算出自己所负责模型部分的梯度。它会累积所有微批次的梯度，直到整个 mini-batch 处理完毕，然后用累积的总梯度进行一次参数更新

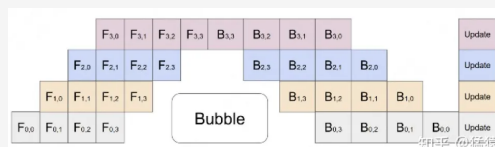


图 10.20: 流水线并行示意图

ZeRO (Zero Redundancy Optimizer)

这是一种**旨在消除数据并行中的冗余、极致节省显存**的技术。它类似于张量并行，也会**对模型的参数、梯度和优化器状态进行分片**。但它的核心特点是：只有在计算需要时，才会动态地将完整的张量临时重建出来，计算一完成就立即丢弃，从而大大降低了显存峰值。它还支持将暂时用不到的数据“卸载 (Offload)”到 CPU 内存或硬盘上，进一步节省宝贵的 GPU 显存。

DDP 的内存冗余问题

在标准的数据并行训练中，每个 GPU 都需要存储三份主要的模型相关数据：

1. **模型参数 (Model Parameters)**：模型的权重，例如 ‘Float16’ 或 ‘Float32’。
2. **梯度 (Gradients)**：反向传播计算出的参数梯度，大小与参数相同。
3. **优化器状态 (Optimizer States)**：例如 Adam 优化器需要为每个参数存储其一阶矩 (**Momentum**) 和二阶矩 (**Variance**)，通常是参数数量的 2 倍。

假设模型参数数量为 M，使用 Adam 优化器和 FP16 混合精度训练，那么在 ‘N’ 个 GPU 上进行数据并行时，每个 GPU 的内存占用大约是：

‘Memory_per_GPU ≈ (2M for FP16 Params) + (2M for FP16 Grads) + (4M for FP32 Momentum) + (4M for FP32 Variance) = 12M’ (这还不包括激活值等其他开销)

总的内存占用是 ‘N * 12M’。你会发现，模型参数、梯度和优化器状态在所有 ‘N’ 个 GPU 上都存在一份完整的副本，这是巨大的**内存冗余**。ZeRO 的使命就是消除这种冗余。

ZeRO 的核心思想：分区 (Partitioning)

ZeRO 通过将这三部分数据 (P, G, O) 在所有参与数据并行的 GPU 之间进行**分区**，**而不是复制**，来解决内存冗余问题。每个 GPU 只负责存储和更新完整数据的一个分片 (Shard)。

ZeRO 分三个递进的阶段来实现这一目标, 每个阶段优化的程度都比前一个更深。

Stage 1: 优化器状态分区 (Partition Optimizer States)

优化器状态通常是内存的大头 (在 Adam 中是参数量的 2 倍, 且通常用 FP32 存储, 总计 8M)。这是最容易优化的部分。

工作流程:

1. **分区**: 将优化器状态 (如 Momentum 和 Variance) 平均切分成 ‘N’ 份, 每个 GPU 只保存其中的 ‘1/N’。
2. **保留**: 每个 GPU 依然保留完整的模型参数和梯度。
3. **更新过程** (`optimizer.step()`):
 - 在反向传播结束后, 所有 GPU 上的梯度会通过一次 ‘All-Reduce’ 操作进行同步, 确保每个 GPU 都有完整的、求和后的梯度。
 - 在参数更新时, 每个 GPU 只使用它本地存储的那一小部分优化器状态, 来更新对应的那一小部分模型参数。
 - 更新完成后, 每个 GPU 只拥有了模型参数的一部分更新。
 - 最后, 通过一次 ‘All-gather’ 通信操作, 所有 GPU 广播自己更新好的那部分参数, 从而在每个 GPU 上重新组合出完整的、更新后的模型。

效果:

- **内存节省**: 大幅减少了优化器状态的内存占用。
- **通信开销**: 与标准 DDP 相比, 增加了一次 ‘All-gather’ 操作来同步更新后的参数。

Stage 2: 梯度和优化器状态分区 (Partition Gradients & Optimizer States)

在 Stage 1 的基础上, 进一步消除梯度的冗余。

工作流程:

1. **分区**: 将优化器状态**和梯度**都进行 ‘N’ 路分区。每个 GPU 只负责 ‘1/N’ 的梯度和优化器状态。
2. **保留**: 每个 GPU 依然保留完整的模型参数。
3. **更新过程**:
 - 在反向传播过程中, 当梯度计算出来后, 不再执行 ‘All-Reduce’。取而代之的是一个更高效的 ‘Reduce-Scatter’ 操作。这个操作会一边计算梯度总和, 一边将结果直接分发给对应的 GPU。例如, 属于分区 1 的梯度会被计算总和后直接发送到 GPU 1, 以此类推。
 - 这样, 反向传播结束后, 每个 GPU 上只存储了它所负责的那部分梯度总和。
 - 后续的优化器更新和参数 ‘All-gather’ 过程与 Stage 1 类似。

效果:

- **内存节省**: 进一步节省了梯度存储。
- **通信效率**: 将 DDP 中的 ‘All-Reduce’ 替换为 ‘Reduce-Scatter’, 通常通信量减半, 效率更高。

Stage 3: 参数、梯度和优化器状态全部分区 (Partition Everything)

这是最极致的优化, 消除了所有冗余, 包括模型参数本身。

工作流程:

1. **分区**: 将模型参数、梯度、优化器状态**全部**进行 ‘N’ 路分区。
2. **保留**: 原则上, 每个 GPU 在任何时刻都只拥有模型的一个分片。
3. **计算过程 (前向/后向)**:
 - 当一个 GPU 需要执行某一层的前向或后向计算时, 如果该层所需的参数不在本地, 它需要通过一次 ‘All-gather’ 操作从其他 GPU 那里动态地获取这些参数。
 - 计算完成后, 为了节省内存, 这些临时的、非本地的参数可以被立即丢弃。
 - 这个过程需要在每一层计算前后动态地进行, 对通信的要求非常高。

效果:

- **内存节省**: 达到理论最优。理论上可以将一个巨大的模型 (如 1 万亿参数) 分散到足够多的 GPU 上进行训练。
- **通信开销**: 通信量巨大, 因为它在计算过程中穿插了大量细粒度的 ‘All-gather’ 操作。

ZeRO 的扩展: ZeRO-Offload & ZeRO-Infinity

为了应对 GPU 内存仍然不足或通信成本过高的情况, ZeRO 还发展出了更高级的形态:

- **ZeRO-Offload**: 将 ZeRO 分区后的数据 (特别是那些不常被访问的, 如优化器状态) 进一步从 GPU 内存**卸载 (Offload)**到 CPU 内存。这极大地扩展了可训练模型的大小, 代价是引入了 GPU 和 CPU 之间的 PCIe 通信延迟, 速度会慢一些。
- **ZeRO-Infinity**: ZeRO-Offload 的终极版本, 不仅利用 CPU 内存, 还利用 NVMe 固态硬盘作为海量的内存池。这使得在有限的硬件上训练万亿级参数模型成为可能, 尽管训练时间会非常长。

10.5. 混合数据并行

10.5.1. DP+PP(数据并行 + 流水线并行)

为什么需要 DP+PP ?

- **单独使用数据并行 (DP)**: 要求**每个 GPU**都能装下完整的模型。当模型增长到百亿甚至千亿参数时, 例如一个 175B 的 GPT-3 模型 (FP16 下需要 350GB 内存), 远超任何单个 GPU 的容量, DP 就失效了。
- **单独使用流水线并行 (PP)**: 解决了模型过大的问题, 可以将模型切分到多个 GPU 上。但它的扩展性受限于模型的层数和“**流水线气泡**”的开销。如果你只有 60 层模型, 使用超过 60 个 GPU 做流水线并行是没有意义的, 而且气泡开销会变得极其巨大。它无法利用更多的机器来处理更多的数据。

DP+PP 的核心思想: 用 PP 来解决单个模型过大的问题, 用 DP 来将这个“切分后的大模型”复制多份, 以扩展到更多的计算设备上, 实现规模化加速。

DP+PP 的工作流程

1. 数据分发:

一个大的 mini-batch 被分成 ‘data_parallel_size’(例如 4) 份, 我们称之为 ‘D1, D2, D3, D4’。

- **第一条流水线 (节点 0)** 获得数据 ‘D1’。
- **第二条流水线 (节点 1)** 获得数据 ‘D2’。
- 以此类推。

2. 前向传播 & 后向传播:

所有流水线并行地、独立地开始工作。

- 在第一条流水线内部, 数据 ‘D1’ 被切分成微批次, 按照 PP 流水线并行的方式, 流过 ‘GPU0 -> GPU1 -> GPU2 -> GPU3’, 并完成前向和后向传播。
- 同时, 在第二条流水线内部, 数据 ‘D2’ 也以同样的方式流过它的四个 GPU。
- 这个阶段, 不同流水线之间**没有任何通信**。

3. 梯度同步 (关键步骤):

- 当一条流水线完成了一个完整 mini-batch(例如 ‘D1’) 的后向传播后, 其上的每个 GPU 都计算出了对应模型分片的**本地梯度**。
- 此时,**数据并行**的同步机制启动。
- **DP Group 0**(包含节点 0 的 GPU0, 节点 1 的 GPU0,) 中的所有 GPU 执行一次 ‘All-Reduce’ 操作, 将它们各自计算出的关于模型第一部分的梯度进行平均。
- **DP Group 1**(包含所有 GPU1) 也执行 ‘All-Reduce’, 同步模型第二部分的梯度。

- 以此类推。

这个 ‘All-Reduce’ 操作是在不同节点之间进行的, 通常通过高速网络 (如 InfiniBand) 完成。

4. 参数更新:

- 梯度同步完成后, 每个 GPU 上都有了**全局平均梯度**。
- 每个 GPU 使用这个梯度来更新它本地存储的那部分模型参数。

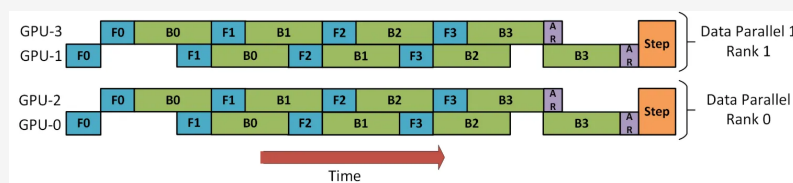


图 10.21: DP+PP 示意图

10.5.2. 3D 并行 (DP+PP+TP)

为什么需要 3D 并行？2D 并行的瓶颈

在我们讨论 DP+PP 时, 我们已经解决了一个核心问题: 如何训练一个**大到单个节点都装不下的模型**, 并将其扩展到多个节点。但这还不够, 因为 DP+PP 组合本身也存在瓶颈:

1. **流水线气泡依然存在**: 即使结合了 DP, 每条流水线 (PP) 内部的“气泡”开销依然是其固有效率损失。为了减少气泡, 我们需要增加微批次数量, 但这会影响模型的收敛性 (小批量噪声问题) 和硬件效率 (小计算量的 GEMM 操作效率低)。
2. **单层计算/内存瓶颈**: 想象一个流水线阶段 (Stage)。虽然整个模型被切分了, 但这个阶段本身可能仍然非常巨大。特别是对于某些 Transformer 层, **单个自注意力 (Self-Attention) 或前馈网络 (FFN) 层的权重矩阵和激活值**, 可能就大到无法放入单个 GPU。PP 是按层 (或层块) 切分, 它无法解决单层内部的瓶颈。

这就是**张量并行 (TP)** 发挥关键作用的地方。TP 可以在一个操作 (如矩阵乘法) 的内部进行并行计算, 从而解决单层过大的问题。

3D 并行的架构: 一个三维计算立方体

3D 并行架构可以用“立方体”结构来直观想象: 每个 GPU 都有一个 ‘(dp_rank, pp_rank, tp_rank)’ 的坐标。

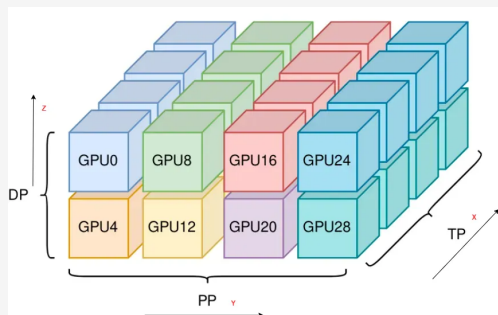


图 10.22: 3D 并行示意图

1. 维度 1: 流水线并行 (PP) - 模型的“深度”

- 它将模型的层 (layers) 沿着这个维度进行切分。
- GPU ‘(x, 0, z)’ 执行模型的第 1 个阶段, ‘(x, 1, z)’ 执行第 2 个阶段, 以此类推。
- 通信发生在 **PP 组** 内, 即 ‘pp_rank’ 不同的 GPU 之间 (例如, 从 ‘(x,0,z)’ 到 ‘(x,1,z)’)。

2. 维度 2: 张量并行 (TP) - 模型的“宽度”

- 它将模型中**每一层的计算** (特别是巨大的权重矩阵) 沿着这个维度进行切分。
- GPU ‘(0, y, z)’ 和 ‘(1, y, z)’ 协同工作, 共同完成一个层的计算。
- 通信发生在 **TP 组** 内, 即 ‘tp_rank’ 不同的 GPU 之间。这种通信非常频繁, **必须使用最高速的互联技术**, 因此 TP 通常被限制在单个节点内部。

3. 维度 3: 数据并行 (DP) - 模型的“数量”

- 它将整个 “PP+TP” 组合出的模型复制多份, 每份处理不同的数据。
- GPU ‘(x, y, 0)’ 和 ‘(x, y, 1)’ 持有完全相同的模型分片, 但处理不同的数据批次。
- 通信发生在 **DP 组** 内, 即 ‘dp_rank’ 不同的 GPU 之间。这种通信是梯度同步, 通常跨节点。

总 GPU 数量 = ‘data_parallel_size’ * ‘pipeline_parallel_size’ * ‘tensor_parallel_size’

一个具体的例子: 用 64 个 GPU 进行 3D 并行

假设我们有 8 个节点, 每个节点 4 个 GPU, 总共 32 个 GPU。我们可以这样配置:

- ‘tensor_parallel_size = 4’: 使用每个节点内的 4 个 GPU 进行张量并行。
- ‘pipeline_parallel_size = 4’: 将模型切分为 4 个流水线阶段。
- ‘data_parallel_size = 2’: 复制出 2 个数据并行的副本。

一个 GPU 的角色：让我们看看 ‘GPU9’ (全局排名第 9 的 GPU) 在做什么。

- 它位于**节点 1**上 (‘GPU8’到 ‘GPU11’在节点 1)。
- 它的**TP 组**是节点 1 上的所有 4 个 GPU (‘GPU8’到 ‘GPU11’)。它们共同负责计算模型中的每一层。
- 它的**PP 阶段**是什么？假设我们将 4 个 PP 阶段分配给 4 组节点，那么节点 0 和 1 可能构成一个数据并行副本，节点 2 和 3 构成另一个。那么 ‘GPU9’位于 PP 阶段 2。
- 它的**DP 组**是哪个？它会和另一个数据副本中处于相同 TP 和 PP 位置的 GPU 构成 DP 组。

工作流程一览：

1. **输入数据 ‘D’**被分成两部分 ‘D1’ 和 ‘D2’，分别送往两个数据并行副本。
2. **在数据副本 1 中：**
 - ‘D1’被切成微批次。
 - **阶段 0 (例如，由节点 0 的 4 个 GPU 负责)：**这 4 个 GPU 通过**张量并行**协同计算模型的前 $\frac{1}{4}$ 的层。计算完成后，将激活值发送给下一阶段。
 - **阶段 1 (例如，由节点 3 的 4 个 GPU 负责)：**接收来自阶段 0 的激活值，通过**张量并行**计算模型的第二个 $\frac{1}{4}$ 的层。
 - 以此类推，完成整个流水线的前向和后向传播。
3. 在**数据副本 2**中，完全相同的过程用数据 ‘D2’并行地发生。
4. **梯度同步：**
 - 在每个副本完成后，处于相同 ‘pp_rank’和 ‘tp_rank’的 GPU 之间进行 ‘All-Reduce’。
 - 例如，节点 0 上的 ‘GPU0’与节点 1 上的 ‘GPU4’进行梯度同步。

10.5.3. 4D 并行 (DP+PP+TP+ZeRO)

为什么需要这个组合？

我们已经知道 3D 并行 (DP+PP+TP) 非常强大，但它仍然有一个根本性的“浪费”：在数据并行的维度上，不同的副本 (Data Parallel Replicas) 之间存在着**完全相同的模型状态冗余**。

例如，在一个 ‘dp_size=8, pp_size=4, tp_size=8’ 的配置中：

- 由 PP 和 TP 组合后，每个 GPU 只负责整个模型 ‘ $1/(4*8) = 1/32$ ’ 的计算和参数。
- 但是，这个 ‘ $1/32$ ’ 的模型分片，在 8 个数据并行副本中是**一模一样的**。这意味着，我们依然为参数、梯度和优化器状态多占用了 7 倍的内存。

对于一个万亿参数的模型，即使是 ‘ $1/32$ ’ 的分片也可能非常巨大。消除这最后的冗余，就是 ZeRO 在这个组合中要完成的使命。

架构和 workflow: ZeRO 如何融入 3D 并行

ZeRO 的操作完全发生在数据并行组 (DP Group) 内部。因此一样用立方体看待。

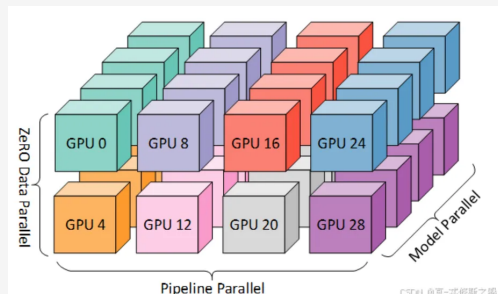


图 10.23: 4D 并行示意图

状态分区

考虑一个特定的模型块，它已经被 PP 和 TP 切分好了，位于坐标 $(pp_rank=p, tp_rank=t)$ 的位置上。这个模型块的状态（参数、梯度、优化器状态）需要被存储。在传统的 3D 并行中，所有 dp_rank 从 0 到 dp_size-1 的 GPU，在 (p, t) 这个位置上，都存储着一份完整的、相同的模型块状态。

引入 ZeRO 后：这个模型块的状态被进一步切分成 dp_size 份，分散存储在多个 DP 组中。

- GPU $(dp=0, pp=p, tp=t)$ 只存储这份状态的第 1 片。
- GPU $(dp=1, pp=p, tp=t)$ 只存储这份状态的第 2 片。
- 以此类推。

训练步骤的变化

这个改变深刻地影响了训练流程，尤其是在 DP 组内的通信模式：

● 前向传播 (ZeRO-Stage 3):

1. 在一个 GPU (d, p, t) 开始计算它的模型块之前，它只拥有该模型块 $1/dp_size$ 的参数。
2. 它必须先在它的 DP 组内发起一次 'All-gather' 操作。
3. 通过这次通信，它从 DP 组的其他伙伴那里收集齐了完整的模型块参数。
4. 然后，它和它的 TP 组内的伙伴们一起，协同完成这一层的前向计算。
5. 计算完成后，为了节省内存，它可以立即丢弃刚刚收集来的那部分不属于自己的参数。

● 后向传播:

1. 与前向传播类似，在计算梯度前，需要通过 'All-gather' 确保拥有完整的参数。
2. 计算完成后，每个 GPU 都得到了它负责的模型块的部分梯度（因为 TP）。

- TP 组内部会进行一次 ‘All-Reduce’, 使得 TP 组内每个 GPU 都拥有了关于这个模型块的**完整梯度**。

● **梯度同步与分区 (取代传统 DP 的 All-Reduce):**

- 现在, DP 组内的每个 GPU 都有一份完整的梯度。
- 它们不再执行简单的 ‘All-Reduce’ 来求平均。
- 取而代之, 它们执行一次 ‘Reduce-Scatter’。这个操作会一边将所有 DP 副本的梯度相加求平均, 一边将结果**直接分区**, 并发送给对应的 GPU。
- 操作结束后, GPU ‘(d, p, t)’ 只会收到它所负责的、平均后的**梯度的一小片**。

● **参数更新:**

- 每个 GPU 使用它本地存储的一小片优化器状态和刚刚收到的一小片梯度, 来更新它本地拥有的一小片模型参数。

10.5.4. 专家并行/MOE 并行

和前面的并行几乎完全不同, 专家并行只针对推理阶段, 它的思想是只利用和输入 token 最相关的几个专家模型 (top-k) 来进行推理最后加权输出。

但模型本身会有很多个专家来“顾问”, 每次只用几个, 来这样实现所谓”并行”。

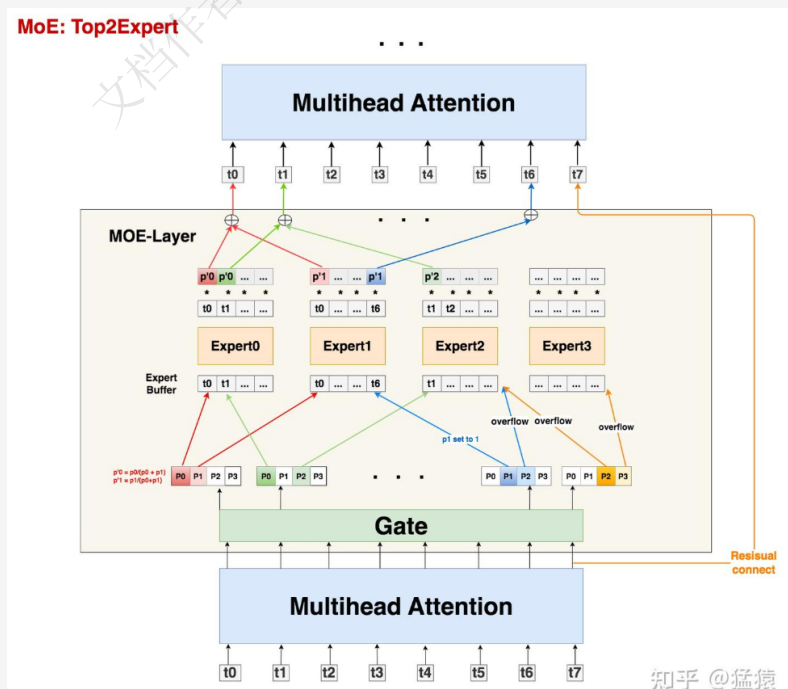


图 10.24: 专家并行示意图

10.6. CUDA 通信机制知识补充

核心原则: 异构计算

Keynote 10.3

异构计算是指结合**不同指令值和体系结构的设备 (计算单元)**之间的联合计算, 比如 *CPU*、*GPU*、*FPGA*。在 *CUDA* 编程中主要就是指 *CPU* 及内存与 *GPU* 和显存。这两者在**物理上独立, 通过 *PCIe* 总线连接通信**。

Keynote 10.4

PCIe 通信特点

- **连接方式**: *PCIe* 采用**点对点的串行连接**方式, 每个设备都有自己的**专用连接**, 可以独享带宽, 无需像 *PCI* 这种旧的并行总线那样共享同一个物理带宽。
- **拓扑结构**: *PCIe* 系统是一个**树状的层次结构**, 通常由代表 *CPU* 接口的根复合体、用于扩展端口的交换器 (*Switch*) 以及终端设备 (*Endpoint*, 如显卡、网卡、固态硬盘等) 组成。
- **分层架构**: *PCIe* 协议采用**分层架构**, 分为事务层、数据链路层和物理层。
 - **事务层**: 负责生成和解析用于数据请求和响应的事务层数据包 (*TLP*)。
 - **数据链路层**: 确保数据传输的完整性, 负责错误检测和纠正。
 - **物理层**: 负责数据的串行化传输和链路管理。

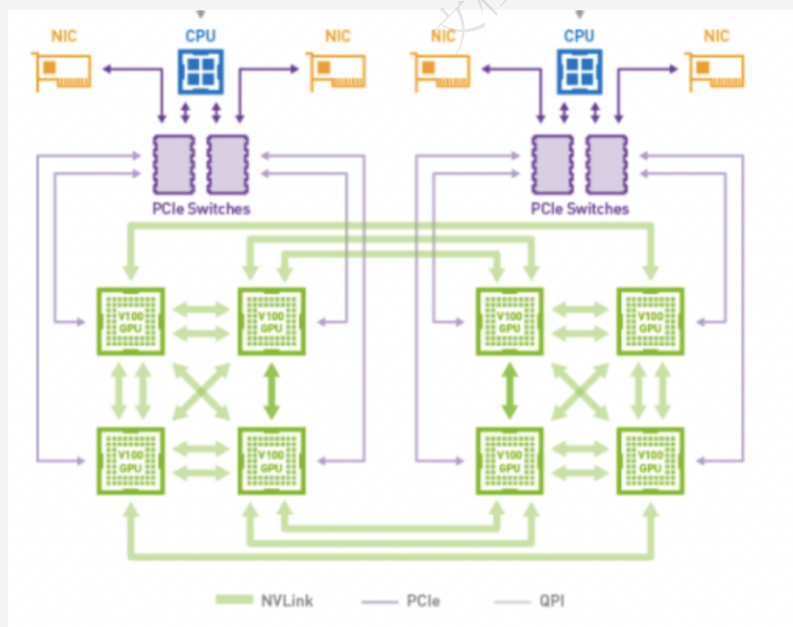


图 10.25: CUDA 通信机制示意图

CUDA 里由于 CPU 与 GPU 物理上的分离使得很有必要研究一下两者之间的通信。这也带来了 GPU 主要的性能瓶颈: 通信与计算之间的不平衡, 即**数据传输的**

速度远低于 GPU 内部的计算速度

CUDA 中的通信可以分为两大类：

1. **主机与设备 (Host-Device) 之间的通信**：宏观层面的数据传输。
2. **设备内部 (Intra-Device) 的通信**：微观层面，线程之间的数据交换。

10.6.1. 主机与设备 (Host-Device) 通信

这是最基础的数据交互。典型的流程是：

1. CPU 将数据从主存复制到显存。
2. GPU 执行计算 (Kernel Launch)。
3. CPU 将结果从显存复制回主存。

同步 (Synchronous) 内存拷贝

特性：

- **阻塞式 (Blocking)**：当 CPU 线程调用拷贝函数时，它会**暂停执行 (阻塞)**，直到数据拷贝**完全完成**，然后才会继续执行后续的 CPU 代码。
- **简单易懂**：逻辑清晰，不易出错。
- **性能瓶颈**：CPU 在等待期间完全闲置，无法与 GPU 的数据传输并行工作，浪费了宝贵的计算资源。

异步 (Asynchronous) 内存拷贝与流 (Streams)

为了克服同步拷贝的性能瓶颈，CUDA 引入了**异步拷贝**和**流 (Stream)**的概念。

流 (Stream)：可以理解为 GPU 上的一个**任务队列**。你向一个流中添加一系列操作 (如内存拷贝、核函数启动)，GPU 会按照你添加的顺序来执行这些操作。

异步拷贝：

- **非阻塞式 (Non-Blocking)**：当 CPU 线程调用异步拷贝函数时，它只是将这个“拷贝任务”**提交到指定的流**中，然后**立即返回**，继续执行后续的 CPU 代码。CPU 不会等待拷贝完成。
- **并行潜力**：这使得 CPU 可以继续准备下一个数据块、启动其他任务，或者让数据传输与 GPU 计算**重叠 (Overlap)**执行，从而极大地提升整体效率。

Keynote 10.5

关键点:

- **异步操作必须指定一个流。**
- **要使异步内存拷贝真正实现非阻塞, 主机内存必须是页锁定 (Page-Locked) 或固定 (Pinned) 内存。**因为操作系统可能会移动常规的可分页内存, 而 GPU 的 DMA 引擎需要一个固定的物理地址。使用 `'cudaMallocHost()'` 或 `'cudaHostAlloc()'` 分配页锁定内存。
- **需要显式同步。**既然 CPU 不等待, 那你如何知道 GPU 任务何时完成? 你需要使用同步函数, 如 `'cudaStreamSynchronize()'` (等待指定流完成) 或 `'cudaDeviceSynchronize()'` (等待设备上所有任务完成)。

10.6.2. 设备内部 (Intra-Device) 通信

当数据已经在 GPU 上后, 成千上万的线程需要相互协作来完成计算。它们的通信方式和范围是严格分级的。

线程块 (Block) 内部通信

这是最高效、最常用的设备内部通信方式。同一线程块内的线程可以**共享数据和进行同步**。

共享内存 (Shared Memory):

- **物理位置:** 位于 GPU 芯片上 (On-chip), 与 L1 缓存共享物理资源。
- **速度:** 访问速度**极快**, 远高于全局内存 (Global Memory), 几乎与寄存器 (Register) 访问速度相当。
- **生命周期:** 与线程块相同。当线程块开始执行时分配, 执行完毕后释放。
- **作用域:** 仅对同一线程块内的所有线程可见。块 A 的线程无法访问块 B 的共享内存。
- **声明:** 使用 `'__shared__'` 限定符在核函数内部声明。

同步原语 (`'__syncthreads__()'`):

- **功能:** 这是一个**栅栏 (Barrier)**。块内的所有线程执行到 `'__syncthreads__()'` 时必须停下来等待, 直到**块内所有线程**都到达这个点, 然后才能继续执行后面的指令。
- **必要性:** 确保数据依赖性。例如, 在线程 A 从共享内存读取数据之前, 必须确保所有向该共享内存写入数据的线程 (如线程 B、C、D) 都已经完成了写入操作。 `'__syncthreads__()'` 就是用来保证这一点的。

这是一个更复杂的问题。CUDA 的编程模型**没有提供直接、高效的块间同步机制**。不同块的执行顺序和并发性由硬件调度器决定, 程序员无法假设块 A 一定在块 B 之前完成。

因此, 块间通信通常通过**全局内存 (Global Memory)**间接实现:

1. 分步执行 (Separate Kernels):

- 这是最常用、最可靠的方法。
- 流程:
 - a. 启动一个核函数 (Kernel 1), 每个块完成其部分计算, 并将中间结果写入全局内存。
 - b. 核函数结束后, 有一个隐式的全局同步。
 - c. 启动第二个核函数 (Kernel 2), 读取 Kernel 1 产生的中间结果, 进行下一阶段的计算。
- 缺点: 多次核函数启动会带来额外的开销。

2. 原子操作 (Atomic Operations):

- 当多个块中的线程需要更新**同一个全局内存地址**时 (例如, 一个全局计数器), 为了避免数据竞争, 必须使用原子操作。
- 函数: 'atomicAdd()', 'atomicExch()', 'atomicCAS()' 等。
- 特性: 保证操作的原子性 (不会被中途打断), 但可能会因为争用而导致线程串行化, 影响性能。

3. 持久化线程 (Persistent Threads) 与网格级同步 (Grid-Level Sync) (高级)

- 对于一些复杂的应用 (如图计算、某些稀疏计算), 可以启动一个覆盖整个设备的持久化网格 (Grid), 线程在循环中不断从全局队列中获取任务。
- 较新的 CUDA 版本 (计算能力 7.x+) 引入了 **Cooperative Groups**, 提供了在整个 Grid 范围内同步的机制 ('grid.sync()')。这允许在**单个核函数内**实现全局同步, 避免了多次启动的开销。但这需要谨慎设计, 且对硬件有要求。